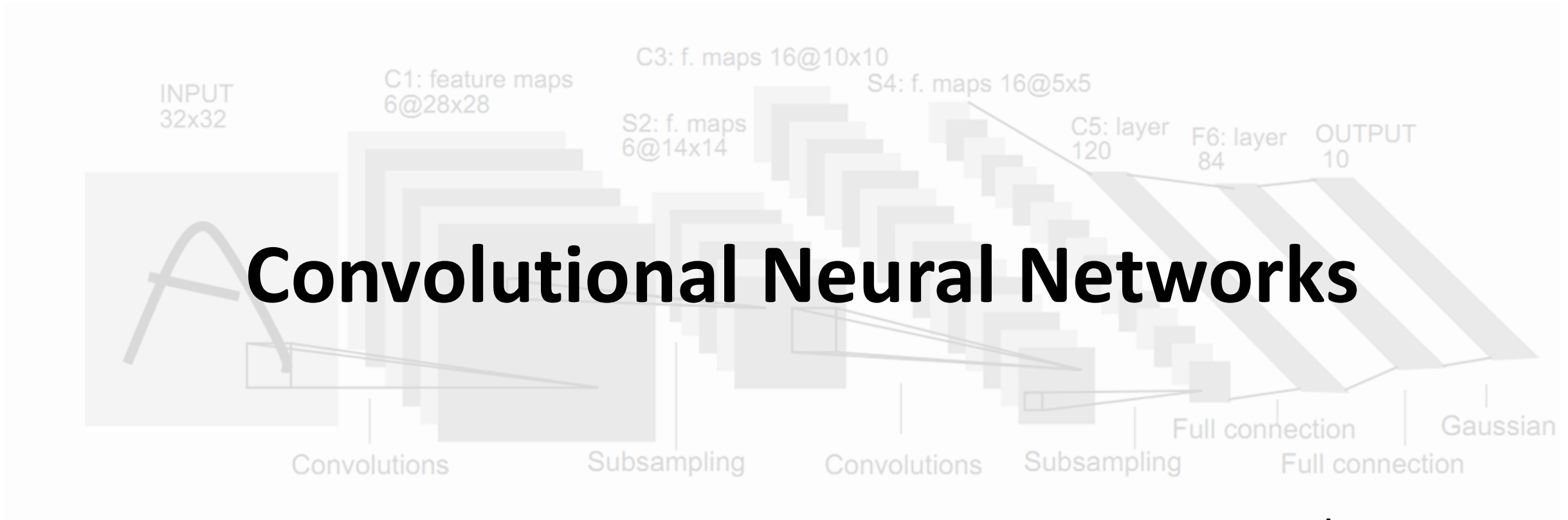


Lecture 4



Speaker:
Kaiming He

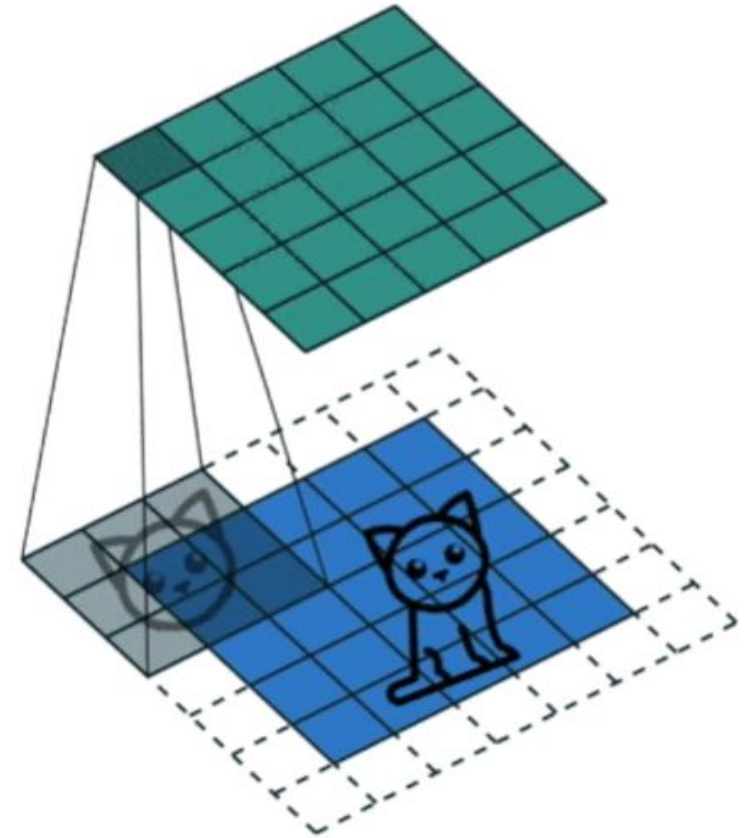
Overview

- Convolution
- Convolutional Neural Networks (ConvNet, CNN)
- Case Study

Convolution

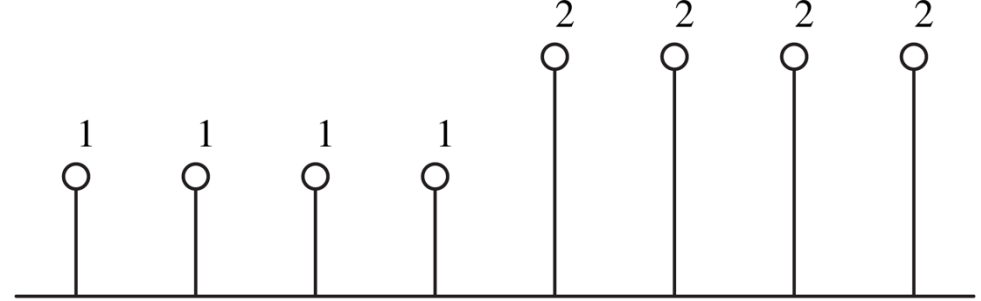
Convolution: Template Matching

- sliding window
- abstract templates
- similarity measured by dot product
- stronger activation, better matching



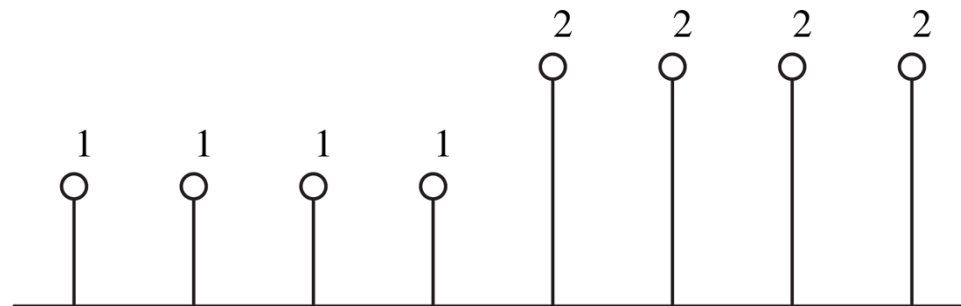
Convolution: a 1-D example

input

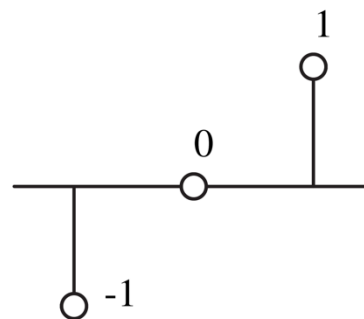


Convolution: a 1-D example

input



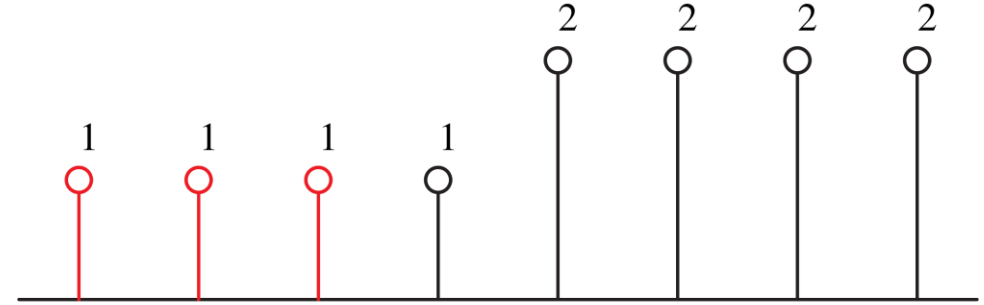
filter



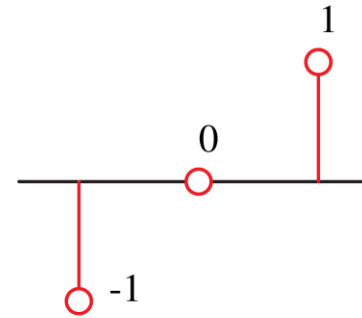
Convolution: a 1-D example

- sliding window
- dot product

input



filter



$$-1 \times 1 + 0 \times 1 + 1 \times 1$$

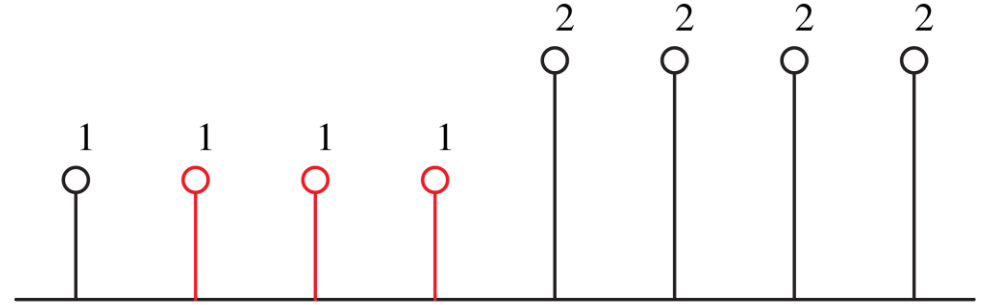
output



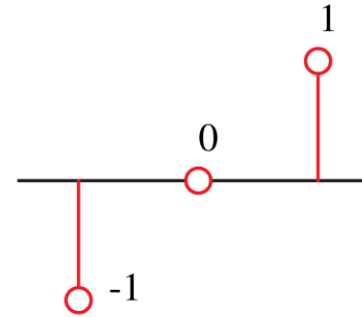
Convolution: a 1-D example

- sliding window
- dot product

input



filter



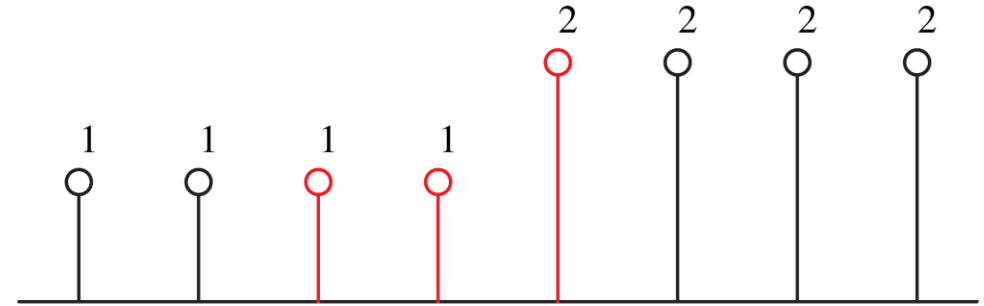
output



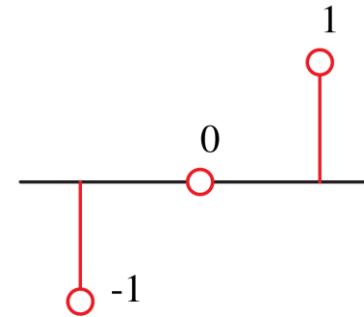
Convolution: a 1-D example

- sliding window
- dot product

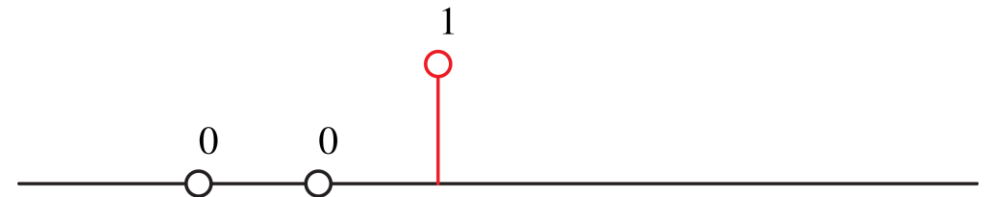
input



filter



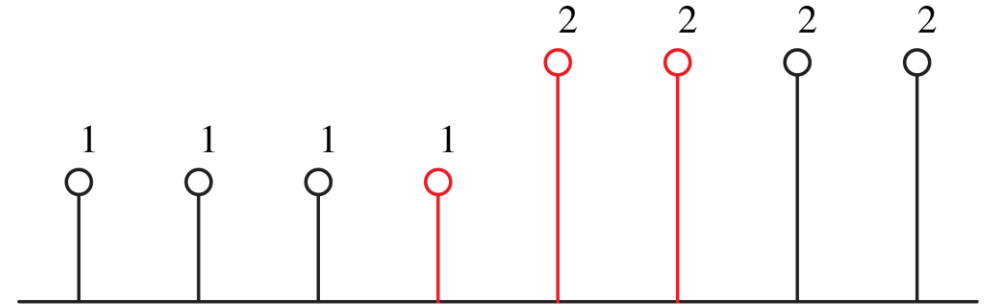
output



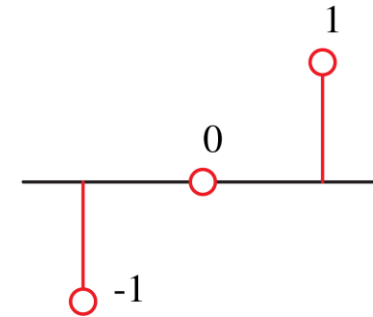
Convolution: a 1-D example

- sliding window
- dot product

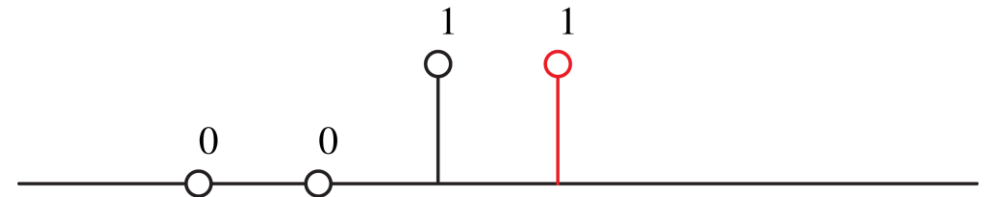
input



filter



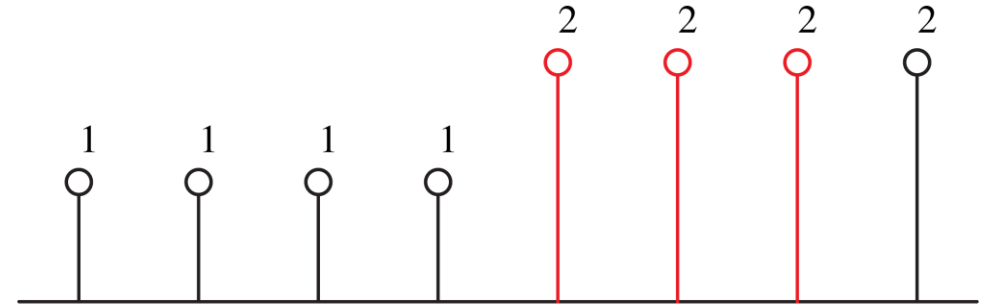
output



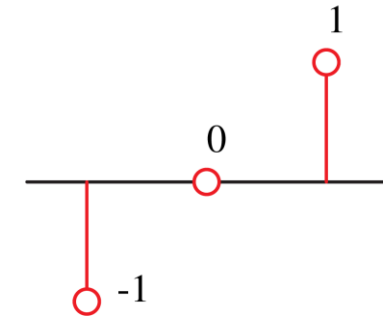
Convolution: a 1-D example

- sliding window
- dot product

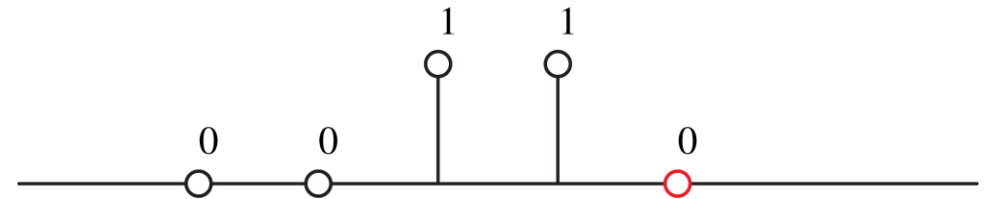
input



filter



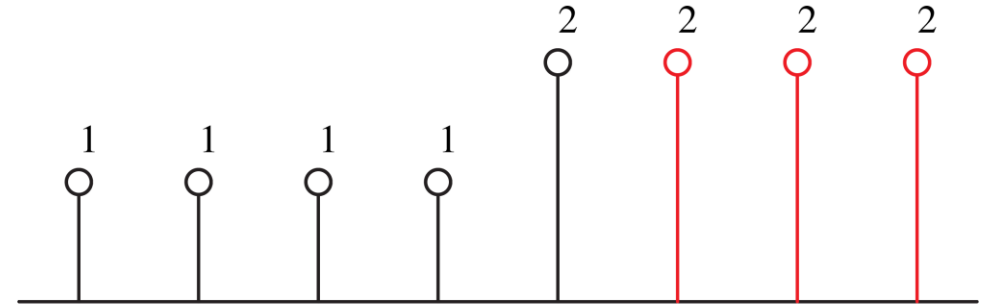
output



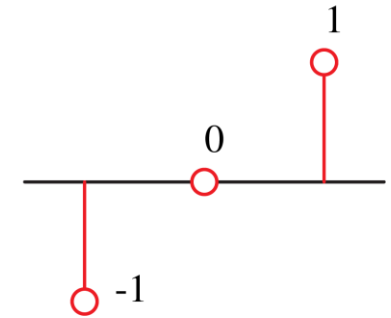
Convolution: a 1-D example

- sliding window
- dot product

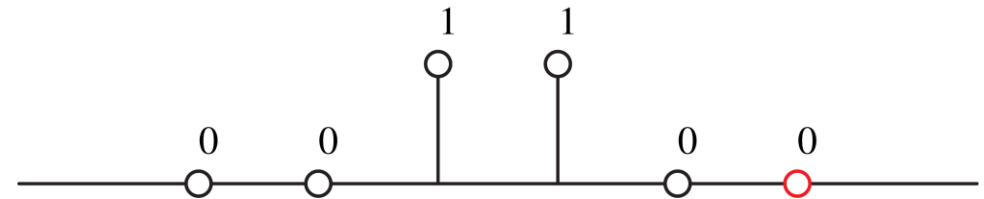
input



filter



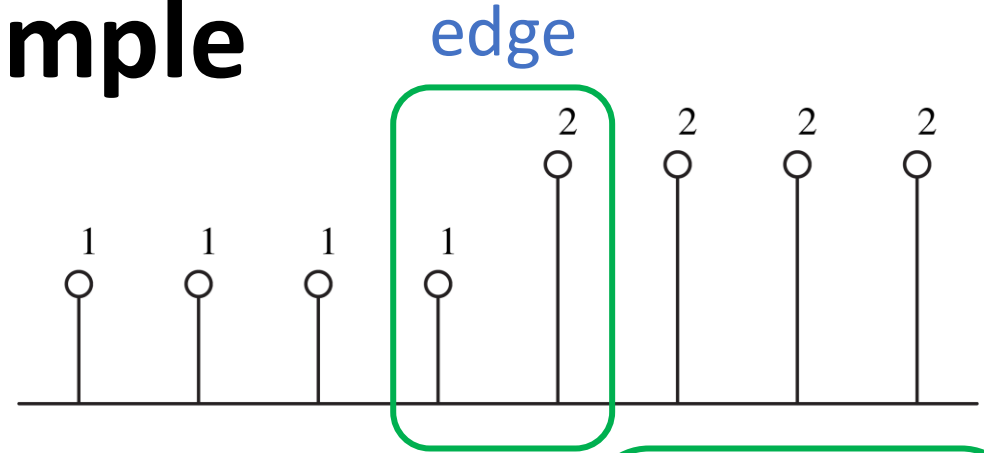
output



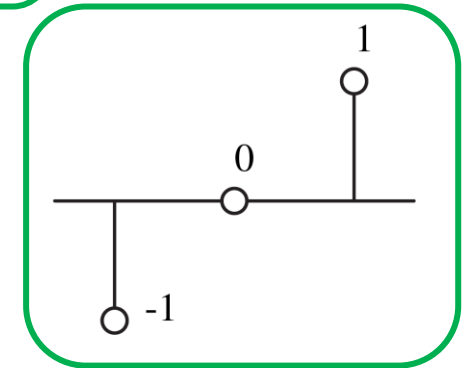
Convolution: a 1-D example

- sliding window
- dot product

input

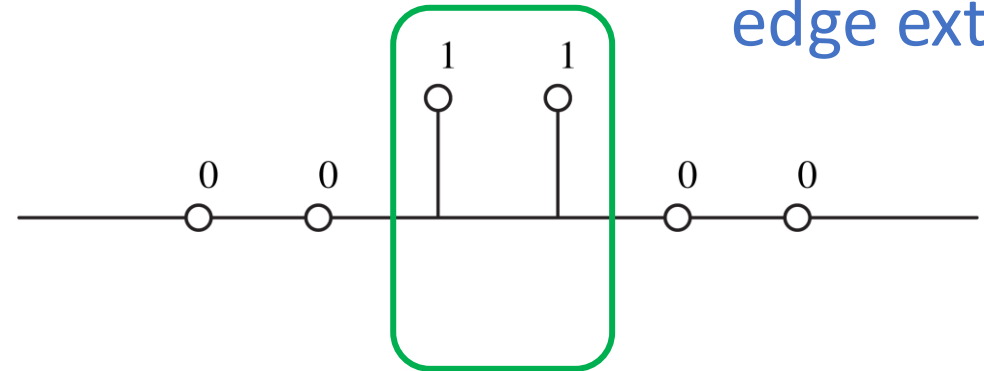


filter



edge extractor

output



indicator of edge ("feature")

Convolution: a 2-D example

input

0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	0
0	1	1	1	1	1	1	0
0	1	1	1	1	1	1	0
0	1	1	1	1	1	1	0
0	0	1	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	0

filter

1	2	1
0	0	0
-1	-2	-1

output

[illegible]

Convolution: a 2-D example

input

0_1	0_2	0_1	0	0	0	0	0
0_0	0_0	0_0	0	0	1	1	0
0_{-1}	1_{-2}	1_{-1}	1	1	1	1	0
0	1	1	1	1	1	1	0
0	1	1	1	1	1	1	0
0	0	1	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	0

filter

1	2	1
0	0	0
-1	-2	-1

- sliding window
- dot product

output

[illegible]

Convolution: a 2-D example

input

0	⁰ 1	⁰ 2	⁰ 1	0	0	0	0
0	⁰ 0	⁰ 0	⁰ 0	0	1	1	0
0	¹ -1	¹ -2	¹ -1	1	1	1	0
0	1	1	1	1	1	1	0
0	1	1	1	1	1	1	0
0	0	1	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	0

filter

1	2	1
0	0	0
-1	-2	-1

- sliding window
- dot product

output

[illegible]

Convolution: a 2-D example

input

0	0	0	0 ⁰ 1	0 ⁰ 2	0 ⁰ 1	0	0
0	0	0	0 ⁰ 0	0 ⁰ 0	1 ¹ 0	1	0
0	1	1	1 ¹ -1	1 ¹ -2	1 ¹ -1	1	0
0	1	1	1	1	1	1	0
0	1	1	1	1	1	1	0
0	0	1	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	0

filter

1	2	1
0	0	0
-1	-2	-1

- sliding window
- dot product

output

-3	-4	-4	-4		

Convolution: a 2-D example

input

0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	0
0	1	1	1	1	1	1	0
0	1	1	1	1	1	1	0
0	1	1	1	1	1	1	0
0	0	1	1	1	⁰ 1	⁰ 2	⁰ 1
0	0	1	1	1	⁰ 0	⁰ 0	⁰ 0
0	0	0	0	0	⁰ -1	⁰ -2	⁰ -1

filter

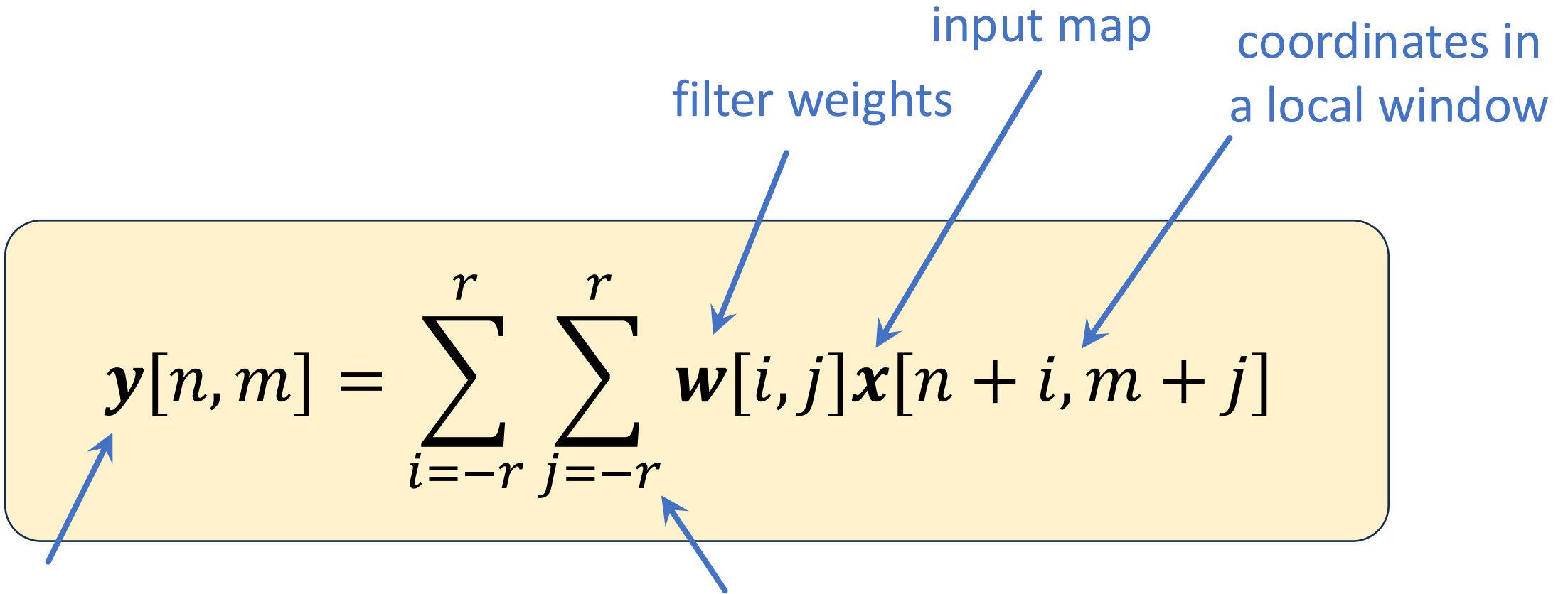
1	2	1
0	0	0
-1	-2	-1

- sliding window
- dot product

output

-3	-4	-4	-4	-4	-3
-3	-4	-4	-3	-1	0
0	0	0	0	0	0
2	1	0	1	3	3
2	1	0	1	3	3
1	3	4	3	1	0

Convolution: 2-D



The diagram shows the 2D convolution equation $y[n, m] = \sum_{i=-r}^r \sum_{j=-r}^r w[i, j] x[n + i, m + j]$ enclosed in a yellow rounded rectangle. Blue arrows point from external labels to parts of the equation: 'output map' points to $y[n, m]$, 'filter weights' points to $w[i, j]$, 'input map' points to $x[n + i, m + j]$, and 'coordinates in a local window' points to $n + i$ and $m + j$. Below the rectangle, text defines r as the kernel radius and states that the kernel size is $2r + 1$.

$$y[n, m] = \sum_{i=-r}^r \sum_{j=-r}^r w[i, j] x[n + i, m + j]$$

output map

filter weights

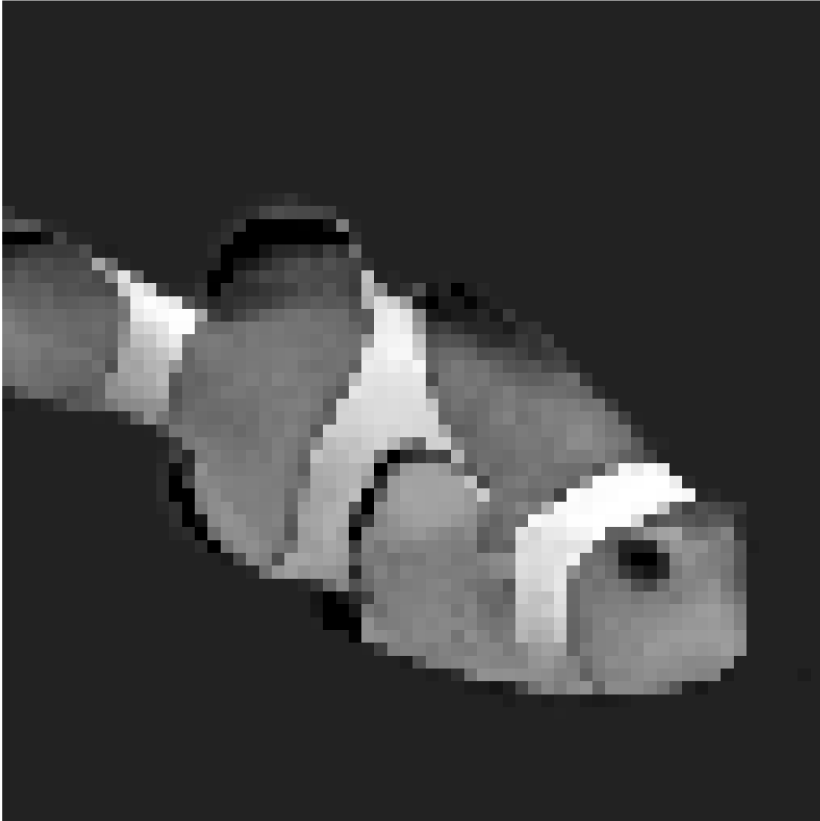
input map

coordinates in a local window

r : kernel radius
kernel size = $2r + 1$

* In ConvNets, convolution is often implemented as **cross-correlation** (no flipping)

Convolution: 2-D

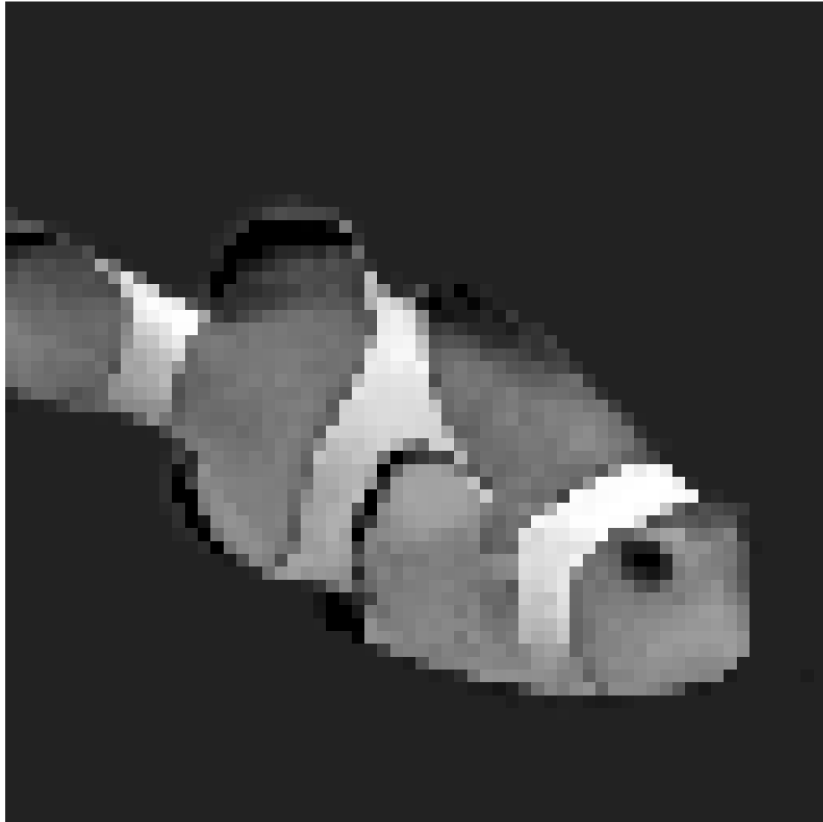


$$* \begin{array}{c} \text{filter} \\ \begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 0 & 0 & 0 \\ \hline -1 & -2 & -1 \\ \hline \end{array} \end{array} =$$



This example is a Sobel edge extractor of horizontal edges.

Convolution: 2-D

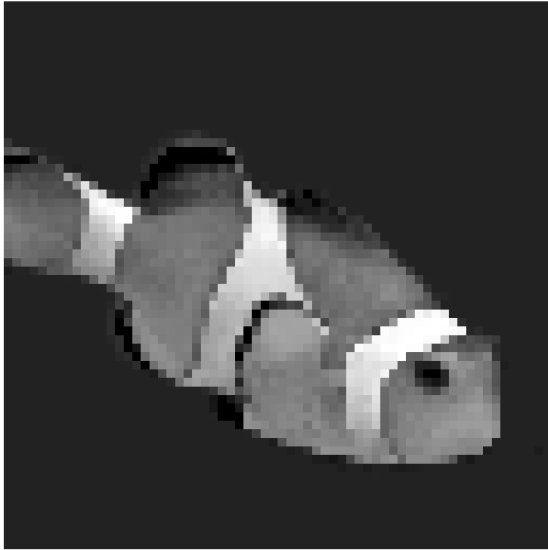


$$\begin{array}{c} \text{filter} \\ * \begin{array}{ccc} \text{light gray} & \text{white} & \text{light gray} \\ \text{medium gray} & \text{medium gray} & \text{medium gray} \\ \text{dark gray} & \text{black} & \text{dark gray} \end{array} = \end{array}$$



This example is a Sobel edge extractor of horizontal edges.

Convolution: Multi-channel outputs



*

1	2	1
0	0	0
-1	-2	-1

=



one filter, one feature

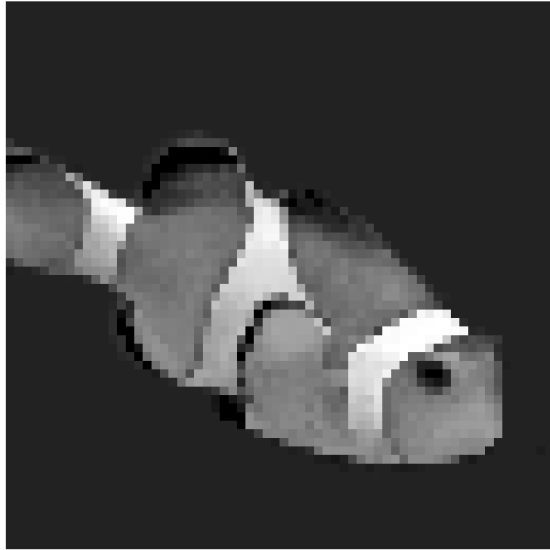
*

1	0	-1
2	0	-2
1	0	-1

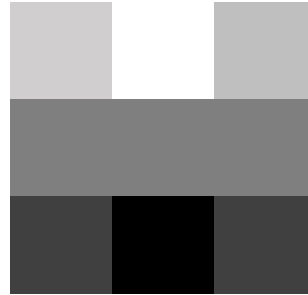
=



Convolution: Multi-channel outputs



*

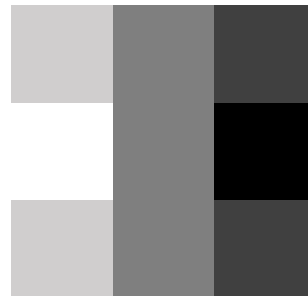


=



one filter, one feature

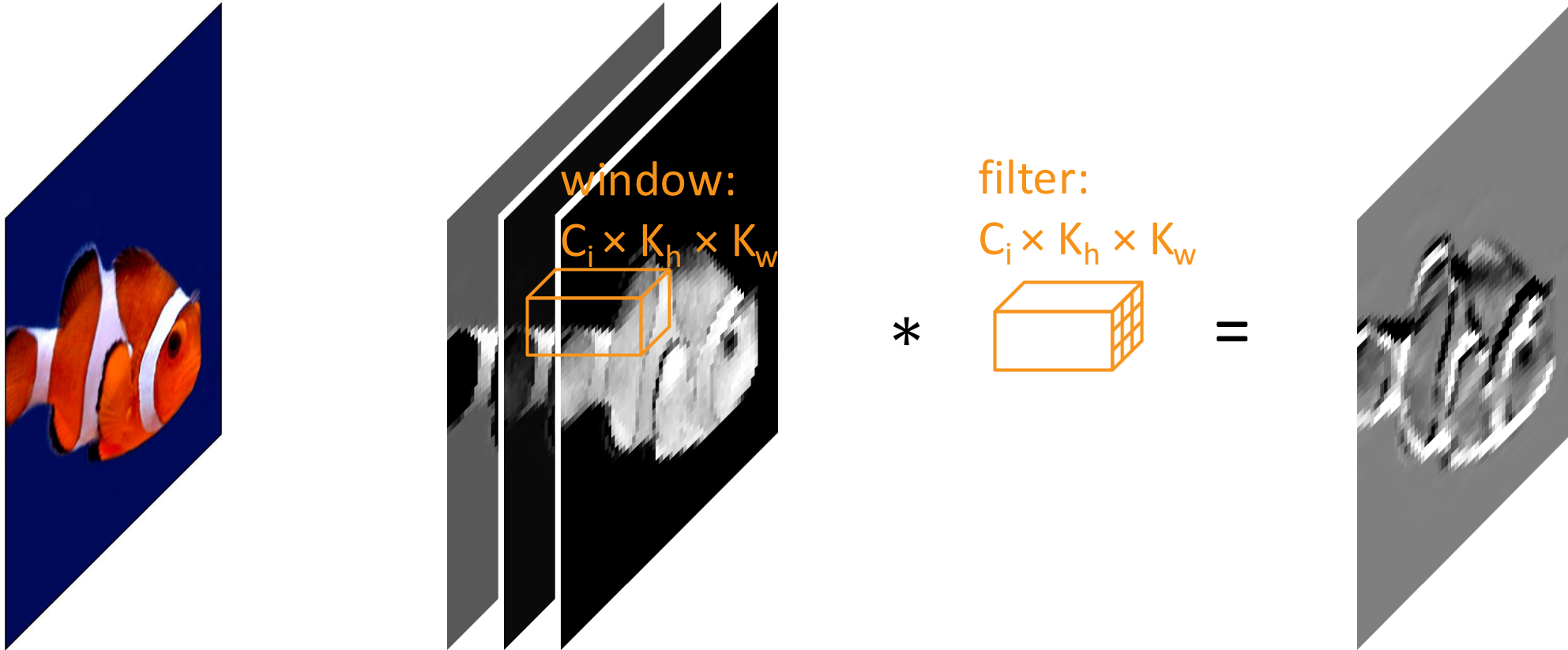
*



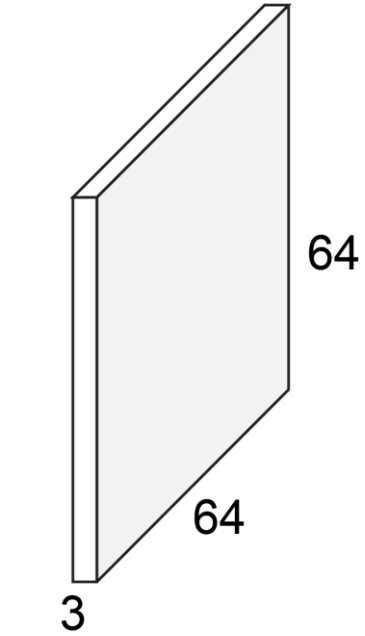
=



Convolution: Multi-channel inputs

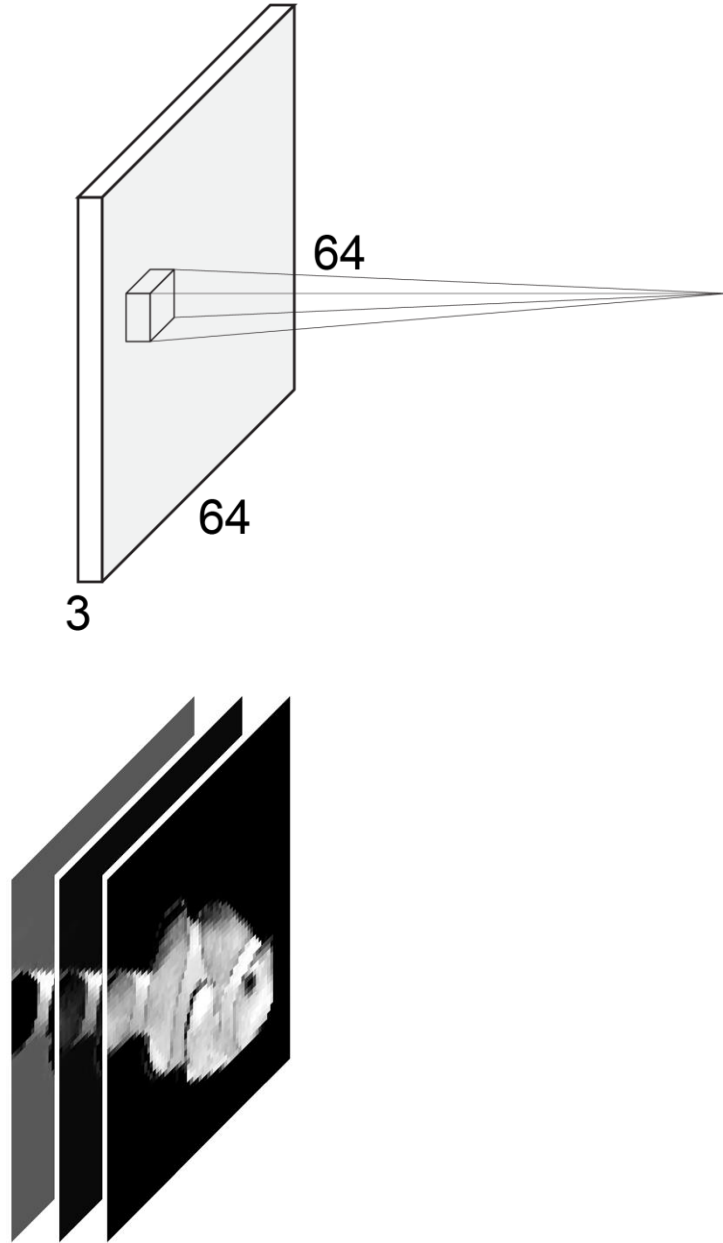


Convolution: tensor views



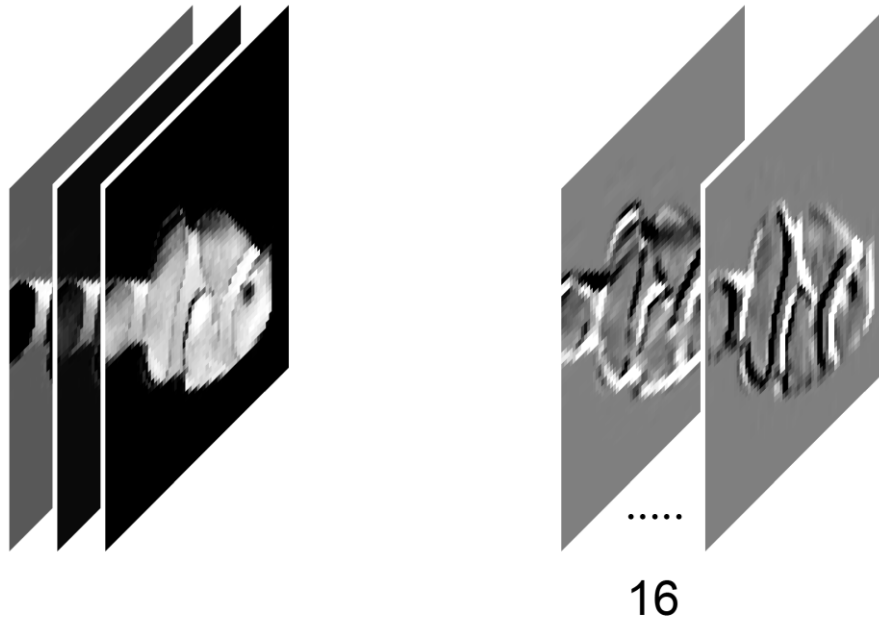
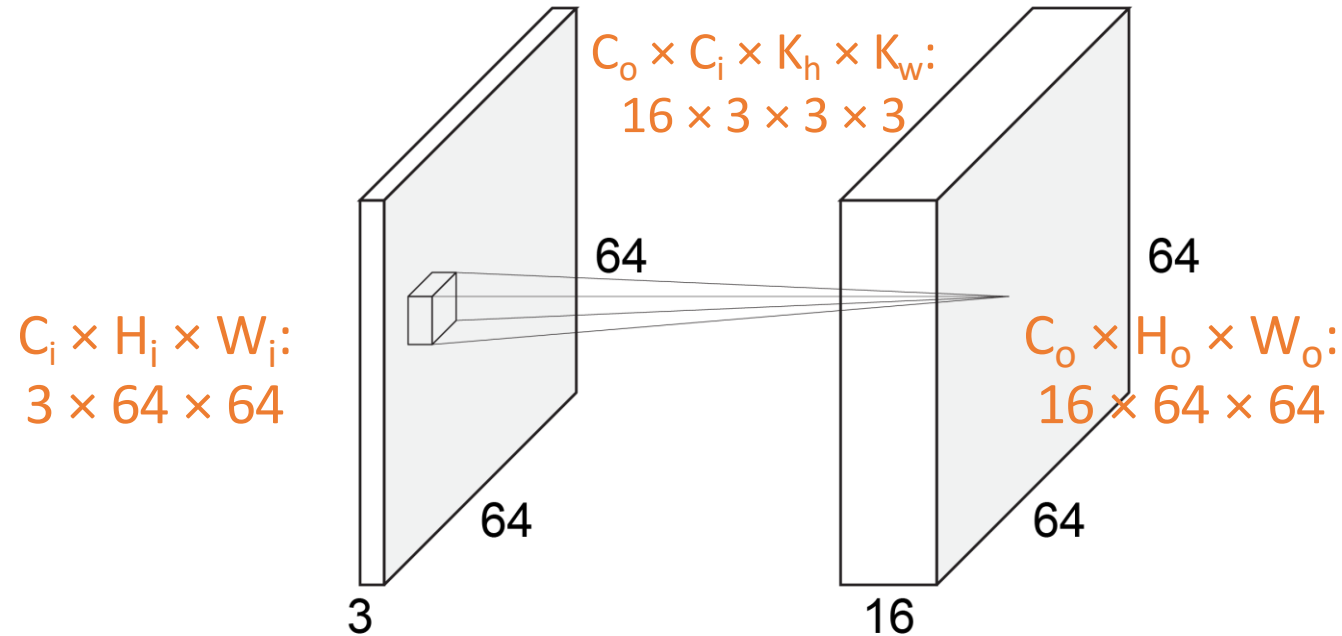
- Tensor: multi-dim array
- feature maps
 - 3-D tensor: $C \times H \times W$
 - C: channels
 - H: height
 - W: width

Convolution: tensor views



- Tensor: multi-dim array
- feature maps
 - 3-D tensor: $C \times H \times W$
 - C : channels
 - H : height
 - W : width
- filters
 - 4-D tensor: $C_o \times C_i \times K_h \times K_w$
 - C_o : output channels
 - C_i : input channels
 - K_h, K_w : filter height, width

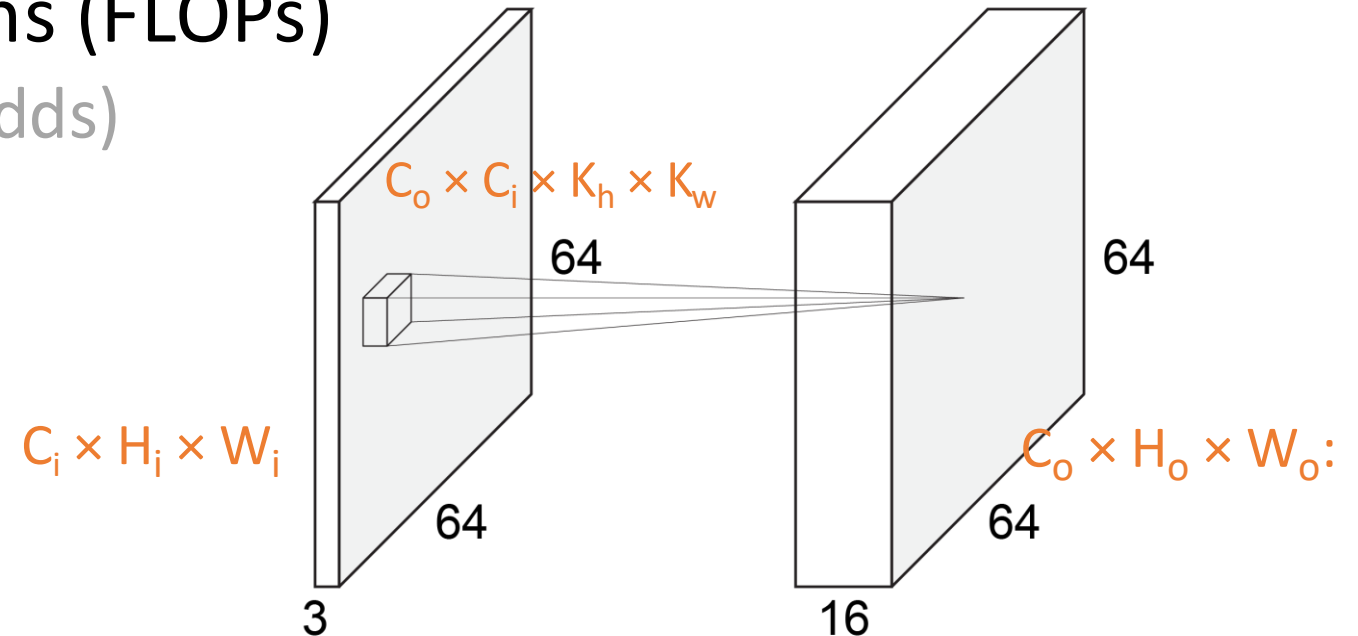
Convolution: tensor views



- Tensor: multi-dim array
- feature maps
 - 3-D tensor: $C \times H \times W$
 - C: channels
 - H: height
 - W: width
- filters
 - 4-D tensor: $C_o \times C_i \times K_h \times K_w$
 - C_o : output channels
 - C_i : input channels
 - K_h, K_w : filter height, width

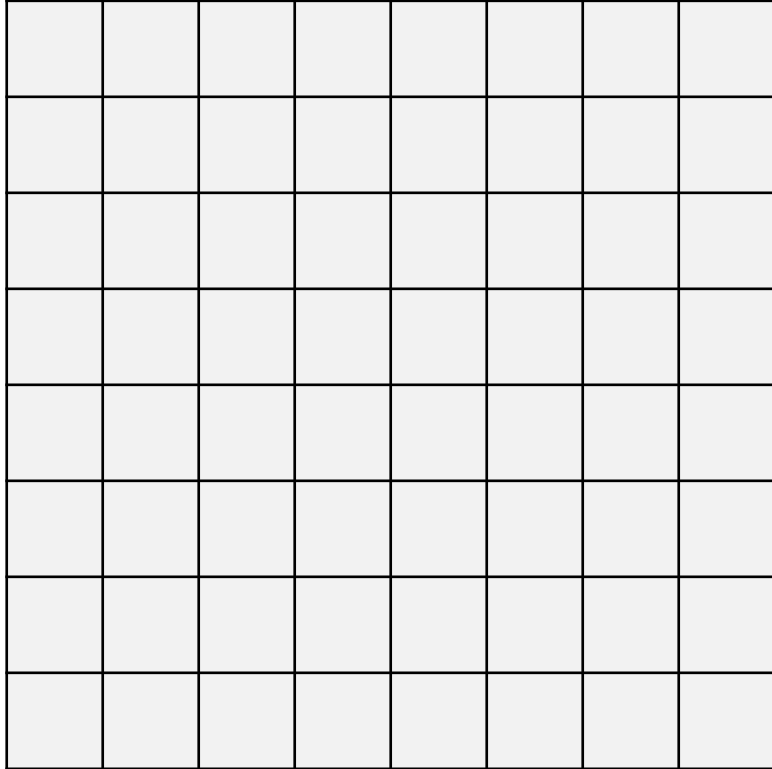
Convolution: # parameters and # operations

- # parameters
 - weights: $C_o \times C_i \times K_h \times K_w$
 - bias: C_o
- # floating-point operations (FLOPs)
 - # params $\times H_o \times W_o$ (mul-adds)
- # params per FLOP
 - 1 / spatial_size
 - parameter-efficient

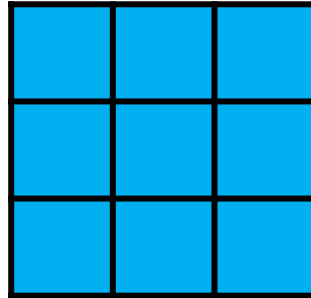


Convolution: padding

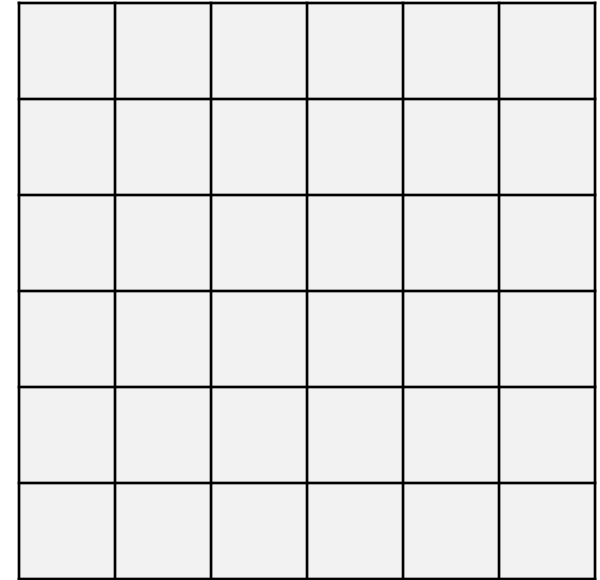
input: $H \times W = 8 \times 8$



filter

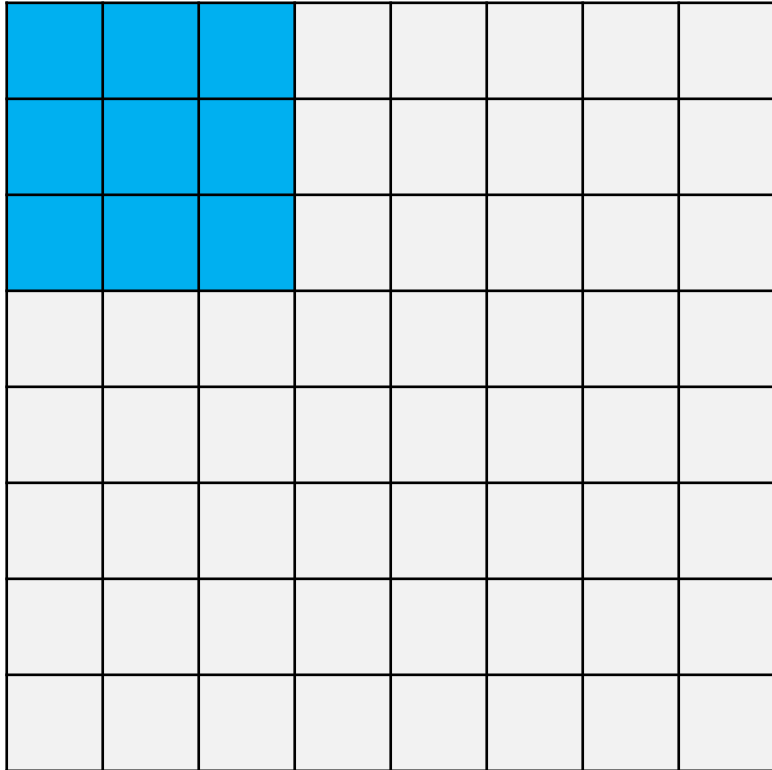


output: $H \times W = 6 \times 6$

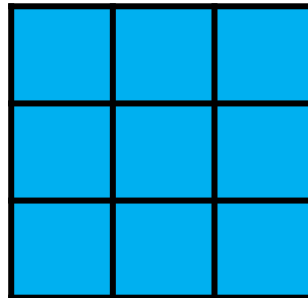


Convolution: padding

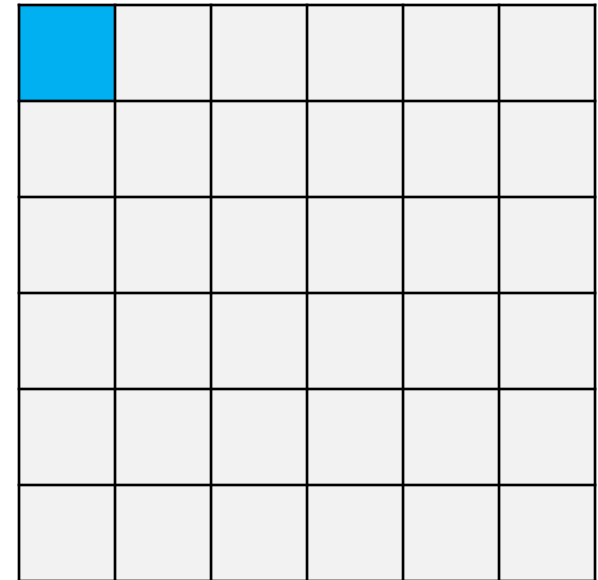
input: $H \times W = 8 \times 8$



filter

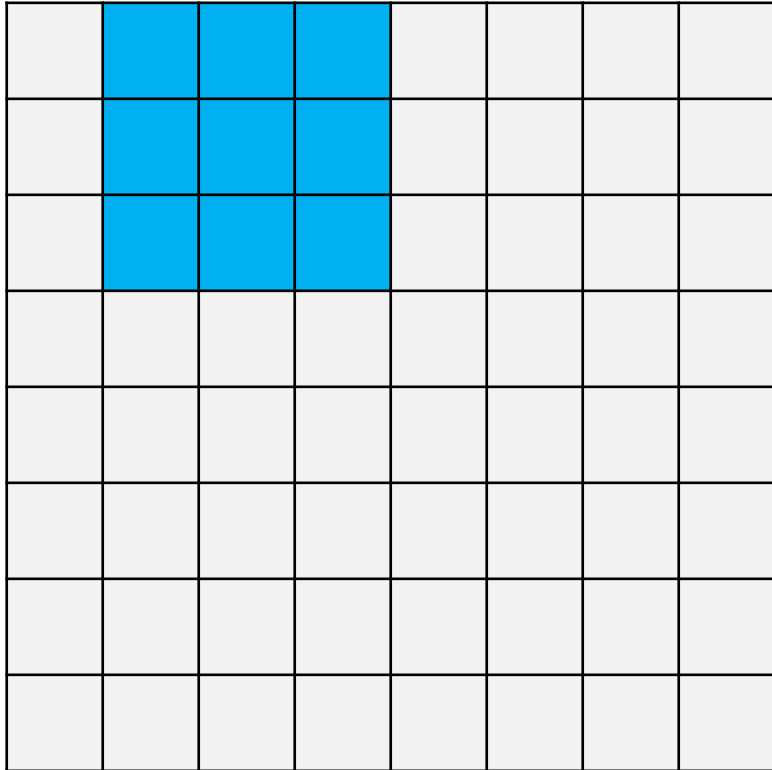


output: $H \times W = 6 \times 6$

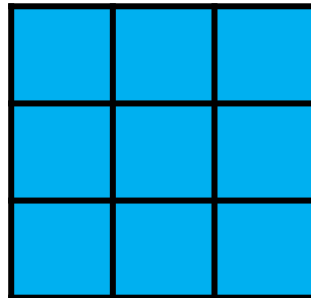


Convolution: padding

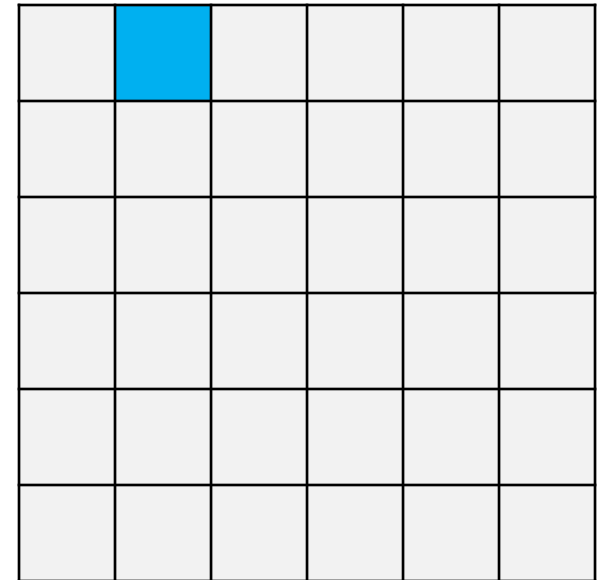
input: $H \times W = 8 \times 8$



filter

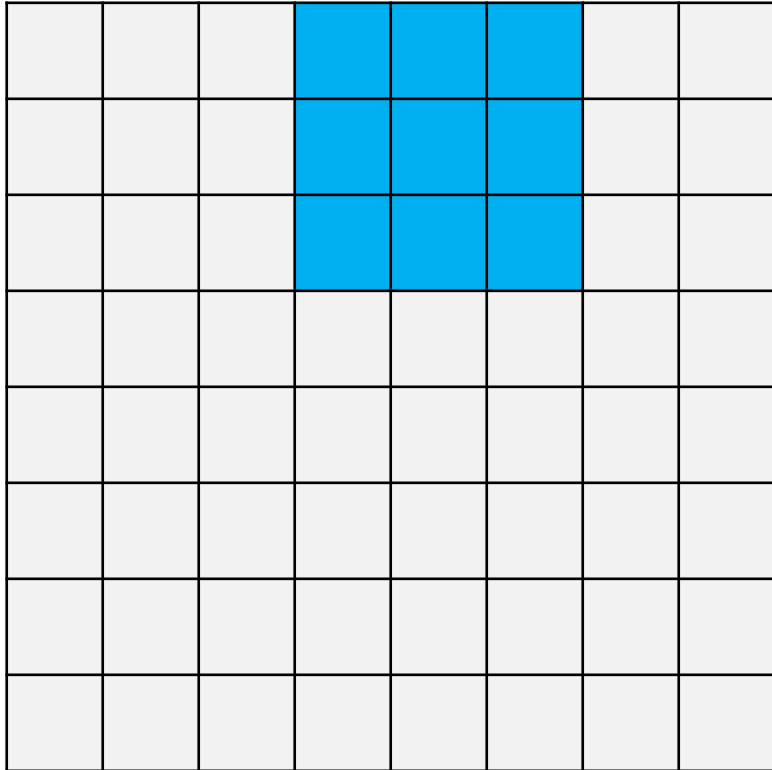


output: $H \times W = 6 \times 6$

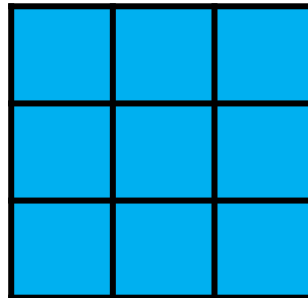


Convolution: padding

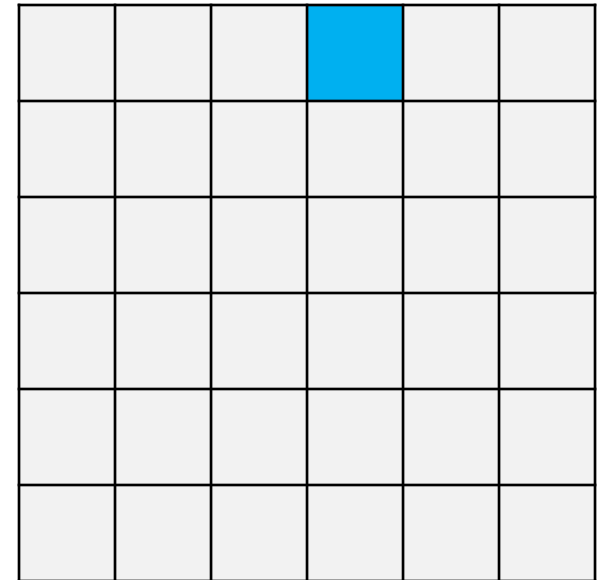
input: $H \times W = 8 \times 8$



filter

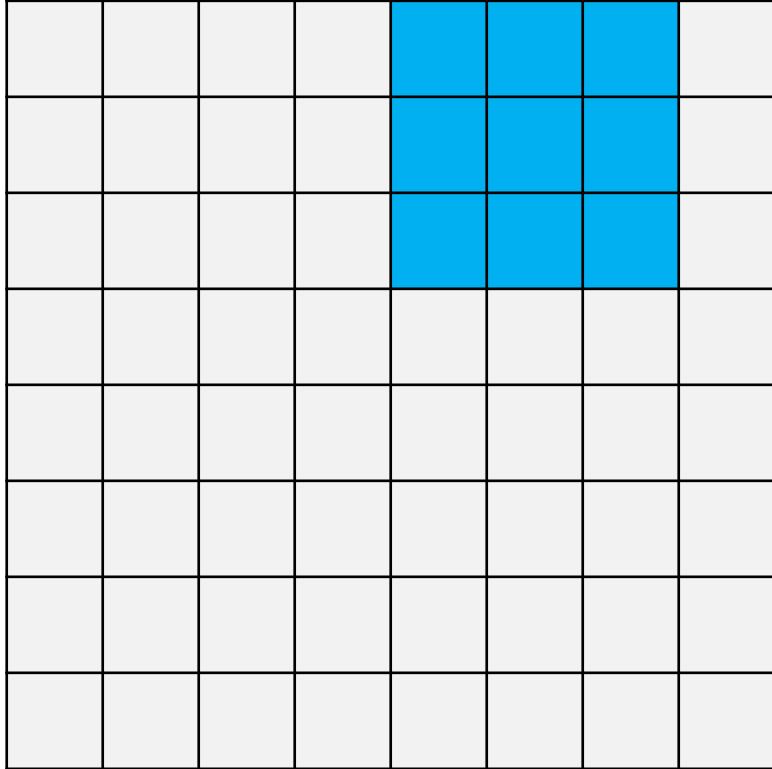


output: $H \times W = 6 \times 6$

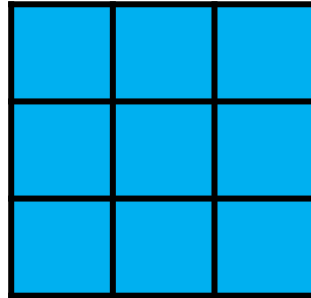


Convolution: padding

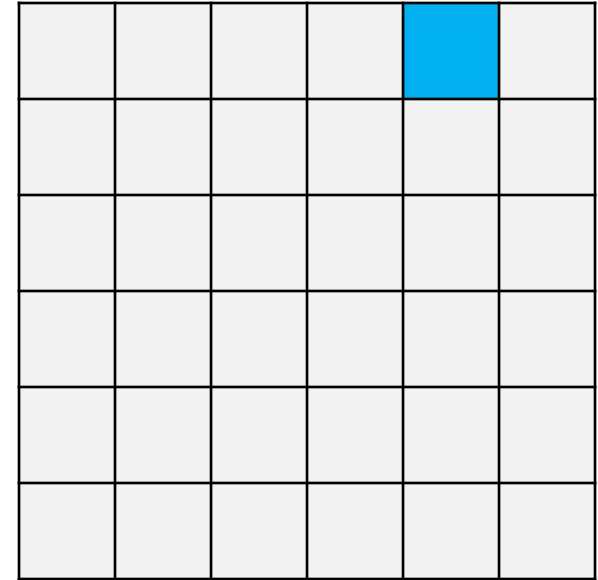
input: $H \times W = 8 \times 8$



filter

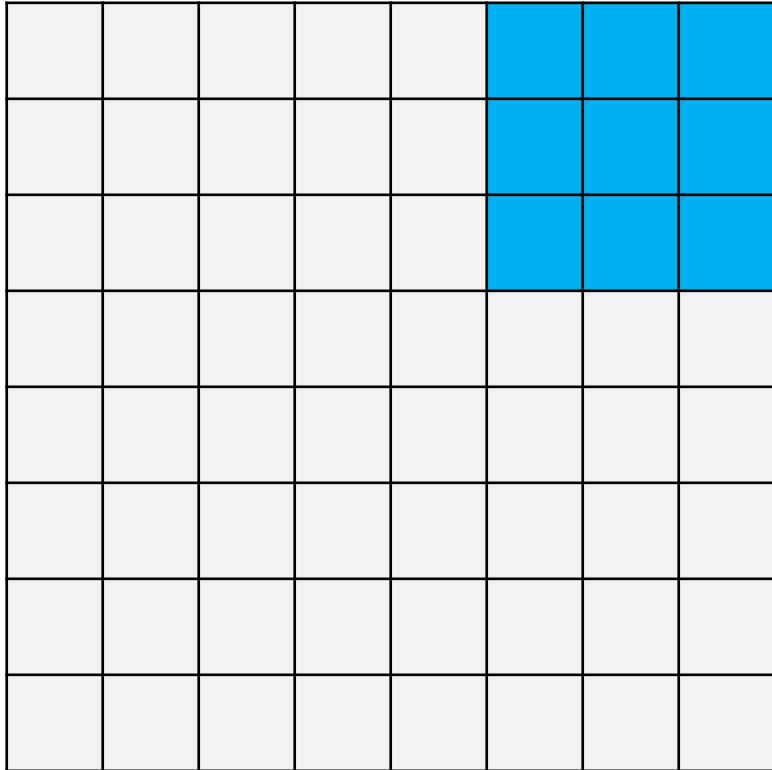


output: $H \times W = 6 \times 6$

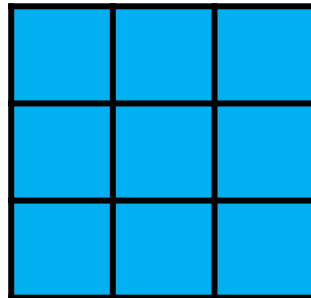


Convolution: padding

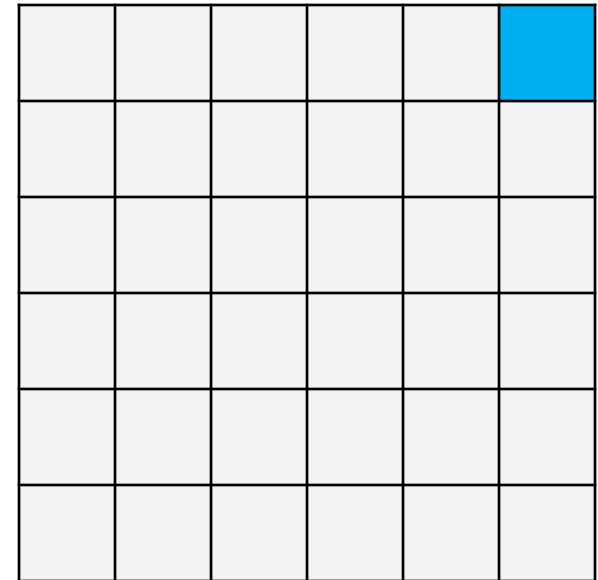
input: $H \times W = 8 \times 8$



filter



output: $H \times W = 6 \times 6$



$$H_{\text{out}} = H_{\text{in}} - K_h + 1$$

Convolution: padding

input: 8×8 , + pad

[illegible]

filter

output: $H \times W = 8 \times 8$

[illegible]

Convolution: padding

input: 8×8 , + pad

[illegible]

filter

output: $H \times W = 8 \times 8$

[illegible]

Convolution: padding

input: 8×8 , + pad

[illegible]

filter

output: $H \times W = 8 \times 8$

[illegible]

Convolution: padding

input: 8×8 , + pad

[illegible]

filter

output: $H \times W = 8 \times 8$

[illegible]

Convolution: padding

input: 8×8 , + pad

[illegible]

filter

output: $H \times W = 8 \times 8$

[illegible]

Convolution: padding

input: 8×8 , + pad

[illegible]

filter

output: $H \times W = 8 \times 8$

[illegible]

Convolution: padding

input: 8×8 , + pad

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

filter

output: $H \times W = 8 \times 8$

[illegible]

Convolution: padding

input: 8×8 , + pad

[illegible]

filter

output: $H \times W = 8 \times 8$

[illegible]

Convolution: padding

input: 8×8 , + pad

[illegible]

filter

output: $H \times W = 8 \times 8$

[illegible]

Convolution: padding

input: 8×8 , + pad

0	0	0	0	0	0	0	0	0	0
0									0
0									0
0									0
0									0
0									0
0									0
0									0
0									0
0									0
0	0	0	0	0	0	0	0	0	0

filter

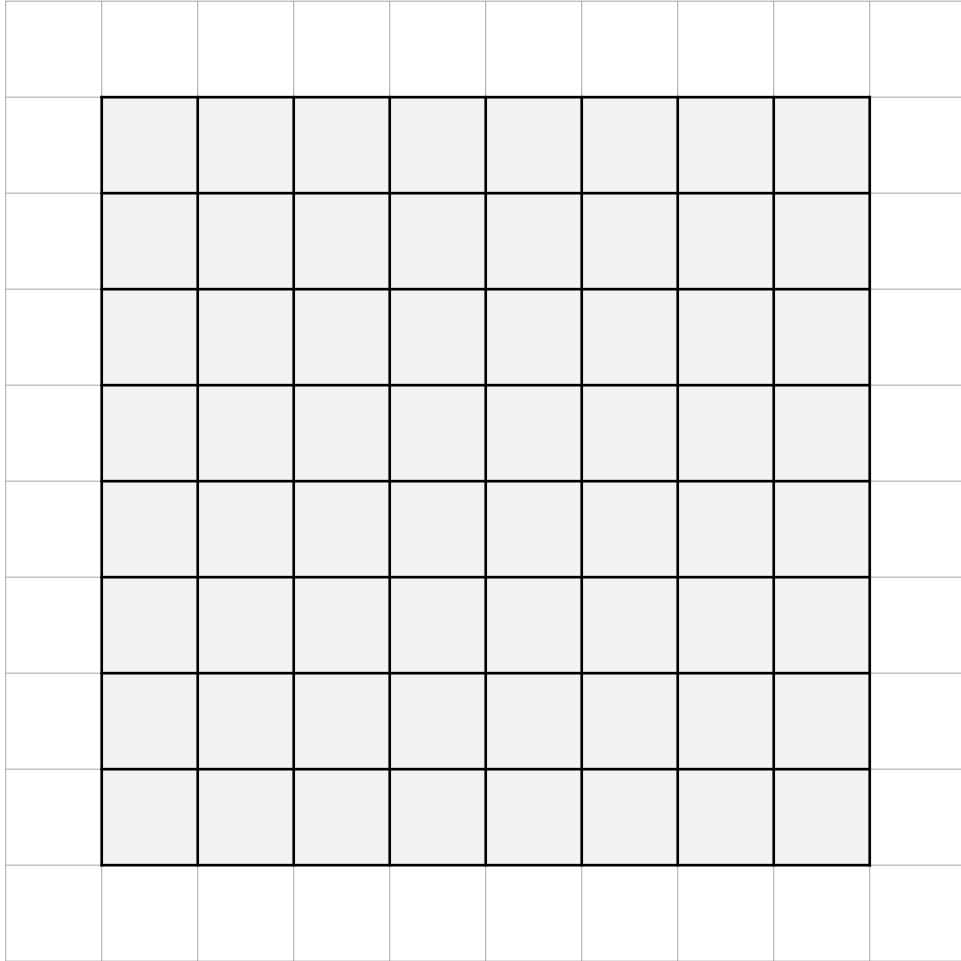
output: $H \times W = 8 \times 8$

- $\text{pad} = \lfloor \text{kernel_size} / 2 \rfloor$
- maintains feature map size

$$H_{\text{out}} = H_{\text{in}} + 2\text{pad}_h - K_h + 1$$

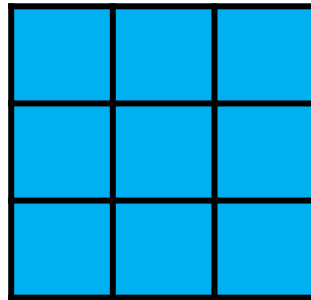
Convolution: stride

input

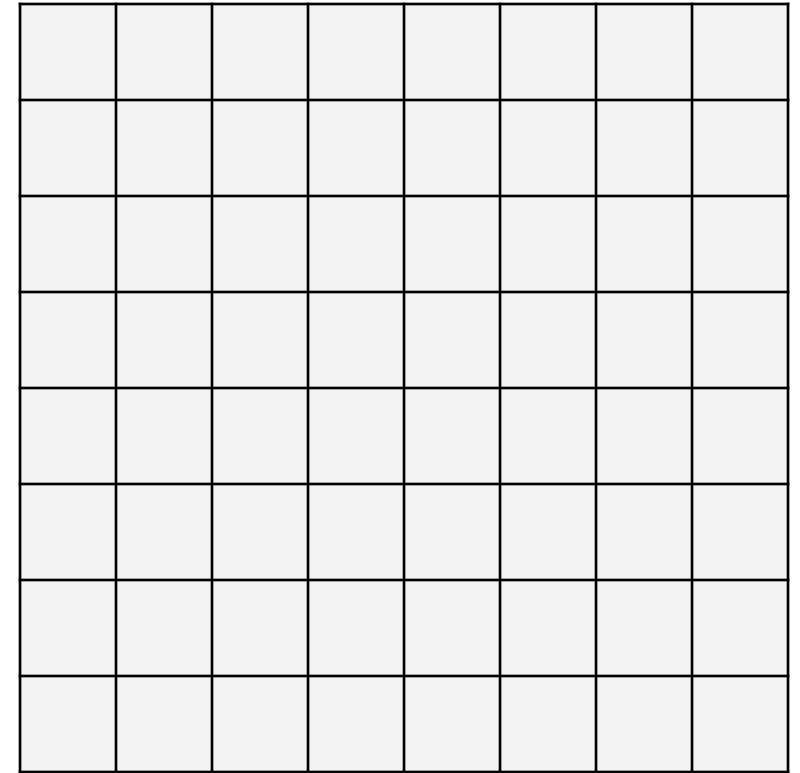


stride = 2

filter

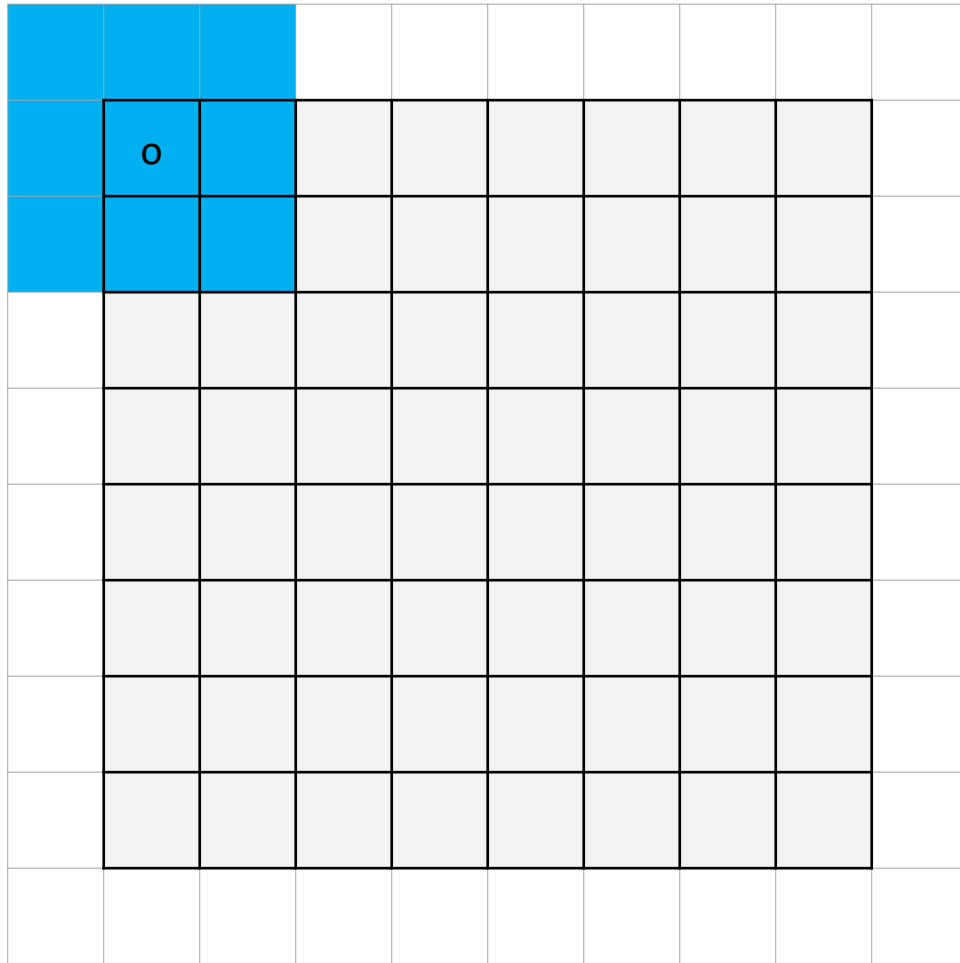


output



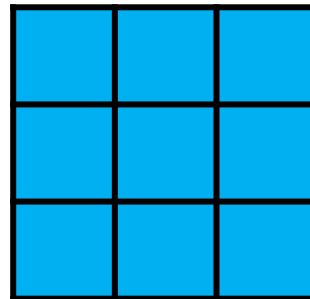
Convolution: stride

input

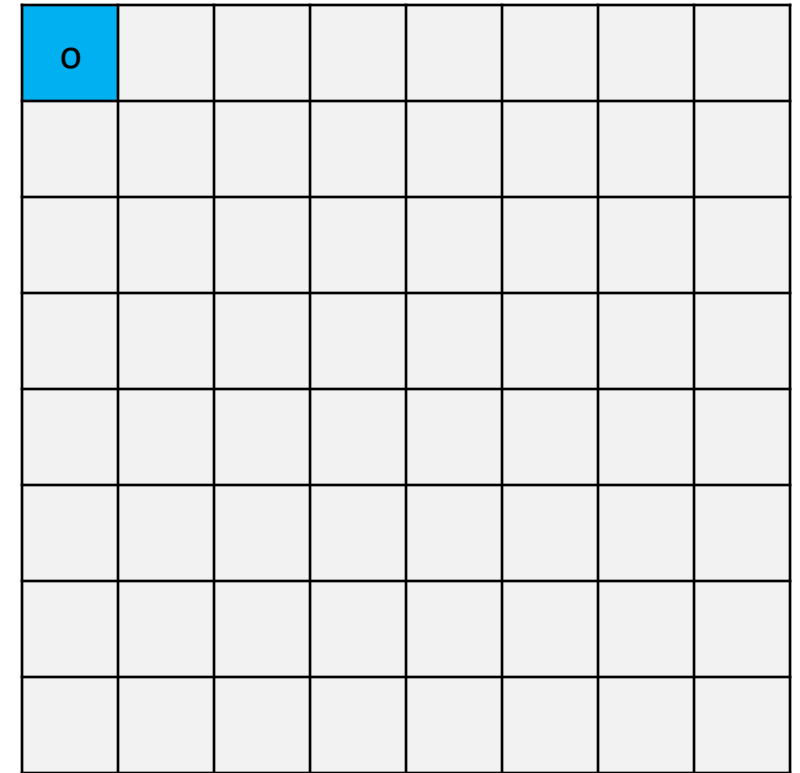


stride = 2

filter

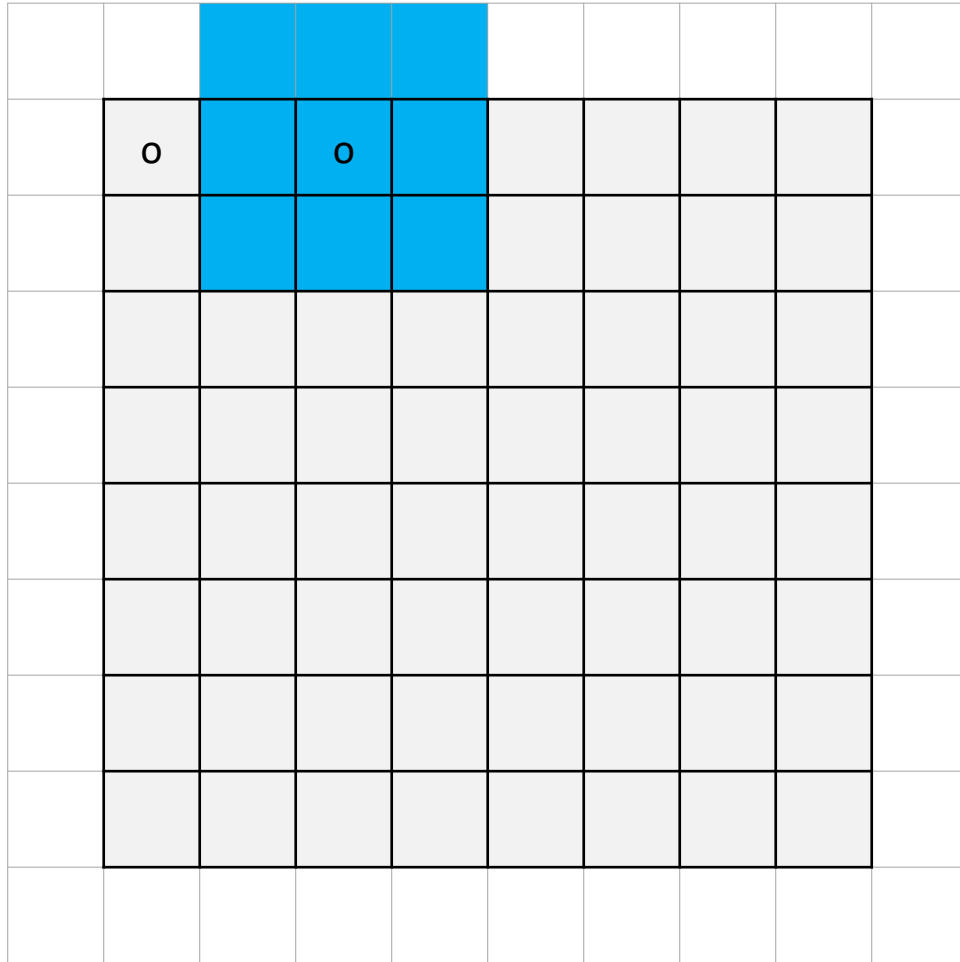


output



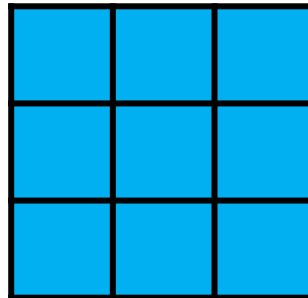
Convolution: stride

input

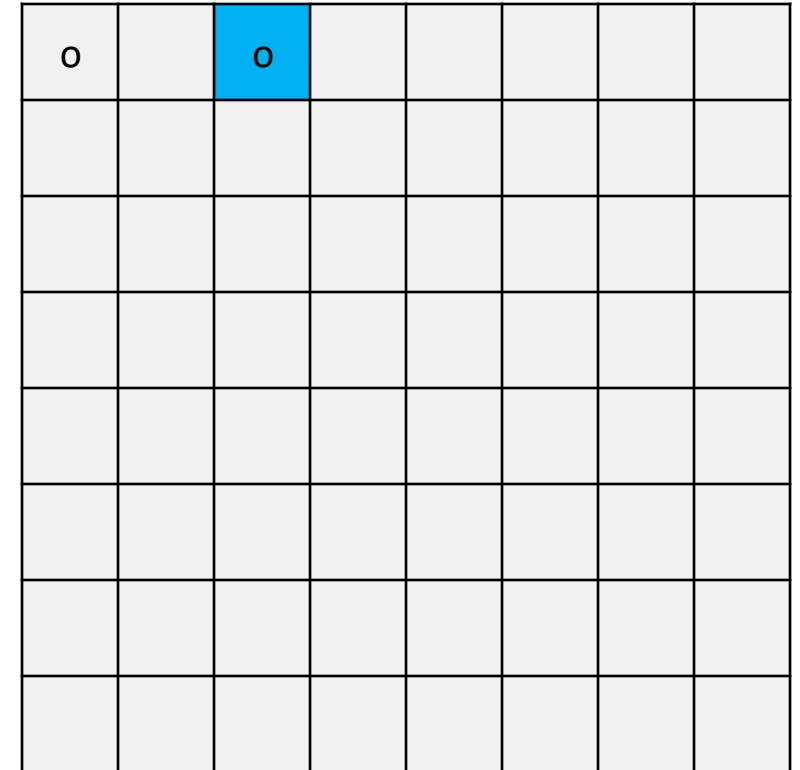


stride = 2

filter

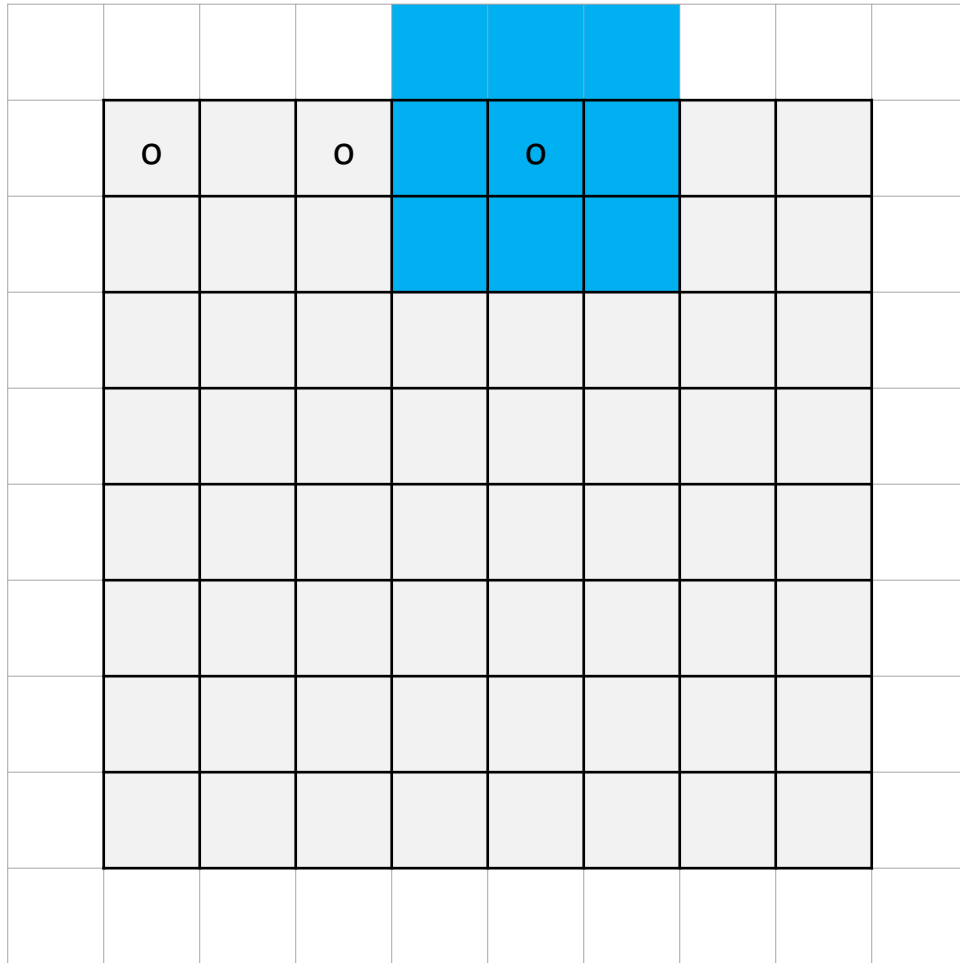


output



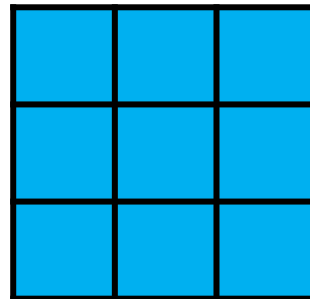
Convolution: stride

input

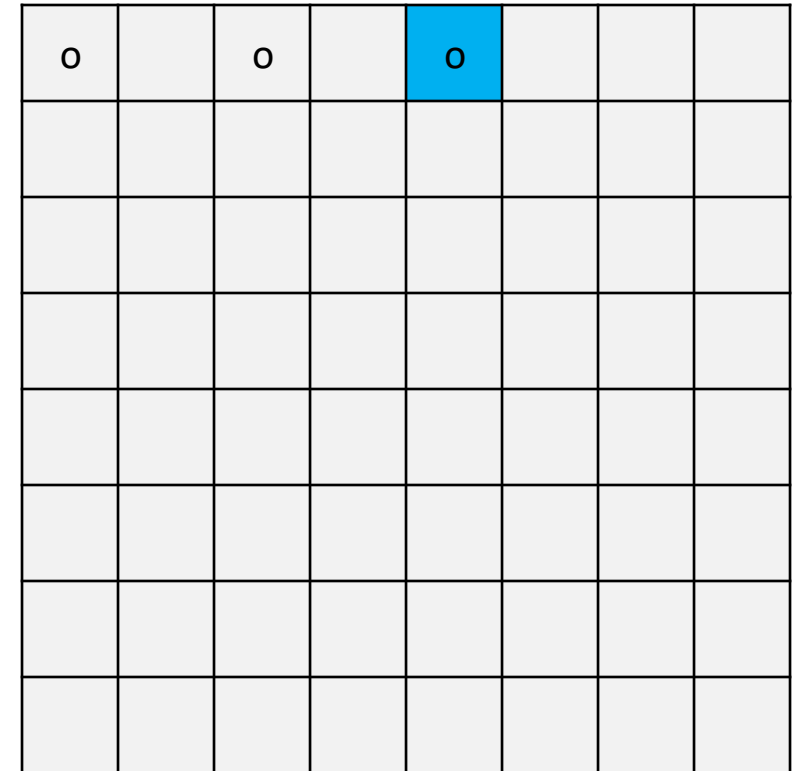


stride = 2

filter

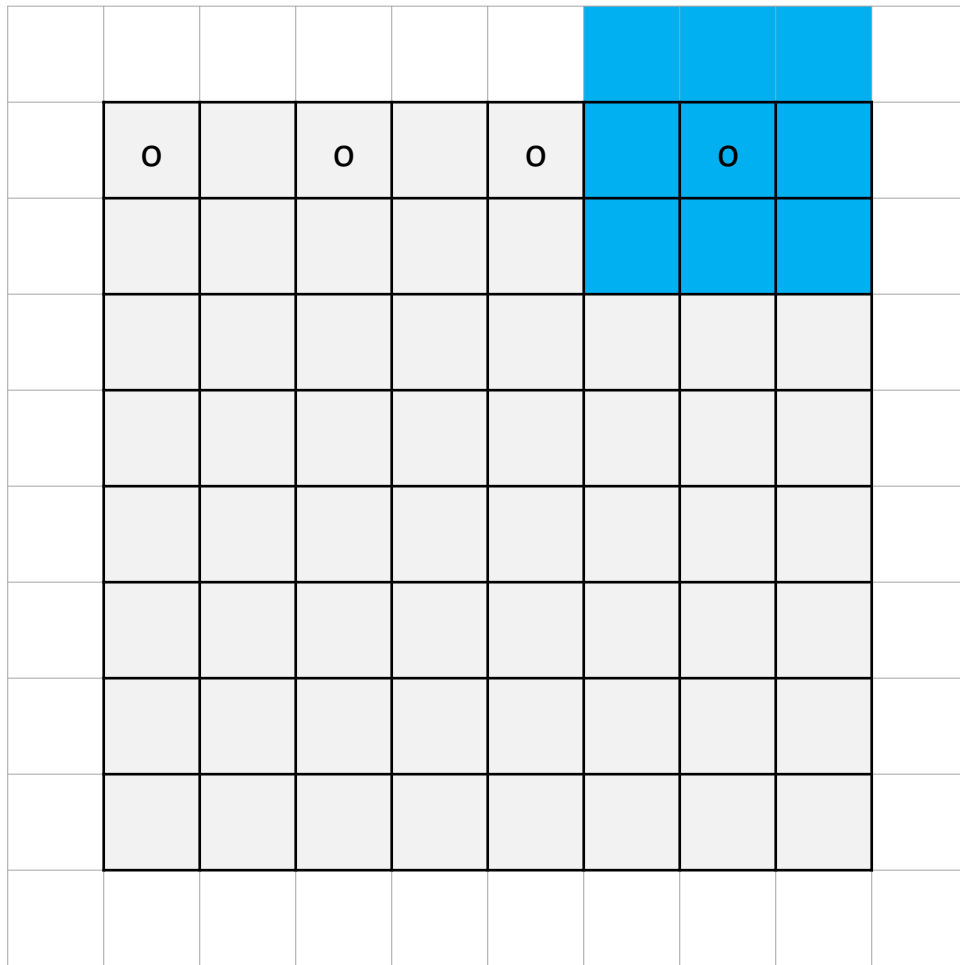


output



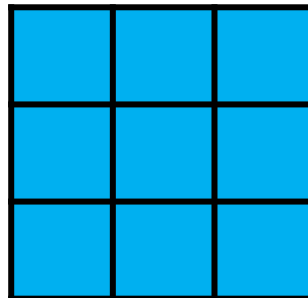
Convolution: stride

input

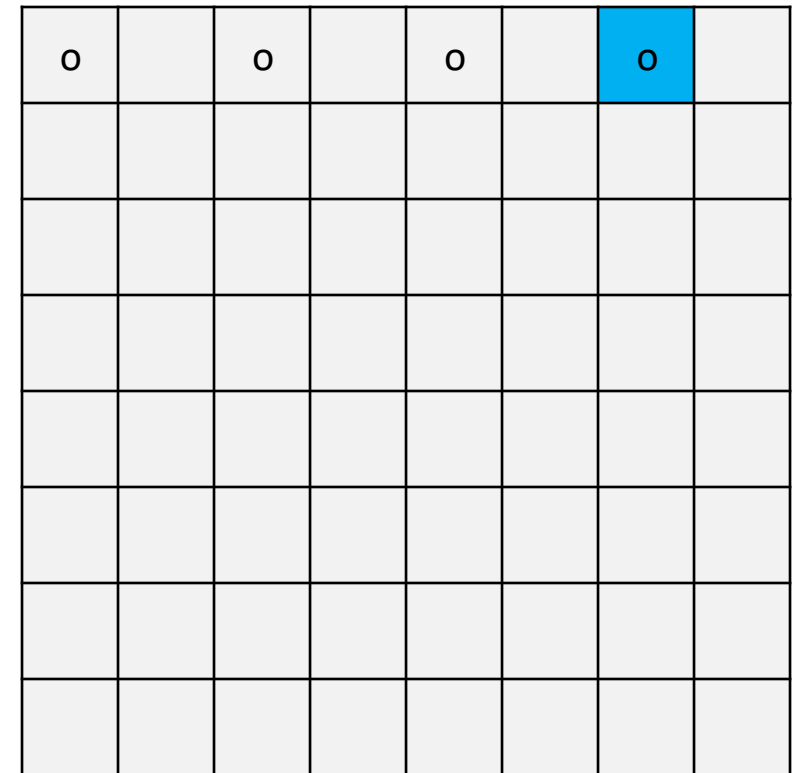


stride = 2

filter

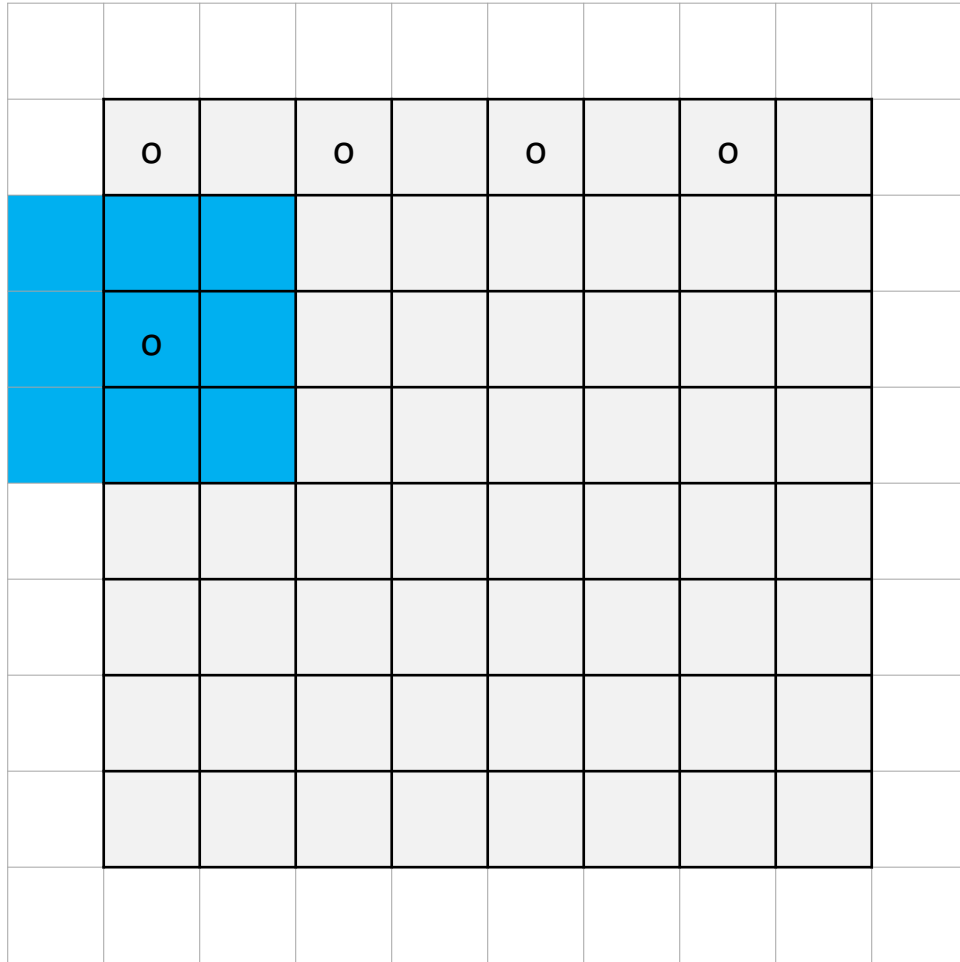


output



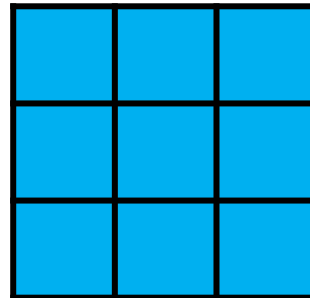
Convolution: stride

input

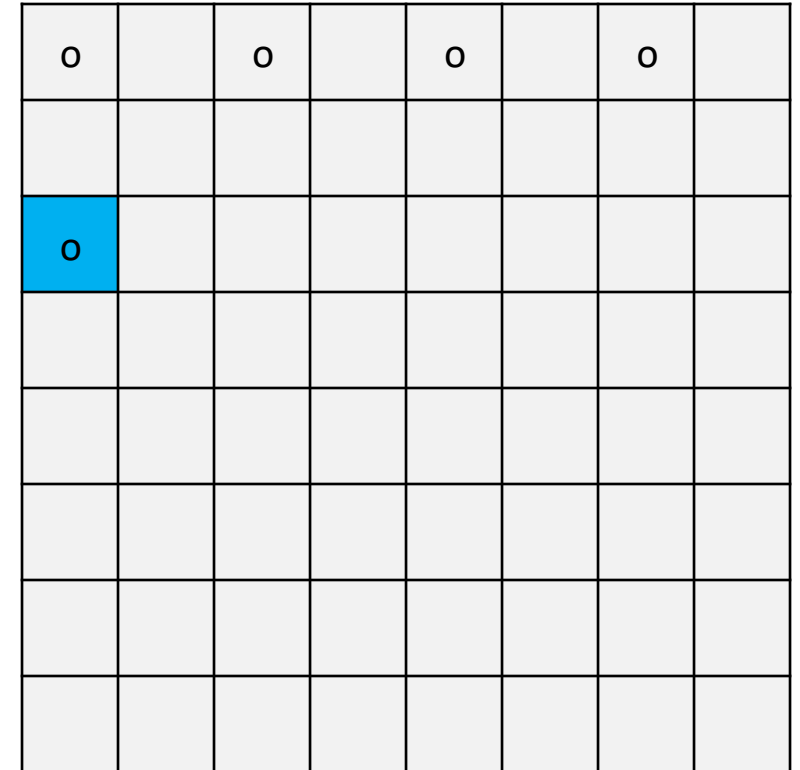


stride = 2

filter



output



Convolution: stride

input

	o		o		o		o		
	o		o		o		o		
	o		o		o		o		
	o		o		o		o		
	o		o		o		o		

filter

output

o		o		o		o	
o		o		o		o	
o		o		o		o	
o		o		o		o	
o		o		o		o	

Convolution: stride

input: $H \times W = 8 \times 8$

	o		o		o		o		
	o		o		o		o		
	o		o		o		o		
	o		o		o		o		

stride = 2

filter

output: $H \times W = 4 \times 4$

o		o		o		o	
o		o		o		o	
o		o		o		o	
o		o		o		o	

Convolution: stride

input: $H \times W = 8 \times 8$

	o		o		o		o		
	o		o		o		o		
	o		o		o		o		
	o		o		o		o		

stride = 2

filter

- reduces feature map size
- compress and abstract

output: $H \times W = 4 \times 4$

o	o	o	o
o	o	o	o
o	o	o	o
o	o	o	o

$$H_{\text{out}} = \lfloor (H_{\text{in}} + 2\text{pad}_h - K_h) / \text{str} \rfloor + 1$$

*rounding operation depends on libraries

Convolution: summary of common configs

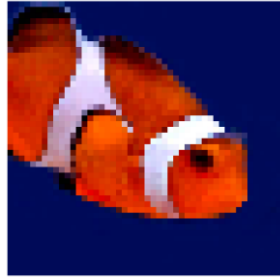
- kernel size: $K_h \times K_w$ (e.g., 3×3)
- output channels: C_o
 - input channels: C_i determined by input
- padding (e.g., 1)
- stride (e.g., 1 or 2)

Convolution: translation-equivariance

- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}(f(x))$

Convolution: translation-equivariance

- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}(f(x))$

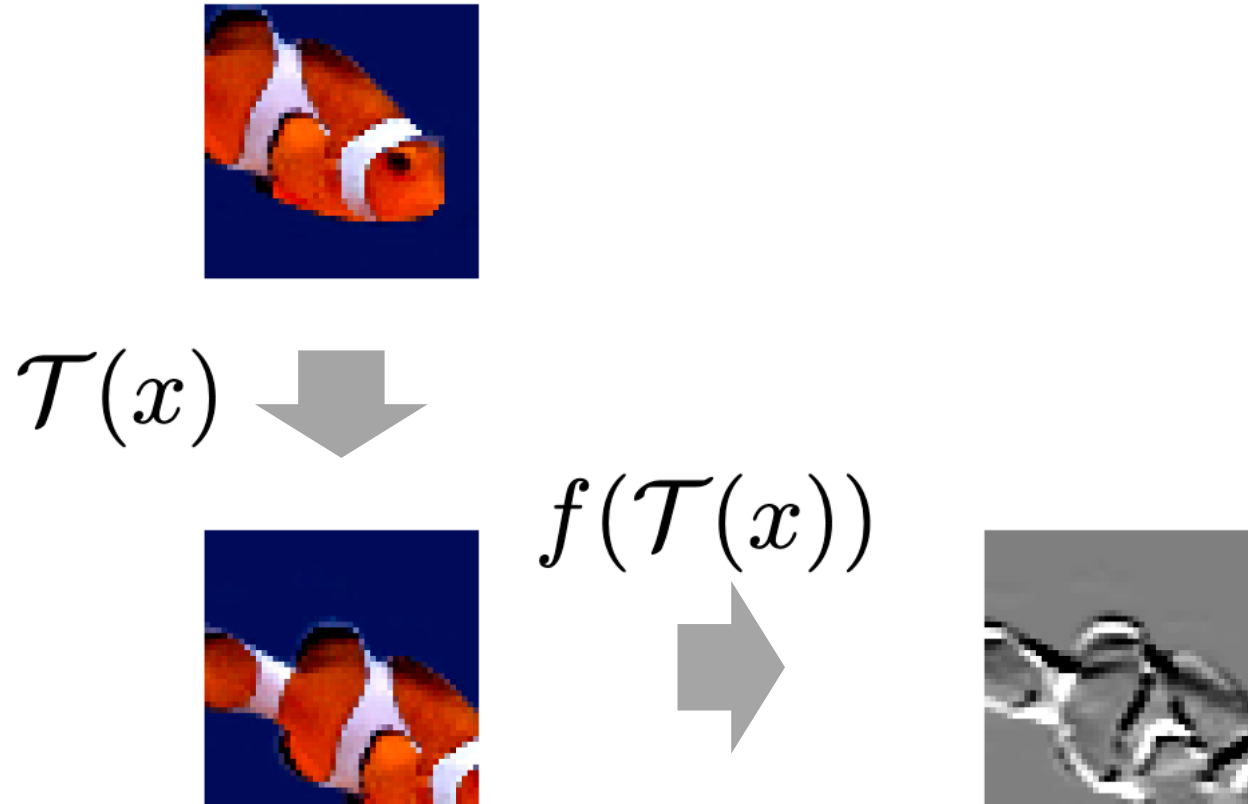


$\mathcal{T}(x)$ 



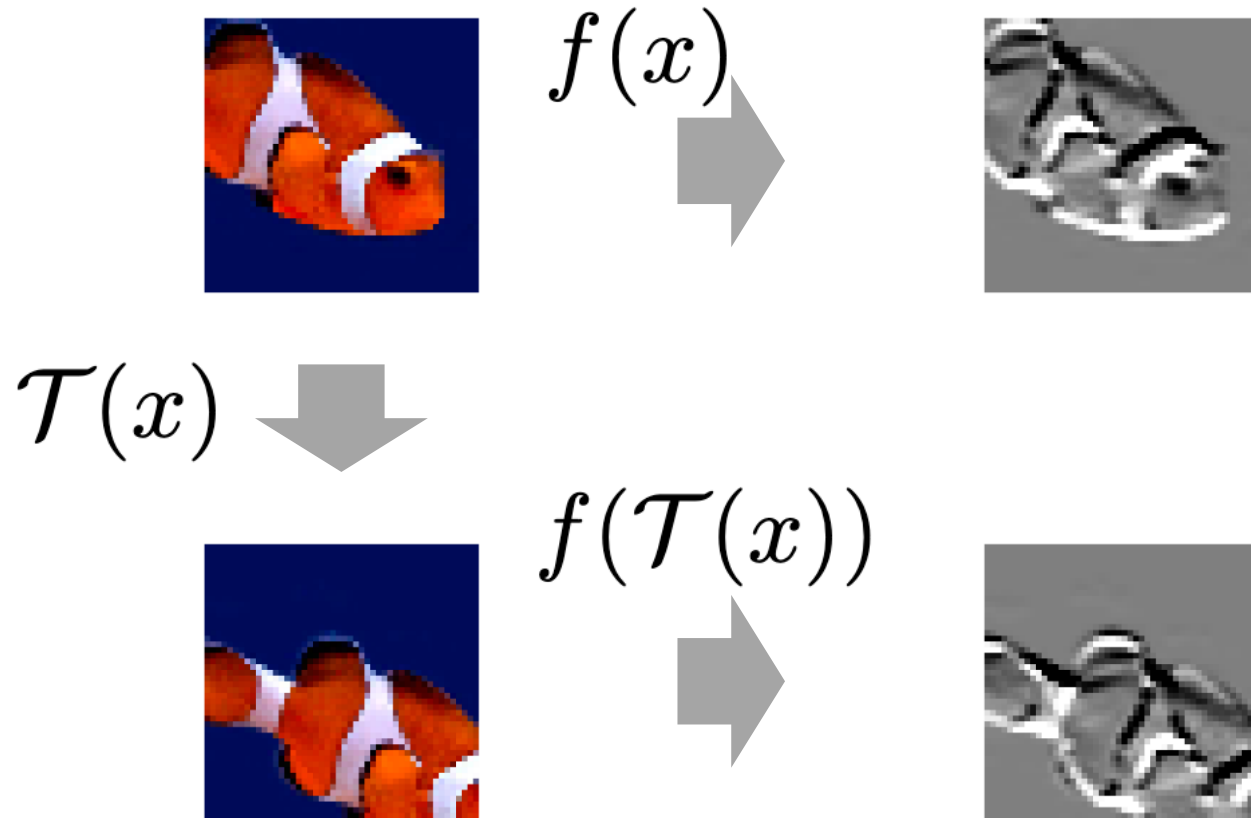
Convolution: translation-equivariance

- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}(f(x))$



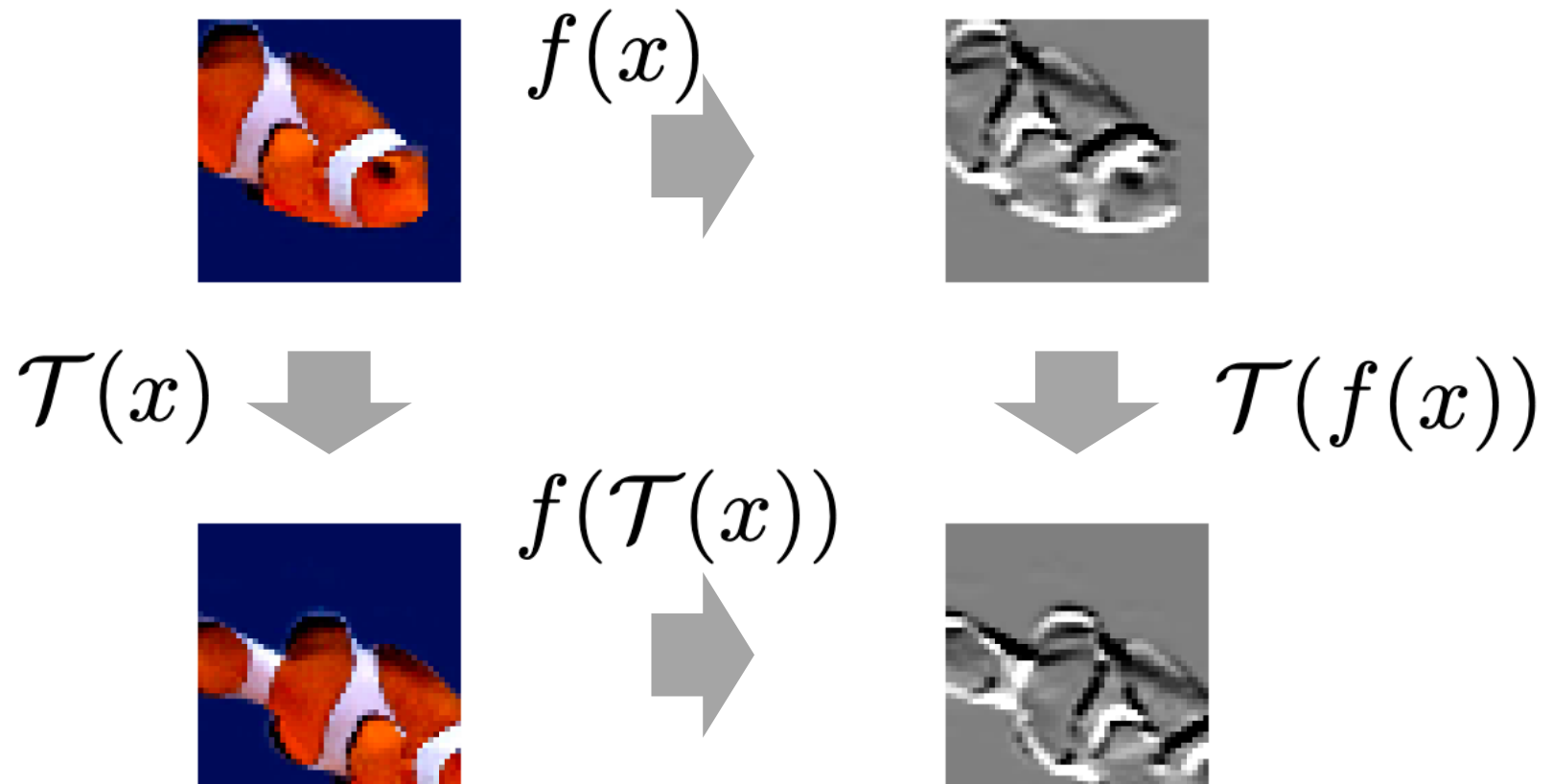
Convolution: translation-equivariance

- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}(f(x))$



Convolution: translation-equivariance

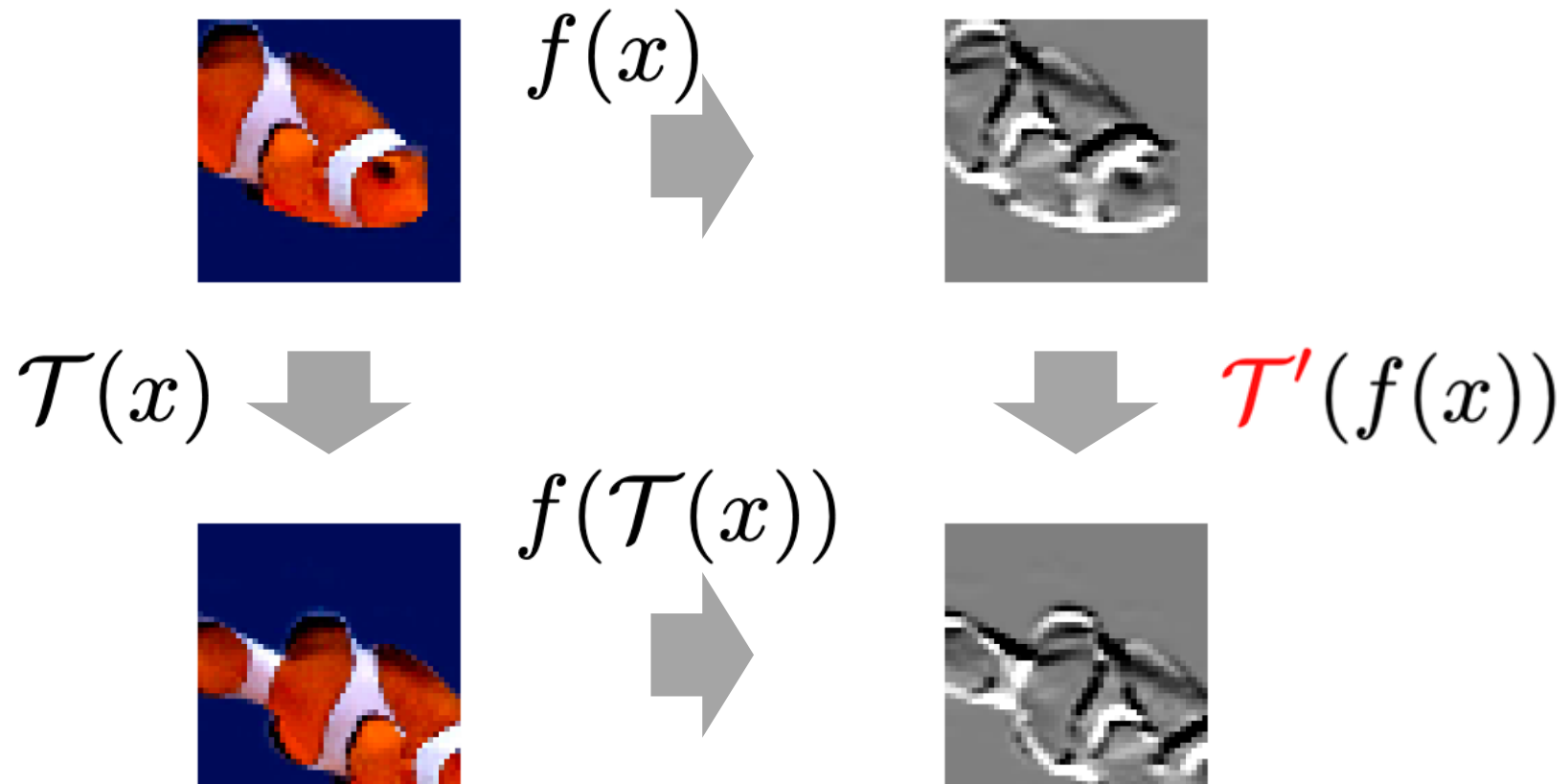
- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}(f(x))$



* assuming infinite canvas

Convolution: translation-equivariance

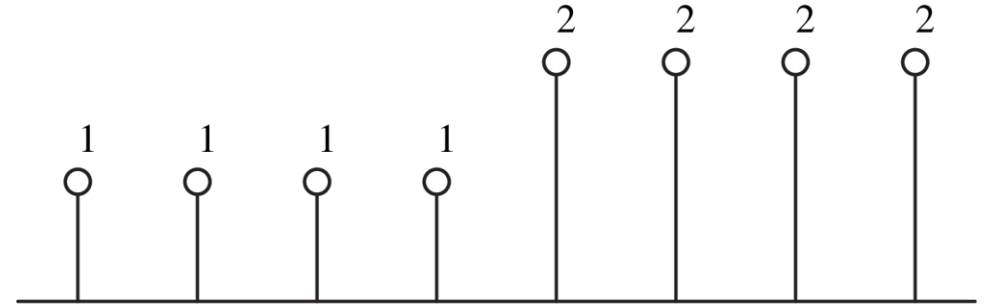
- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}'(f(x))$ $\mathcal{T}'$ does not depend on x



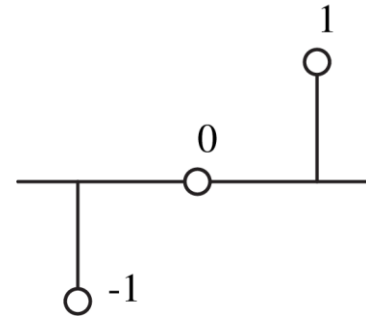
* assuming infinite canvas

Convolution = local connection + weight-sharing

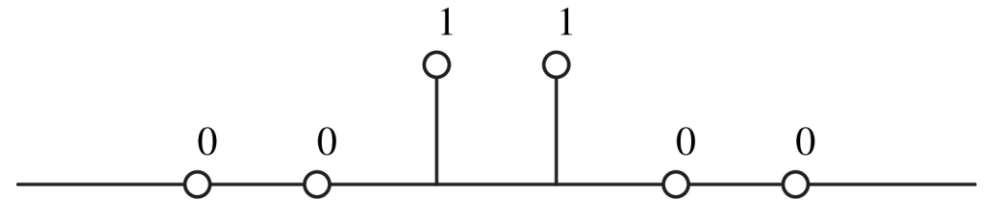
input



filter

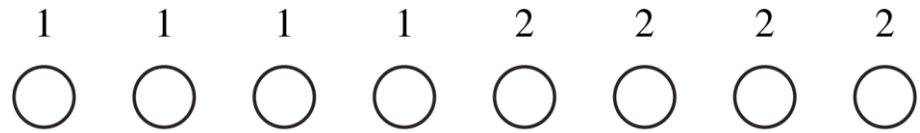


output

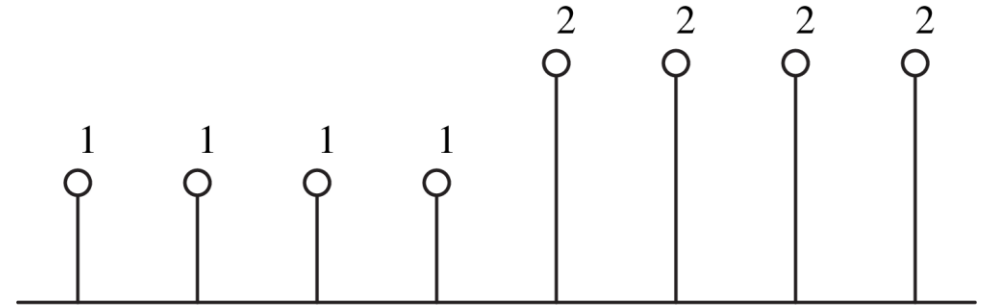


Convolution = local connection + weight-sharing

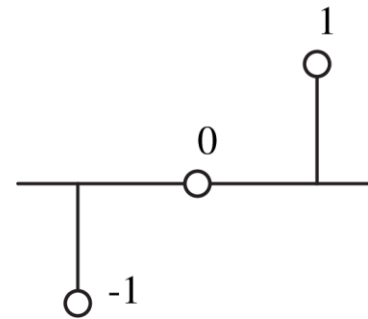
Let's plot it in a different way!



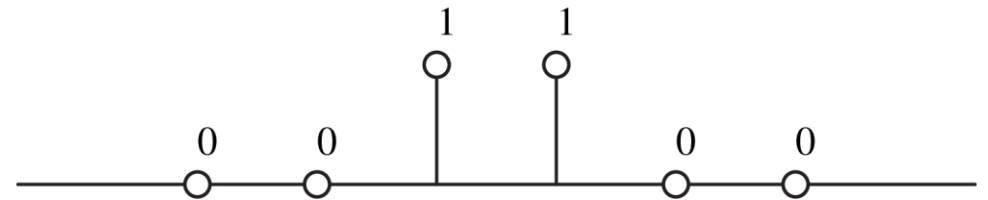
input



filter

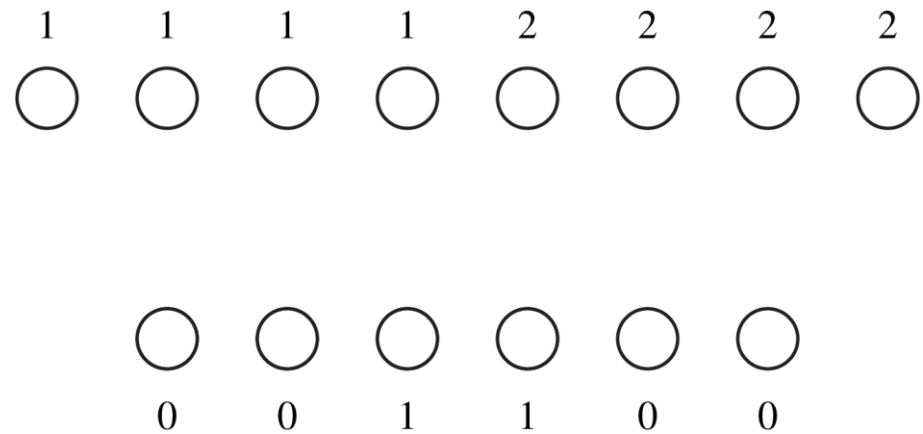


output

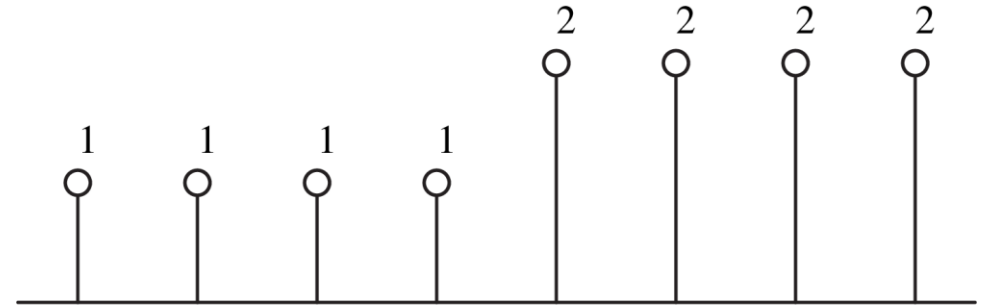


Convolution = local connection + weight-sharing

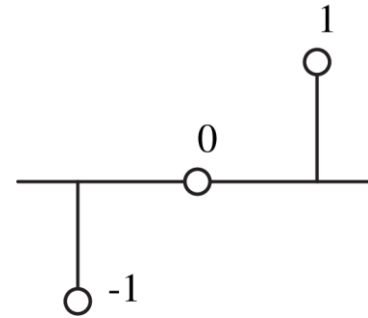
Let's plot it in a different way!



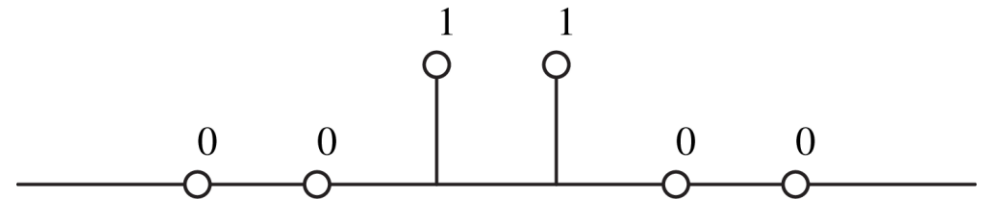
input



filter

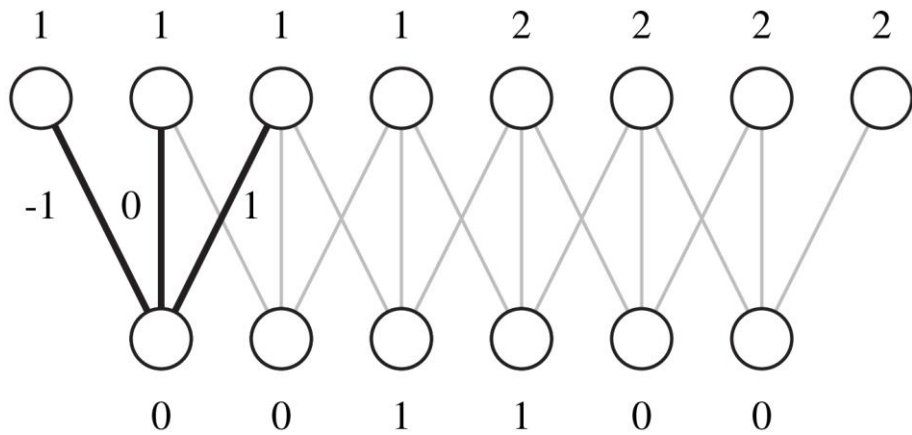


output

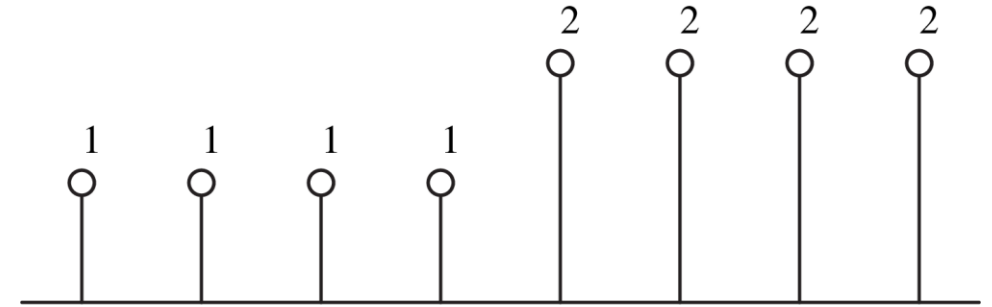


Convolution = local connection + weight-sharing

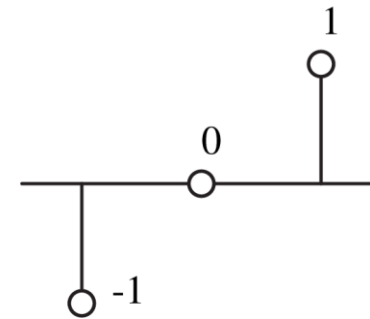
Let's plot it in a different way!



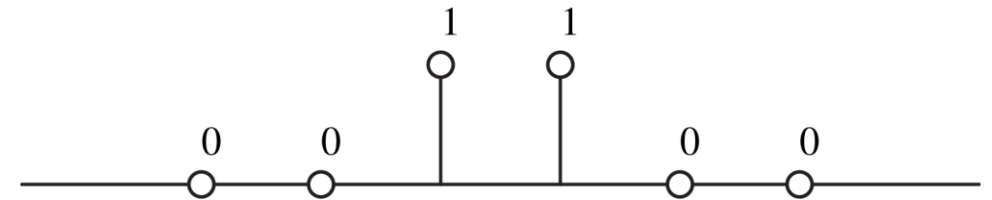
input



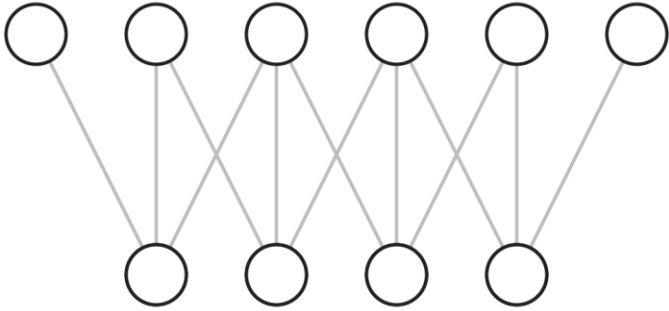
filter



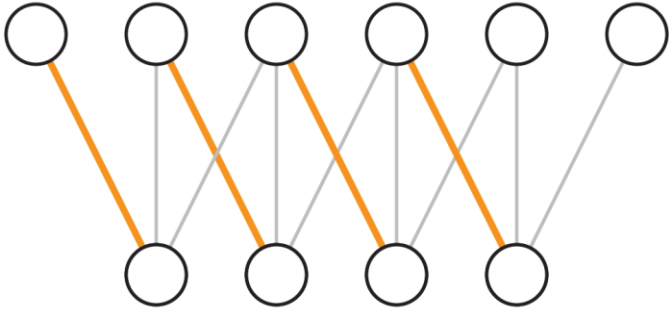
output



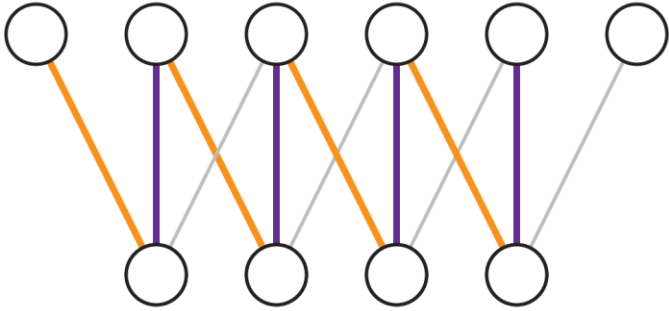
Convolution = local connection + weight-sharing



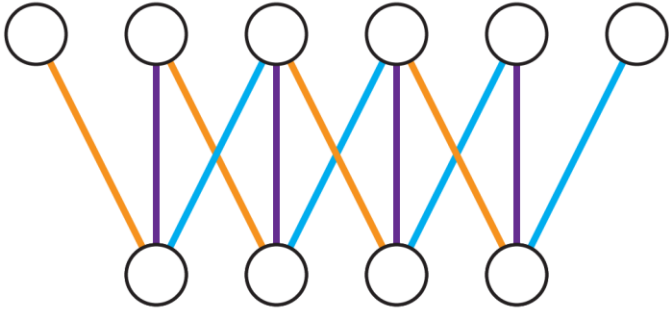
Convolution = local connection + weight-sharing



Convolution = local connection + weight-sharing

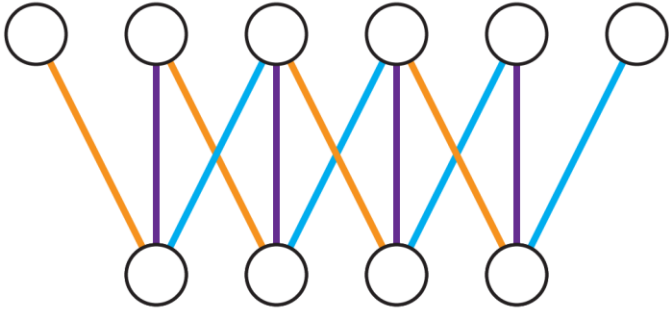


Convolution = local connection + weight-sharing



Convolution = local connection + weight-sharing

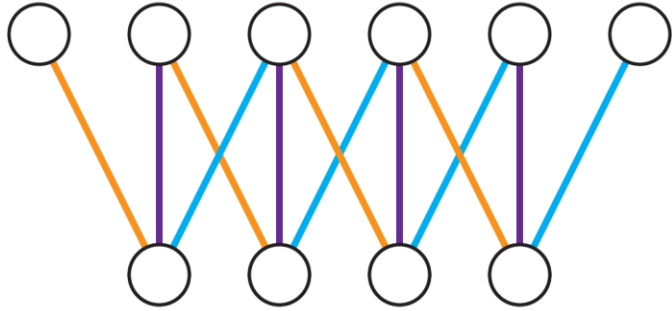
convolution



params = 3 here

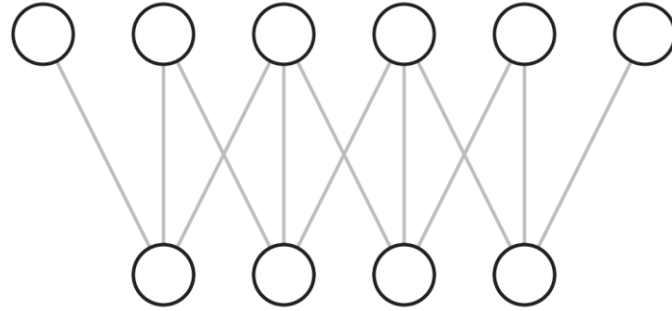
Convolution = local connection + weight-sharing

convolution



params = 3 here

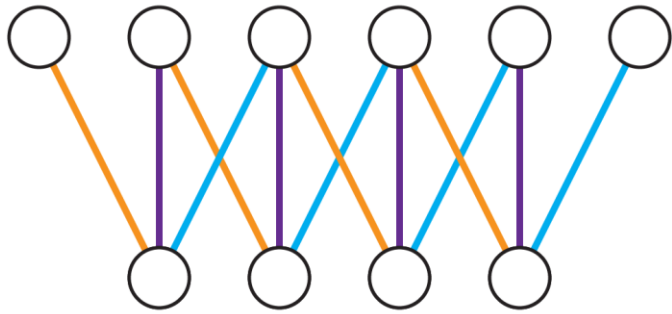
locally connected



params = 12 here
no weight-sharing

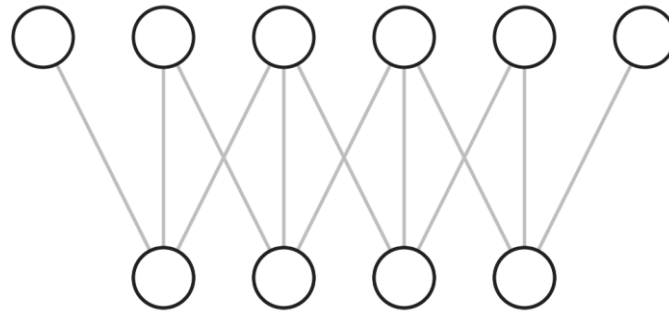
Convolution = local connection + weight-sharing

convolution



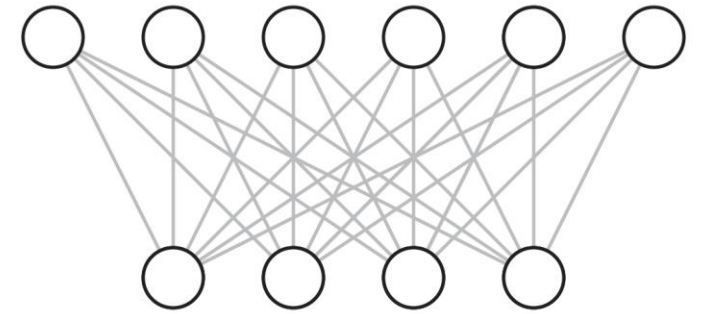
params = 3 here

locally connected



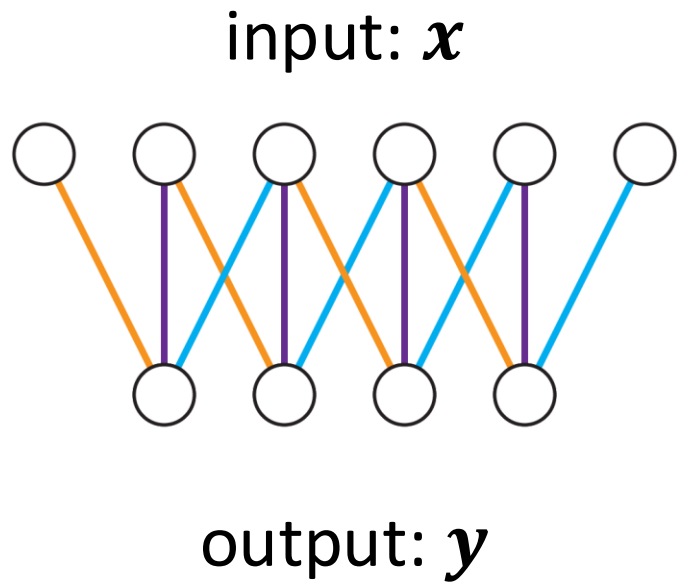
params = 12 here
no weight-sharing

fully connected

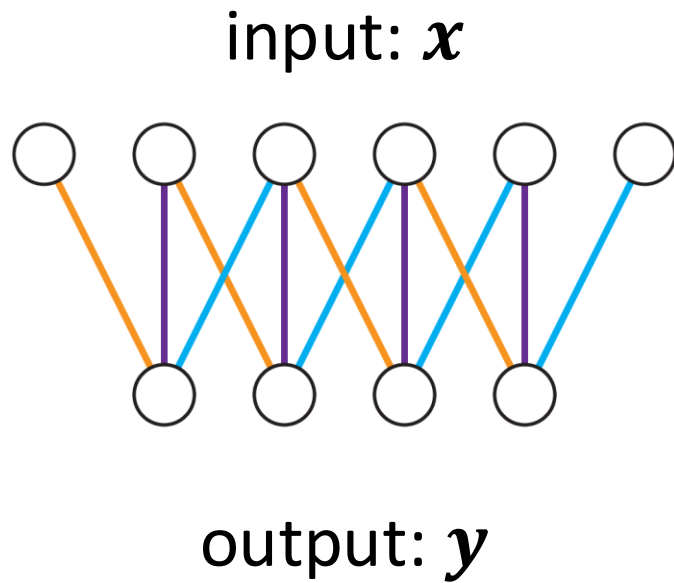


params = 24 here
no weight-sharing

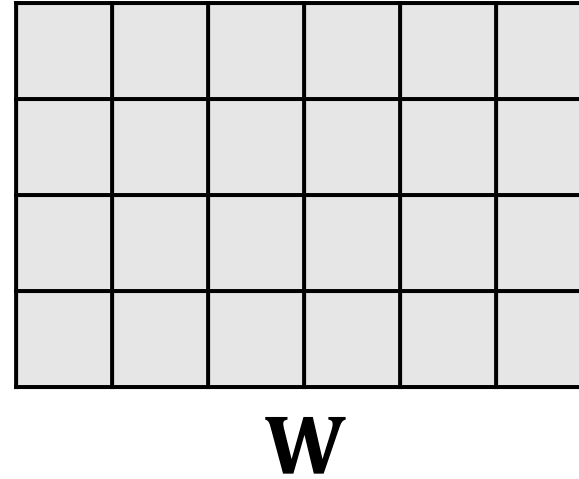
Convolution: linear transform



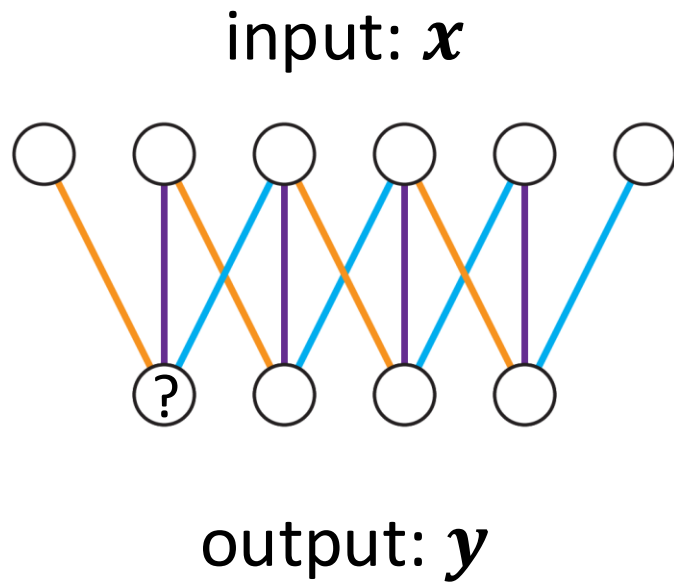
Convolution: linear transform



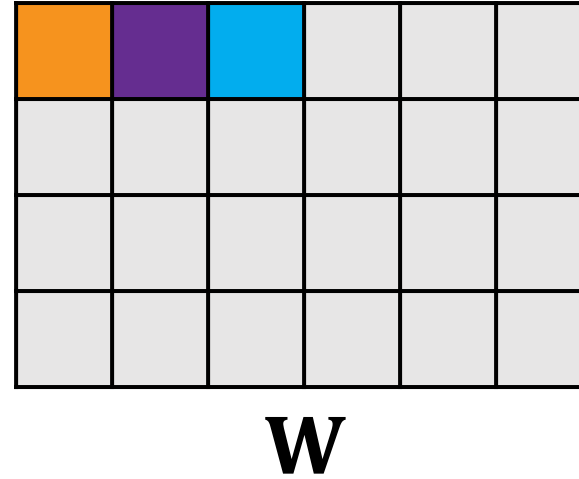
=



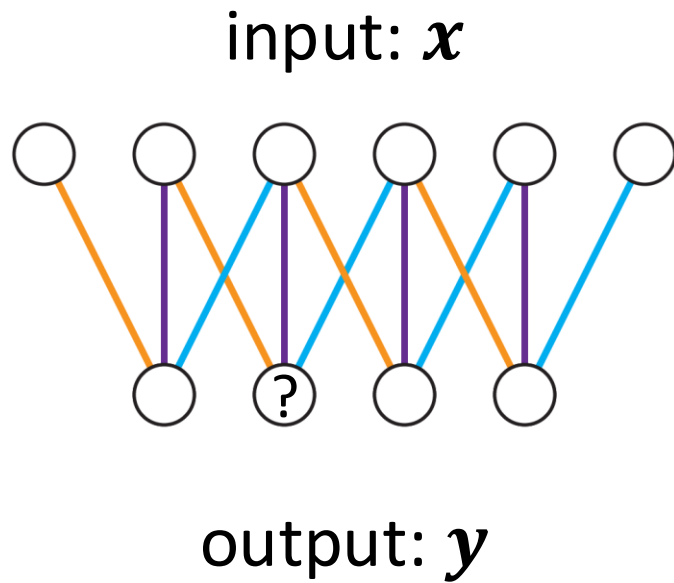
Convolution: linear transform



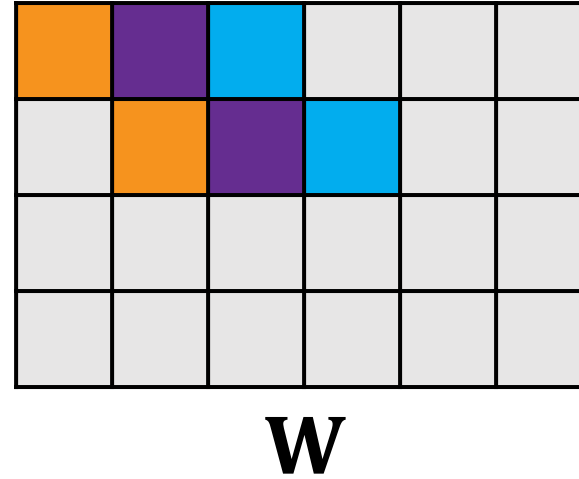
=



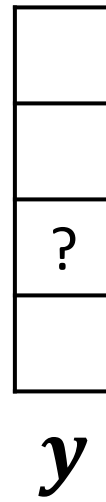
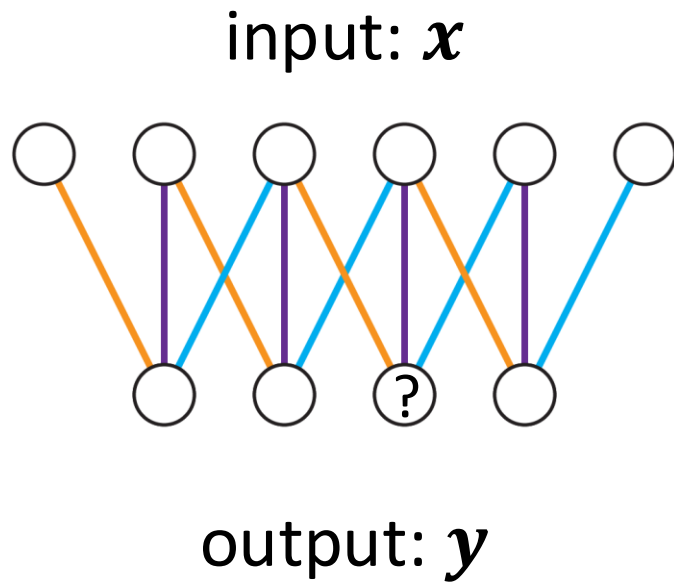
Convolution: linear transform



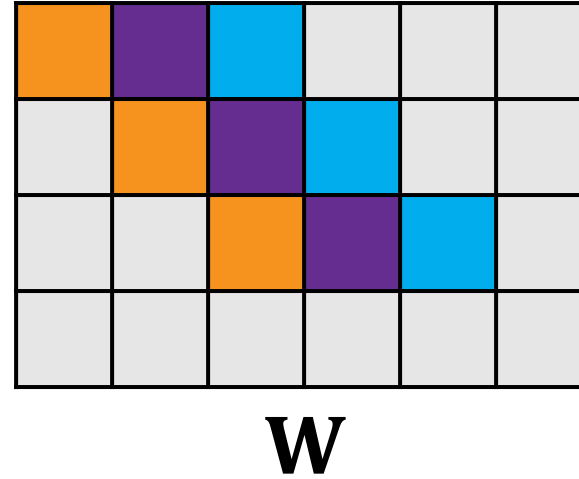
=



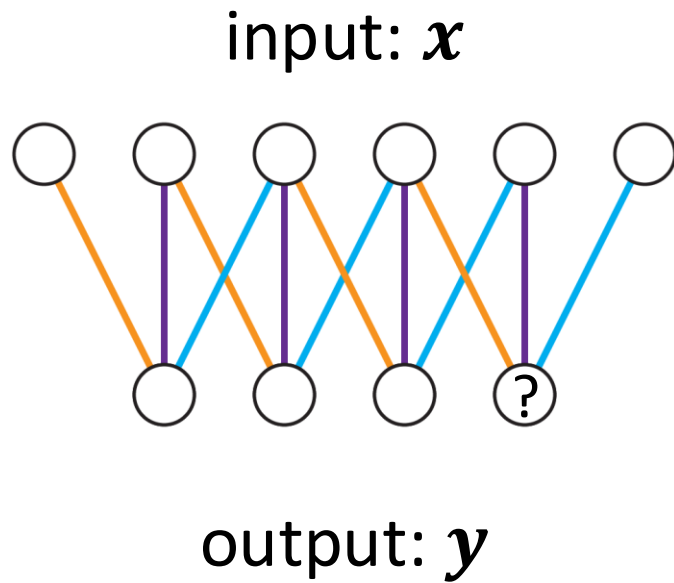
Convolution: linear transform



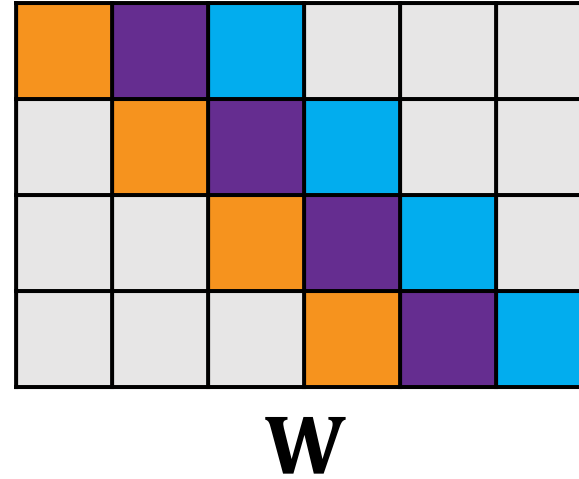
=



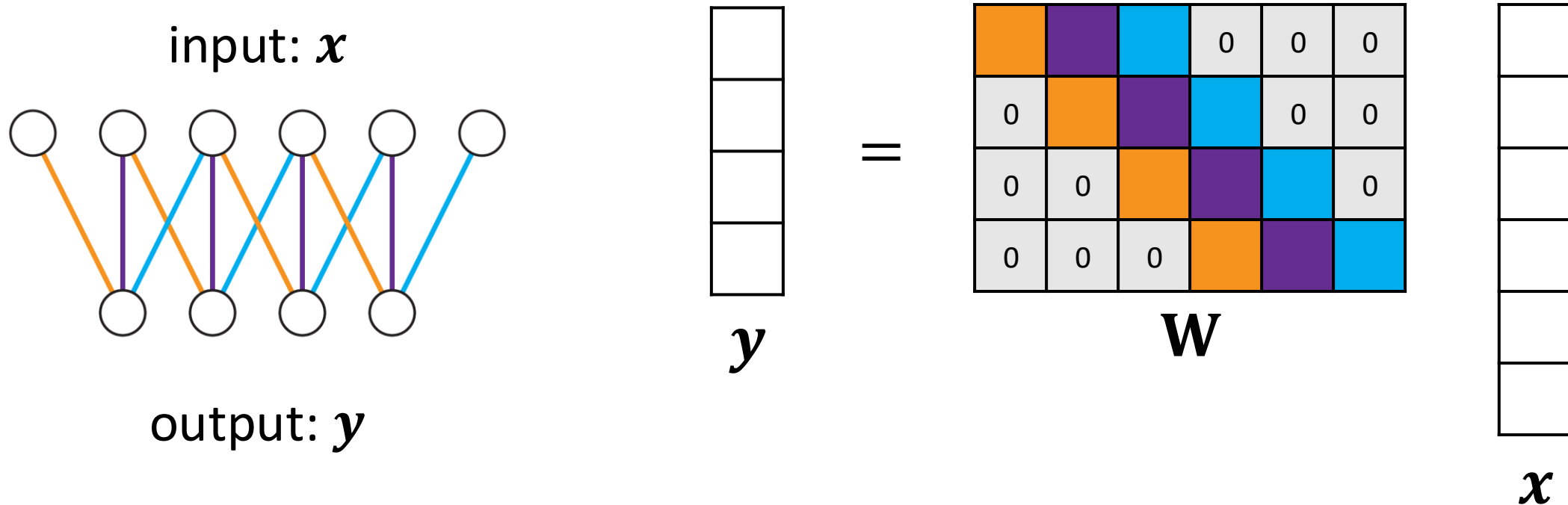
Convolution: linear transform



=

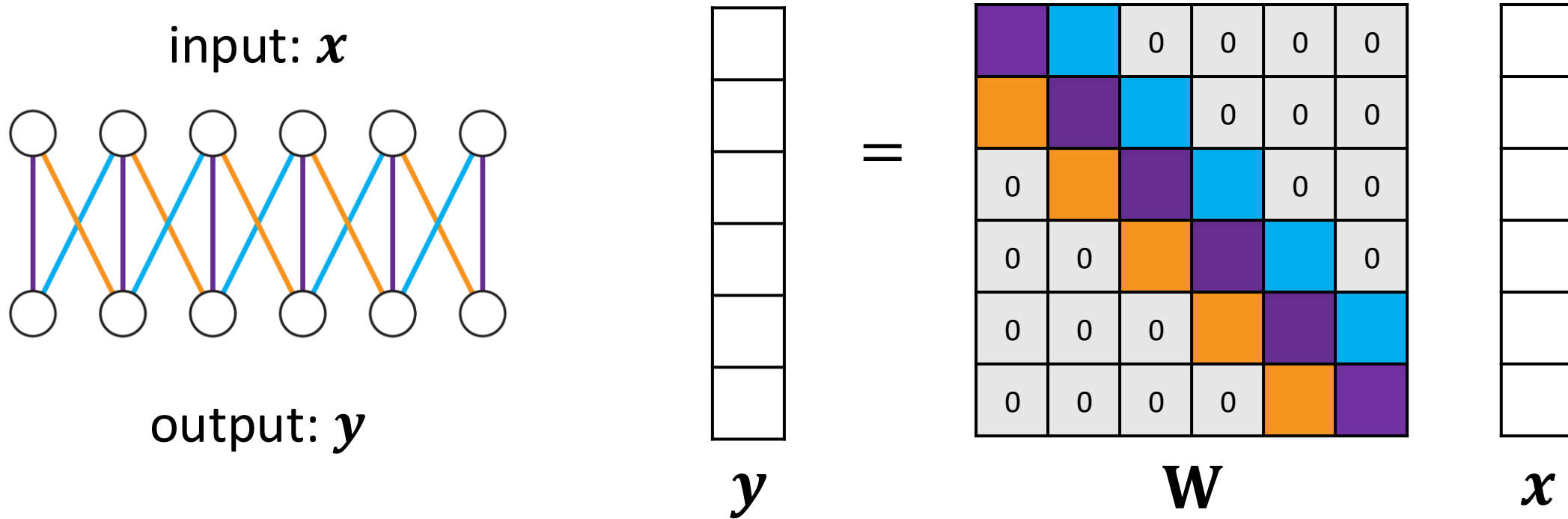


Convolution: linear transform



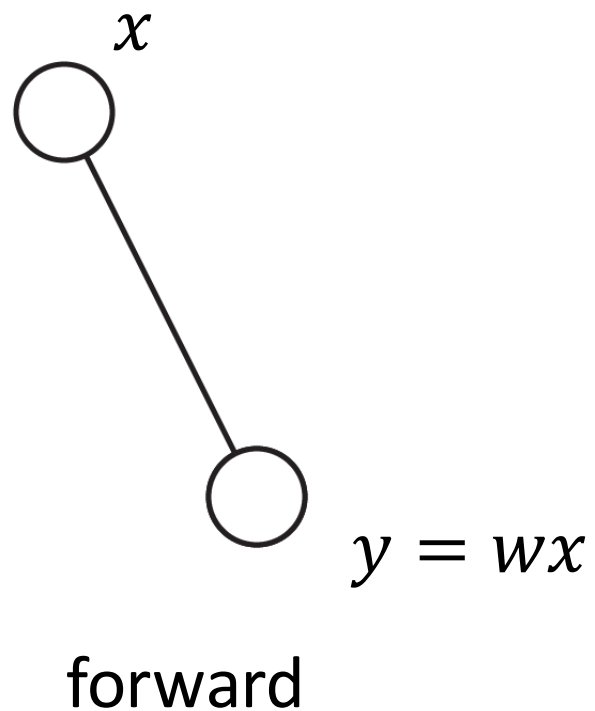
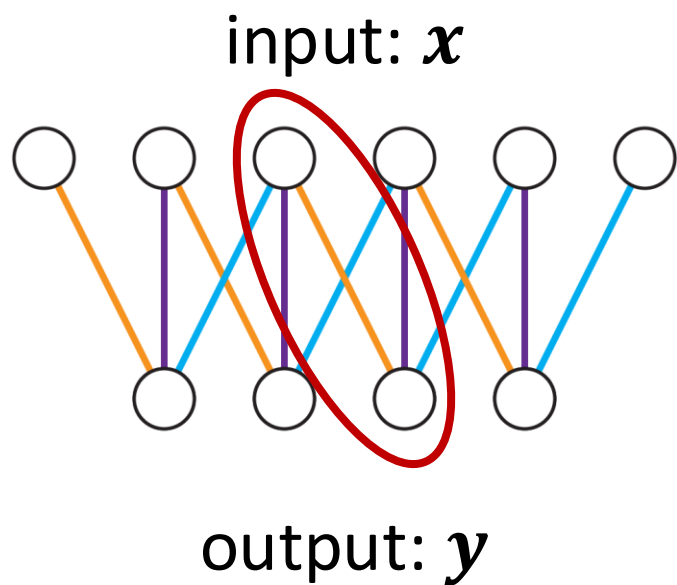
- **local connection:** W is sparse
- **weight-sharing:** # params = 3 here in W

Convolution: linear transform

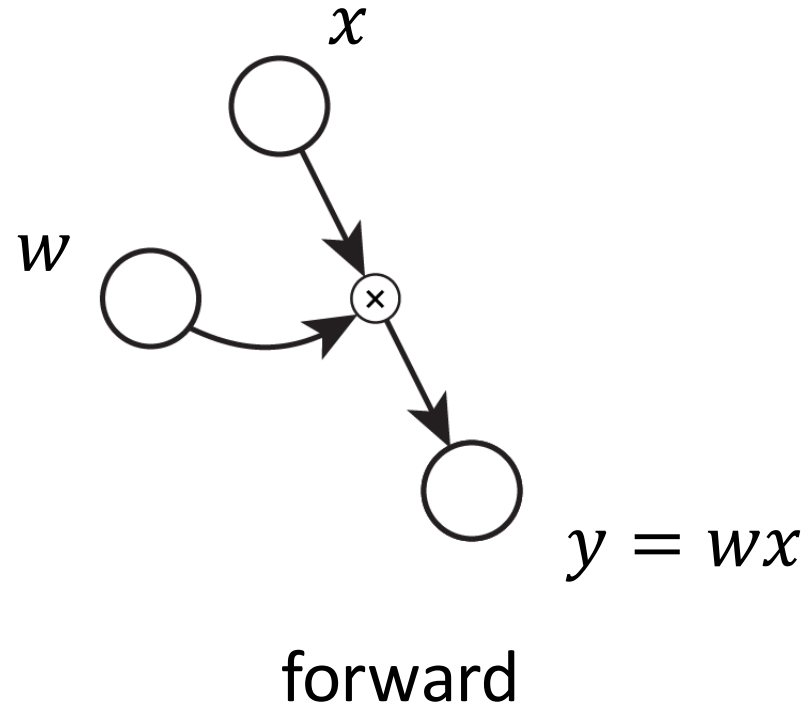
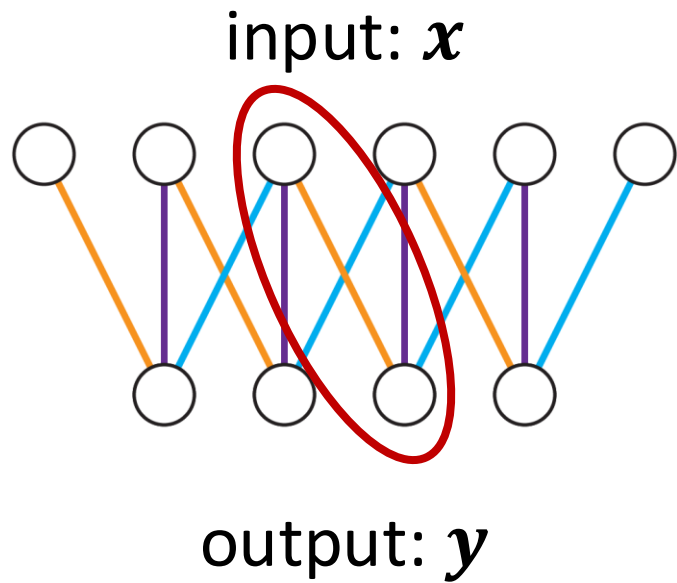


- This is the case with padding
- *Exercise:* what about stride = 2?
- *Exercise:* what about 2-D convolution?

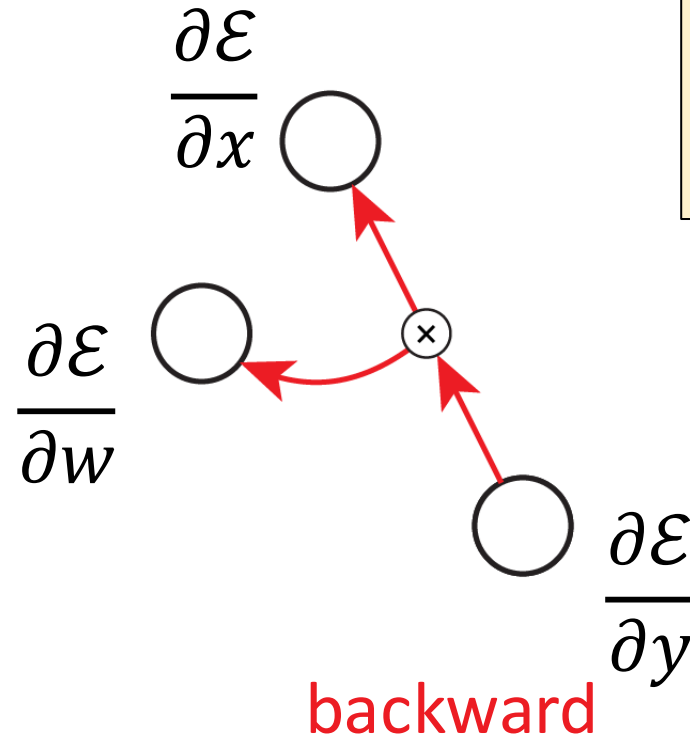
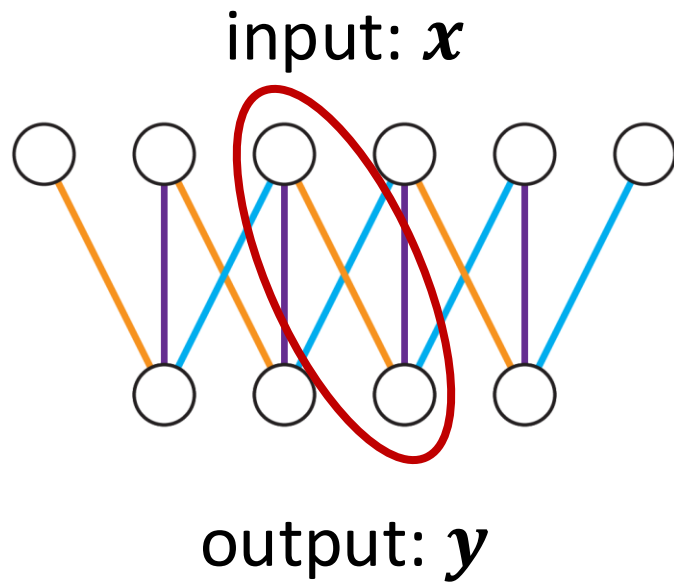
Convolution: BackProp



Convolution: BackProp



Convolution: BackProp



chain rule (scalars):

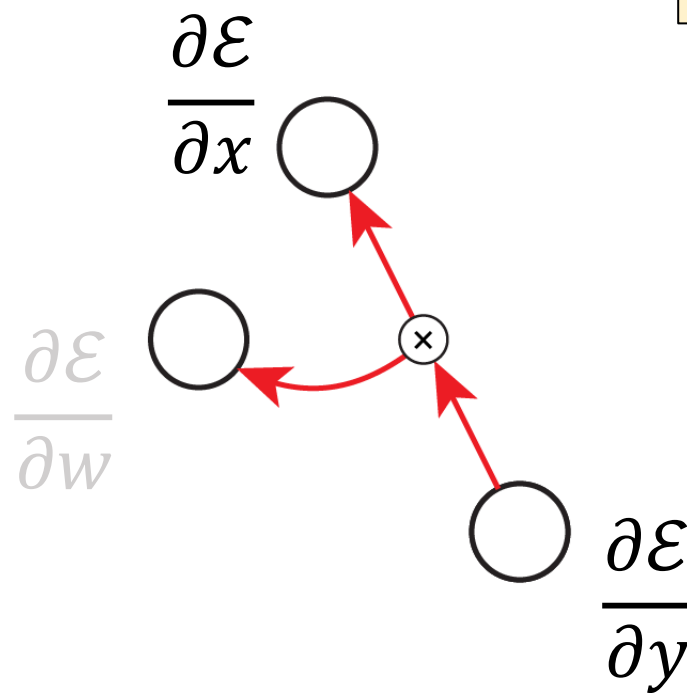
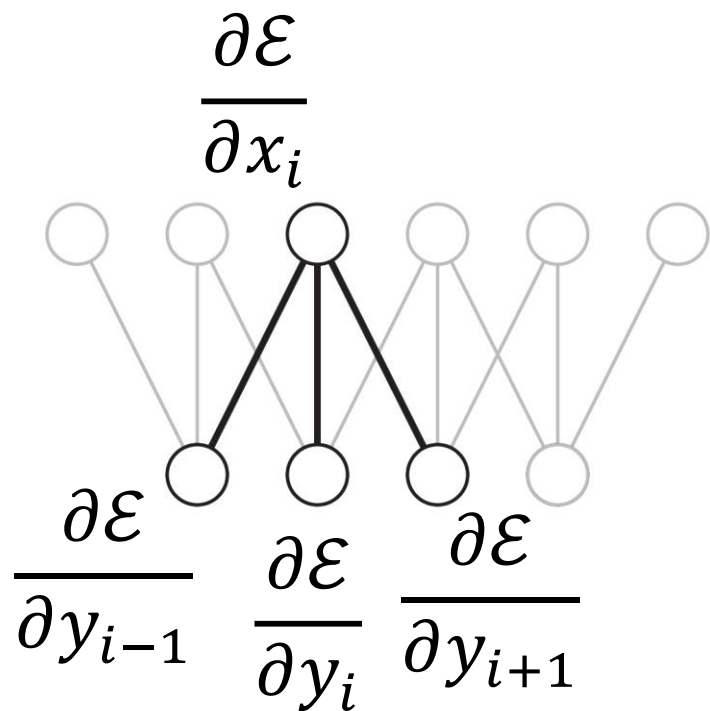
$$\frac{\partial \mathcal{E}}{\partial x} = \frac{\partial \mathcal{E}}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial \mathcal{E}}{\partial y} w$$

$$\frac{\partial \mathcal{E}}{\partial w} = \frac{\partial \mathcal{E}}{\partial y} \frac{\partial y}{\partial w} = \frac{\partial \mathcal{E}}{\partial y} x$$

Convolution: BackProp

chain rule (scalars):

$$\frac{\partial \mathcal{E}}{\partial x} = \frac{\partial \mathcal{E}}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial \mathcal{E}}{\partial y} w$$

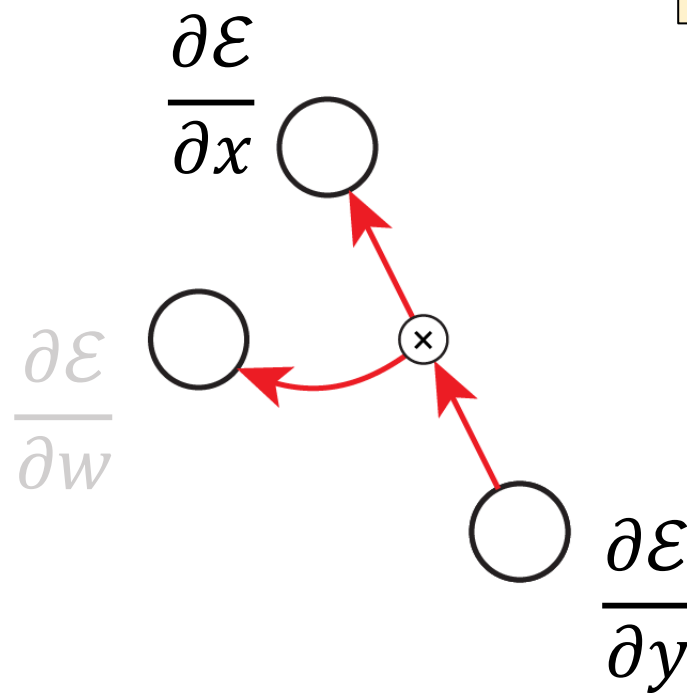


Convolution: BackProp

chain rule (scalars):

$$\frac{\partial \mathcal{E}}{\partial x} = \frac{\partial \mathcal{E}}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial \mathcal{E}}{\partial y} w$$

$$\frac{\partial \mathcal{E}}{\partial x_i} = \sum_{j=-1}^1 \frac{\partial \mathcal{E}}{\partial y_{i+j}} w_{-j}$$

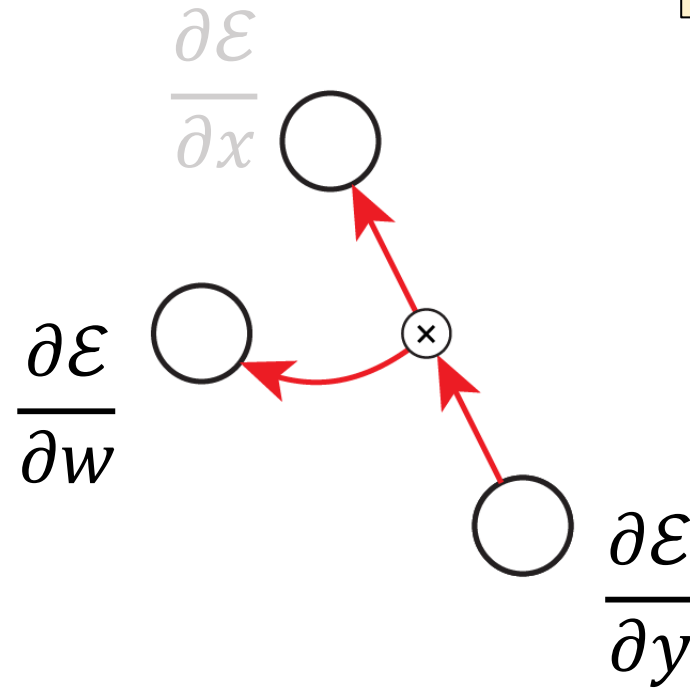
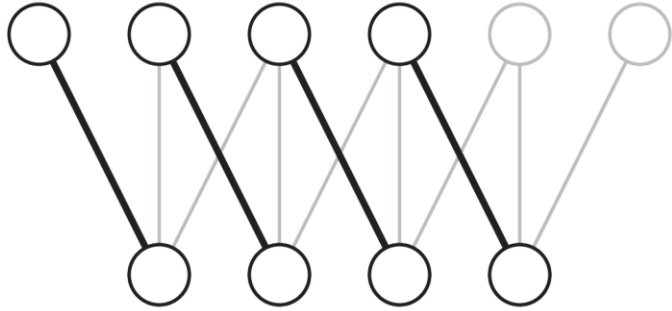


backprop wrt x is conv

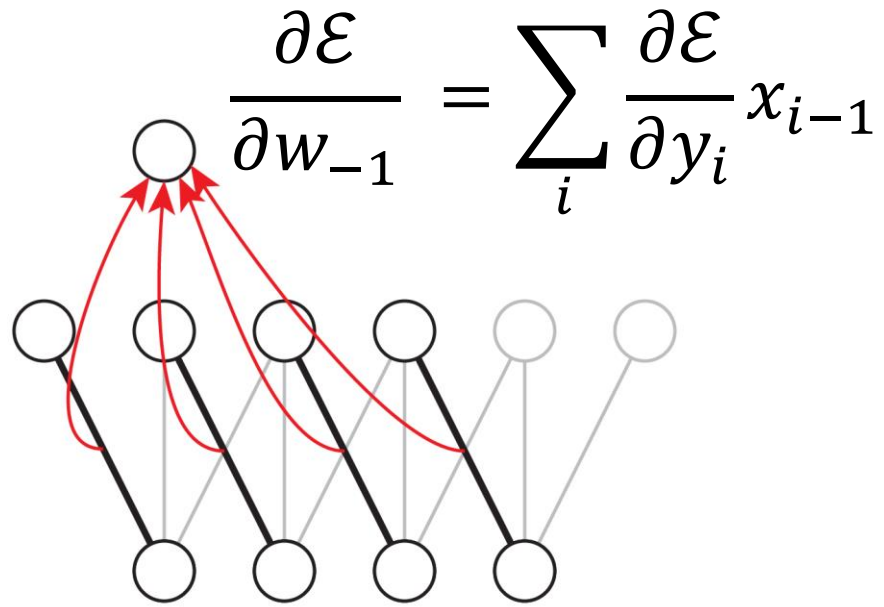
Convolution: BackProp

chain rule (scalars):

$$\frac{\partial \mathcal{E}}{\partial w} = \frac{\partial \mathcal{E}}{\partial y} \frac{\partial y}{\partial w} = \frac{\partial \mathcal{E}}{\partial y} x$$

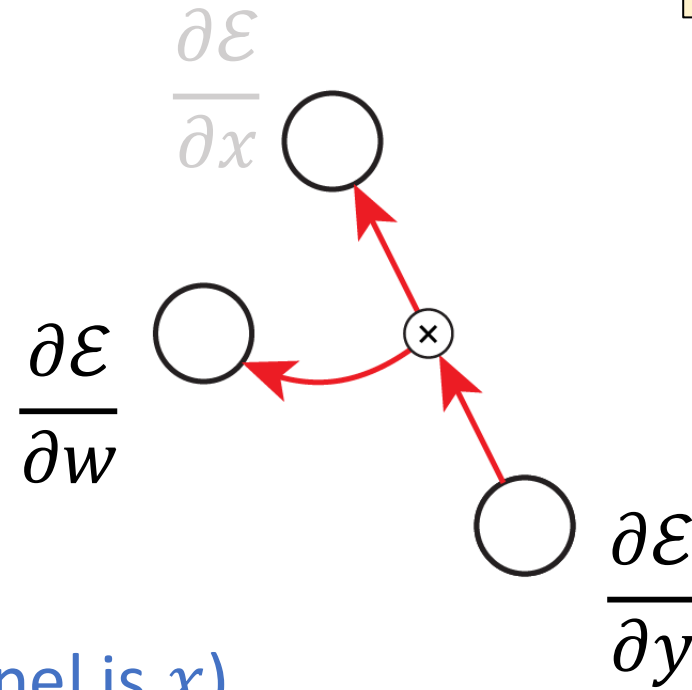


Convolution: BackProp



chain rule (scalars):

$$\frac{\partial \mathcal{E}}{\partial w} = \frac{\partial \mathcal{E}}{\partial y} \frac{\partial y}{\partial w} = \frac{\partial \mathcal{E}}{\partial y} x$$



backprop wrt w is also conv (kernel is x)

Exercise: derive the backprop in matrix forms

Takeaways: Convolution

- sliding window
- local connection + weight-sharing
- linear transform
- translation-equivariant
- backprop is also conv

Convolutional Neural Networks

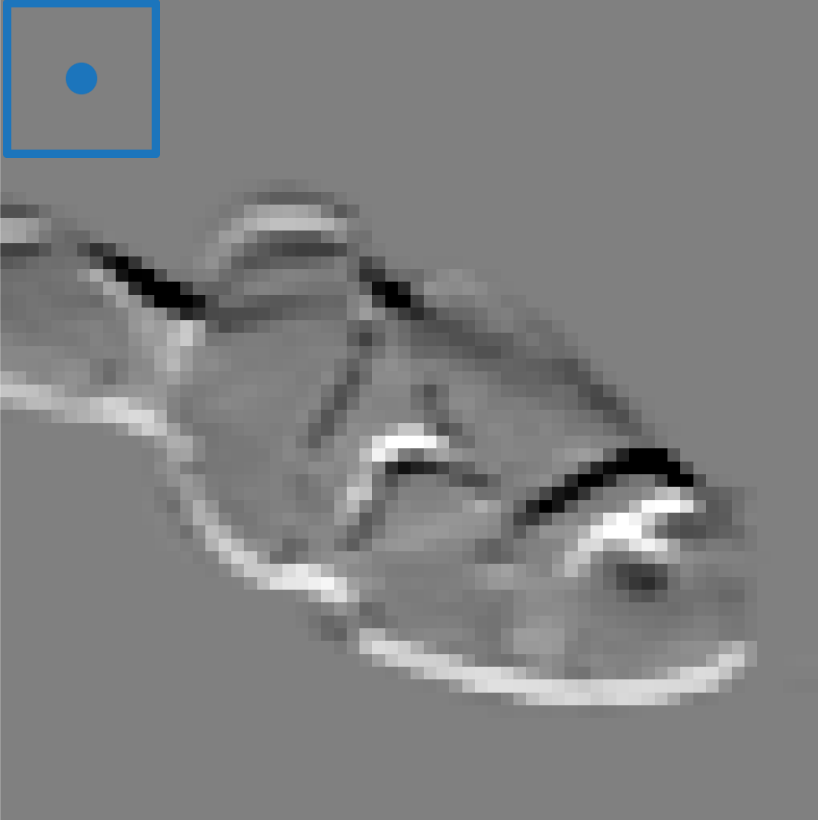
Convolutional Neural Network (ConvNet, CNN)

- **Core idea of deep learning:**
Composing basic operations into complex functions
- **Common operations of ConvNet**
 - convolution
 - activation function
 - pooling
 - fully-connected layer
 - softmax



Composing basic operations

output of prev. layer

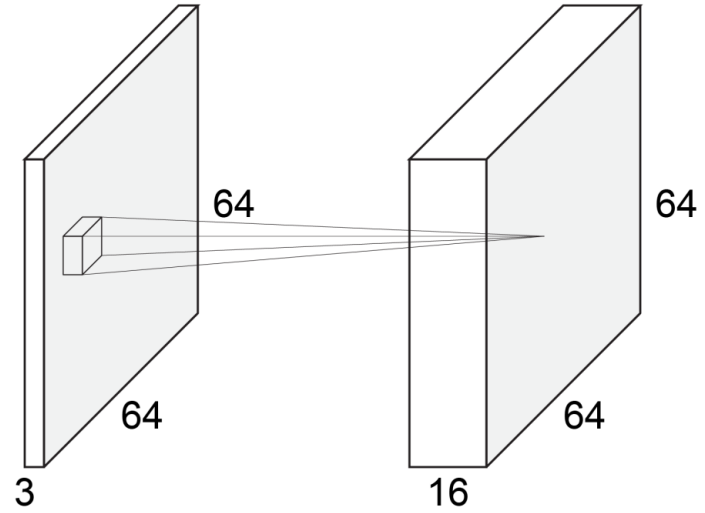


$$\begin{matrix} & \text{filter} \\ * & \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \\ & = \end{matrix}$$

output of this layer

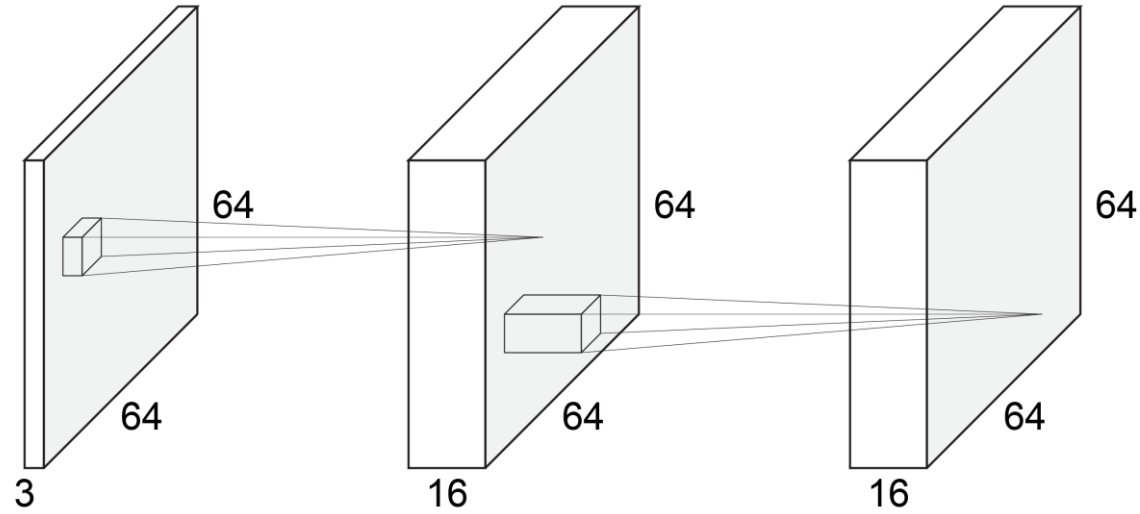


Composing basic operations



conv
3×3, 16

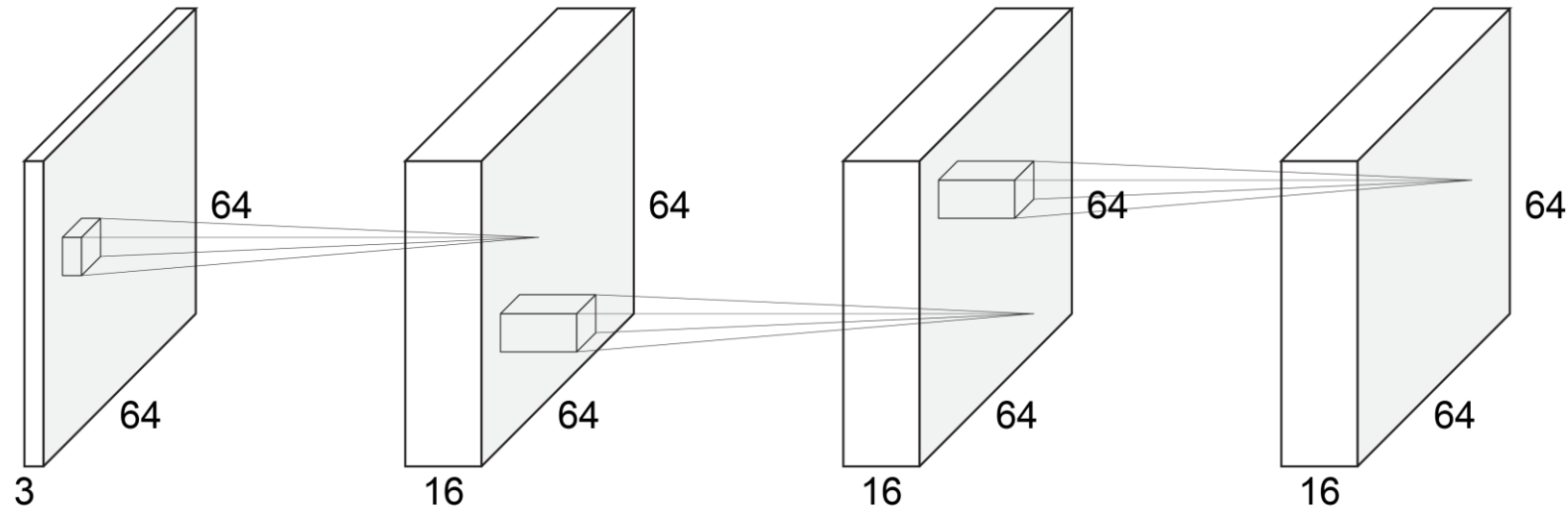
Composing basic operations



conv
3×3, 16

conv
3×3, 16

Composing basic operations

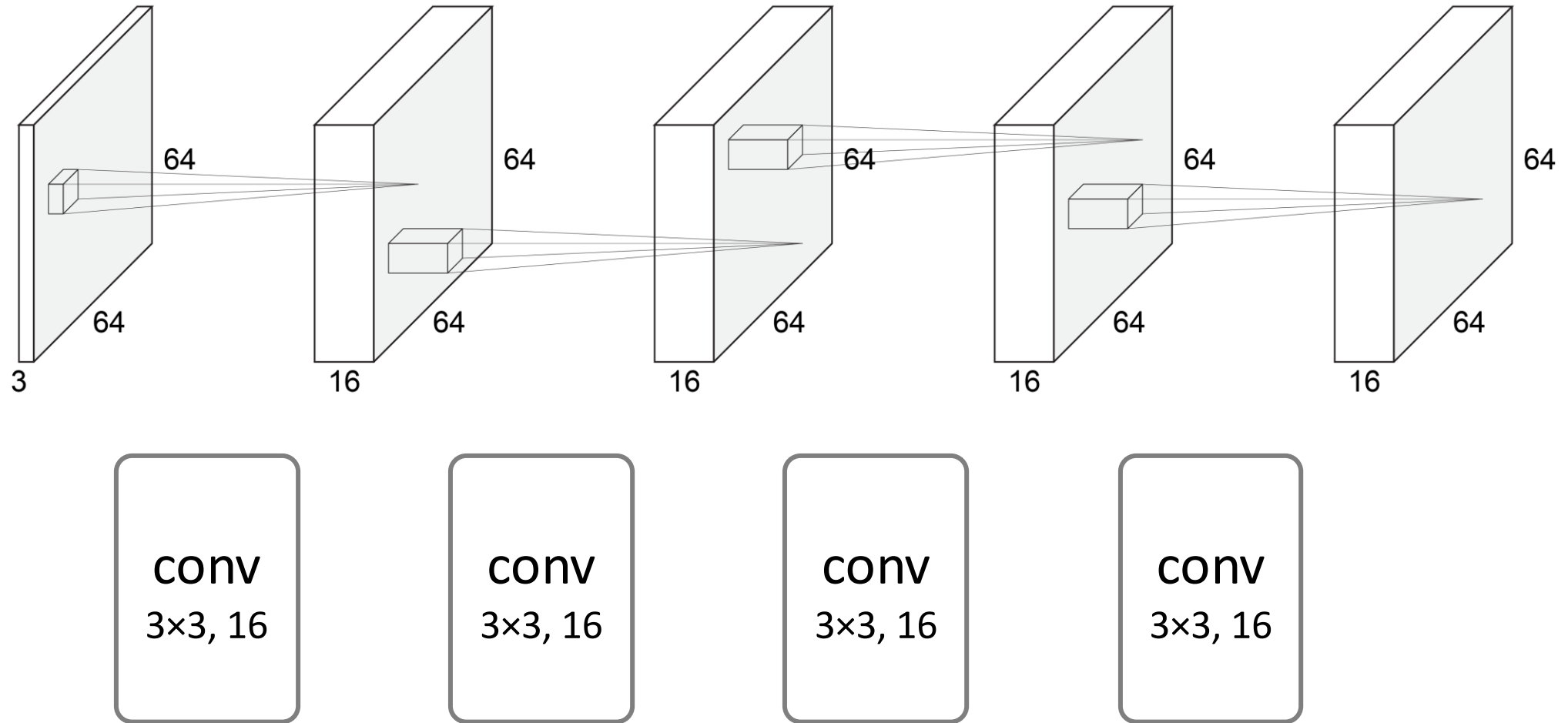


conv
3×3, 16

conv
3×3, 16

conv
3×3, 16

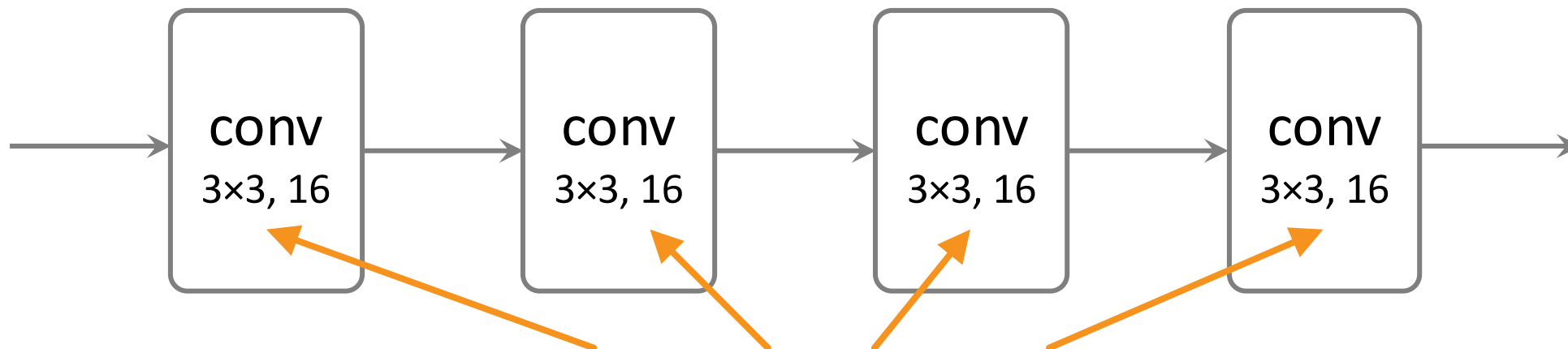
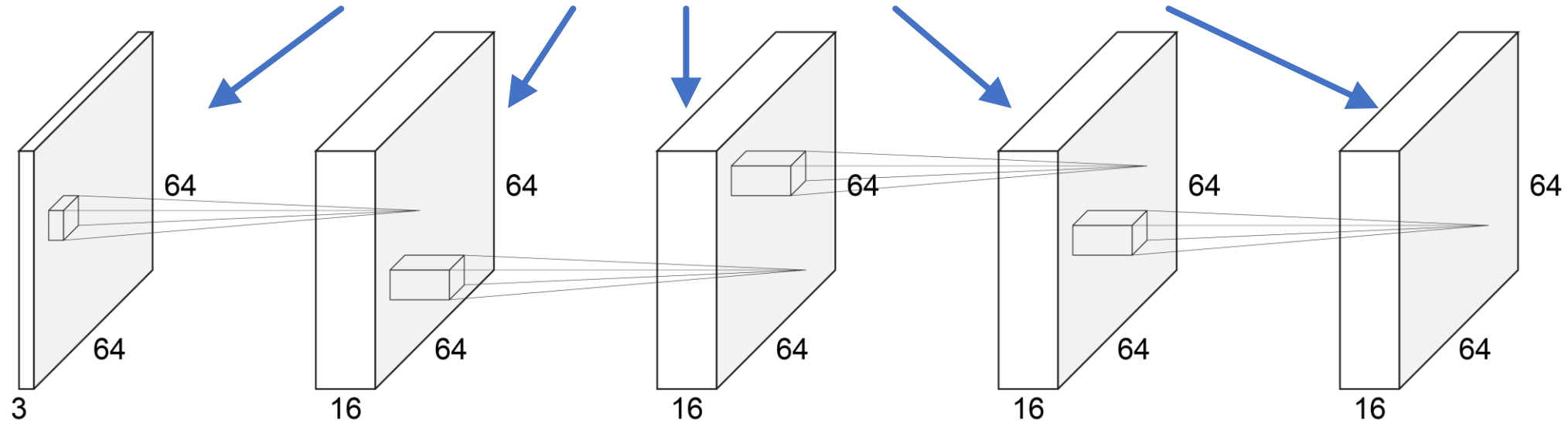
Composing basic operations



Composing basic operations

two ways of showing
neural nets

these are activations (features, embeddings, tensors ...)

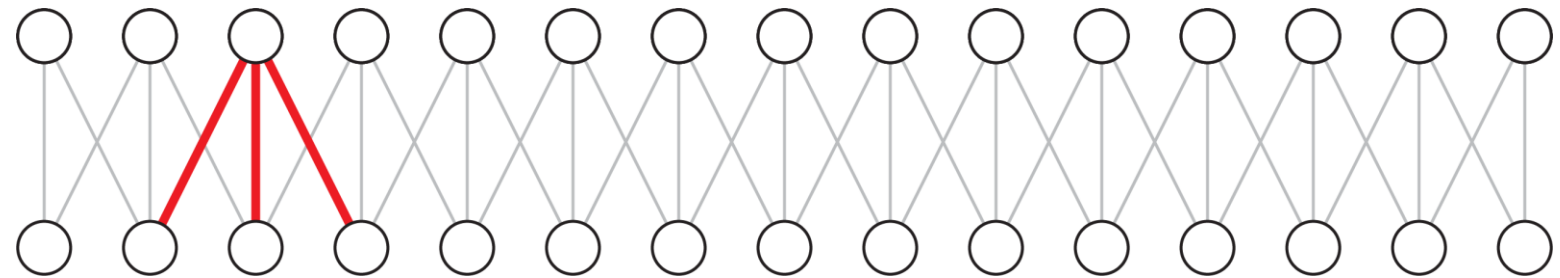


these are operations (functions, transforms, layers ...)

Receptive Field

- The input subset that can influence a particular unit's activation.

kernel = 3



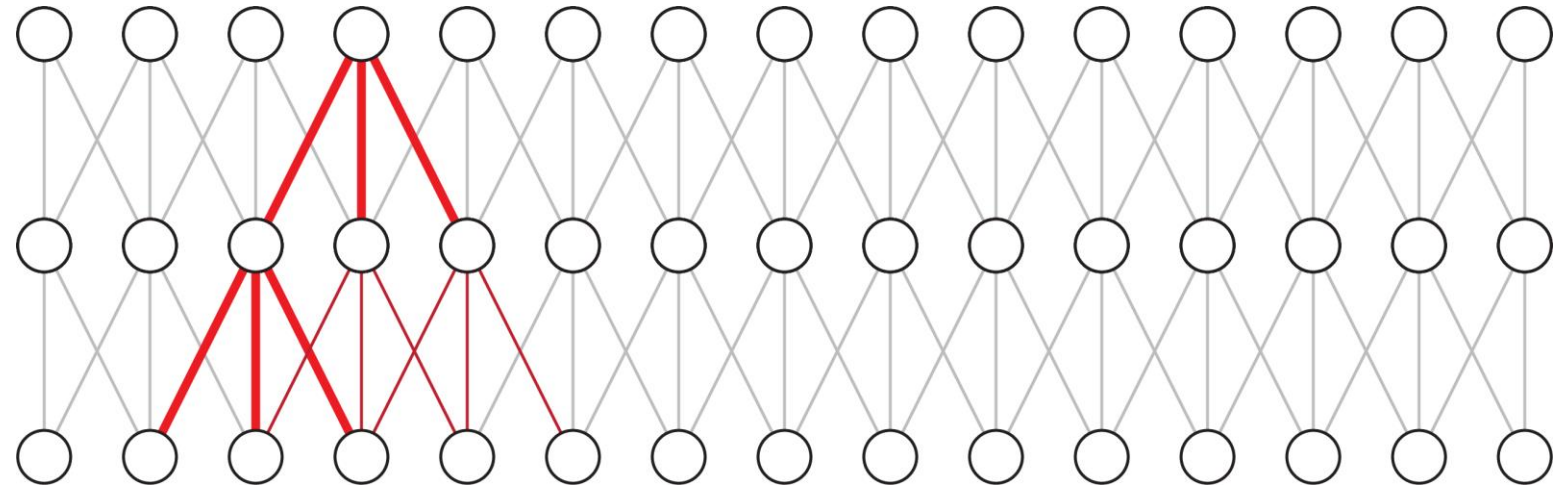
receptive field = 3

Receptive Field

- The input subset that can influence a particular unit's activation.

kernel = 3

kernel = 3



receptive field = 5

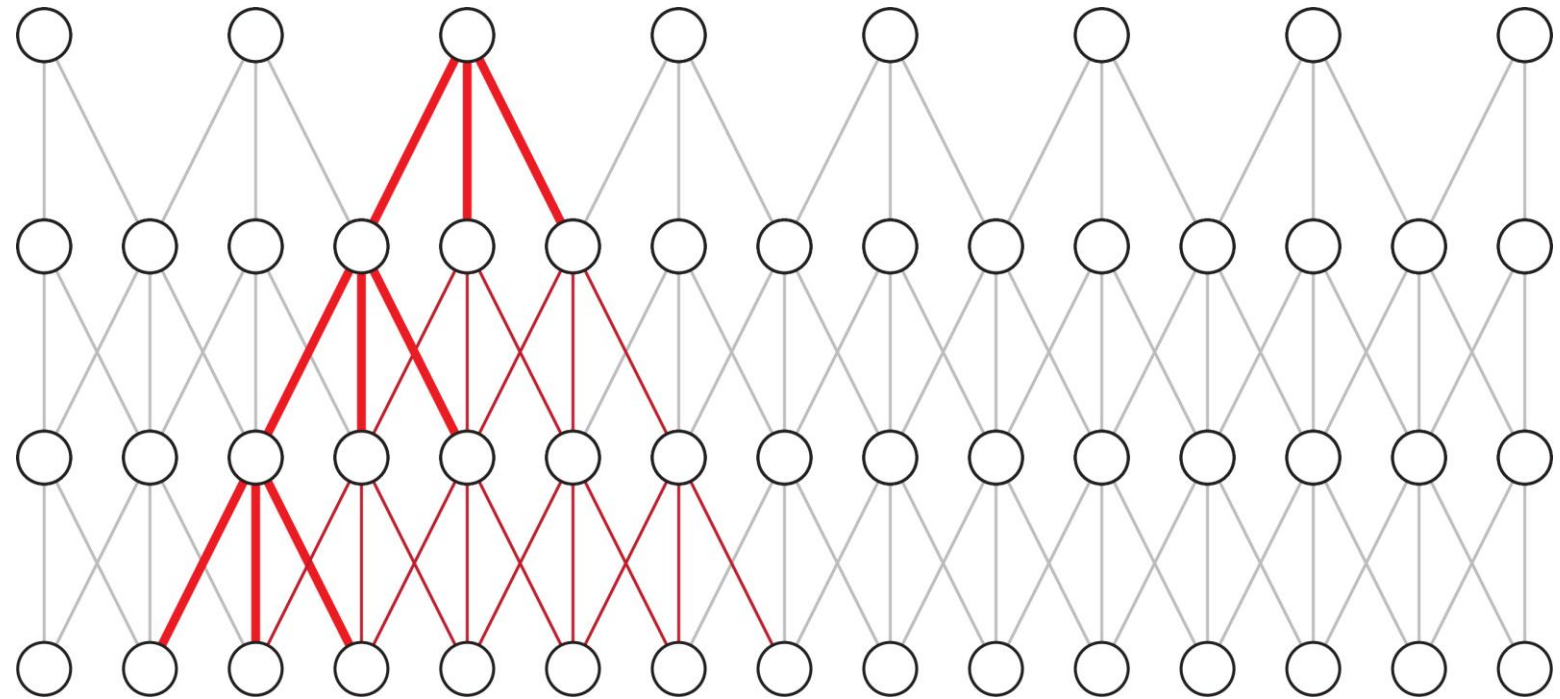
Receptive Field

- The input subset that can influence a particular unit's activation.

kernel = 3, str = 2

kernel = 3

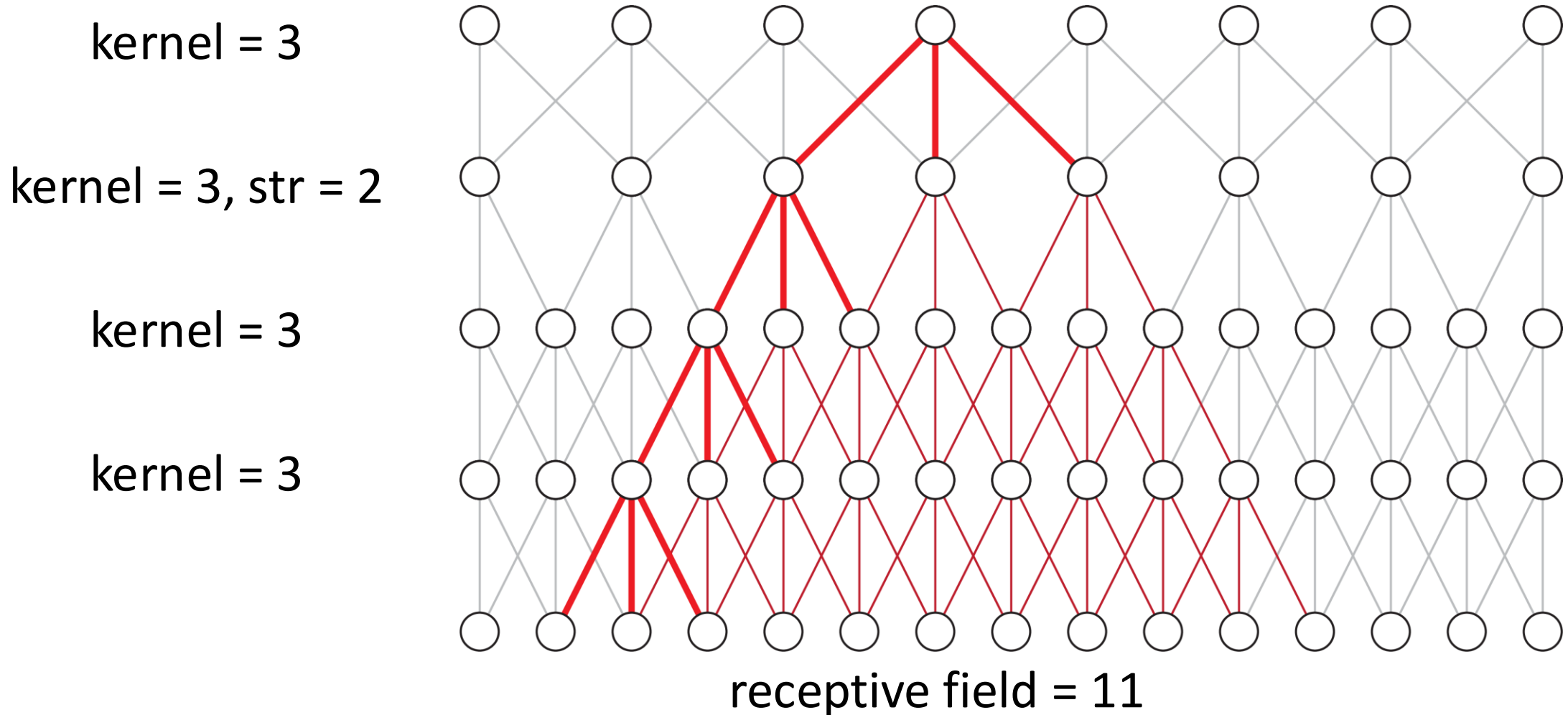
kernel = 3



receptive field = 7

Receptive Field

- The input subset that can influence a particular unit's activation.



Pooling

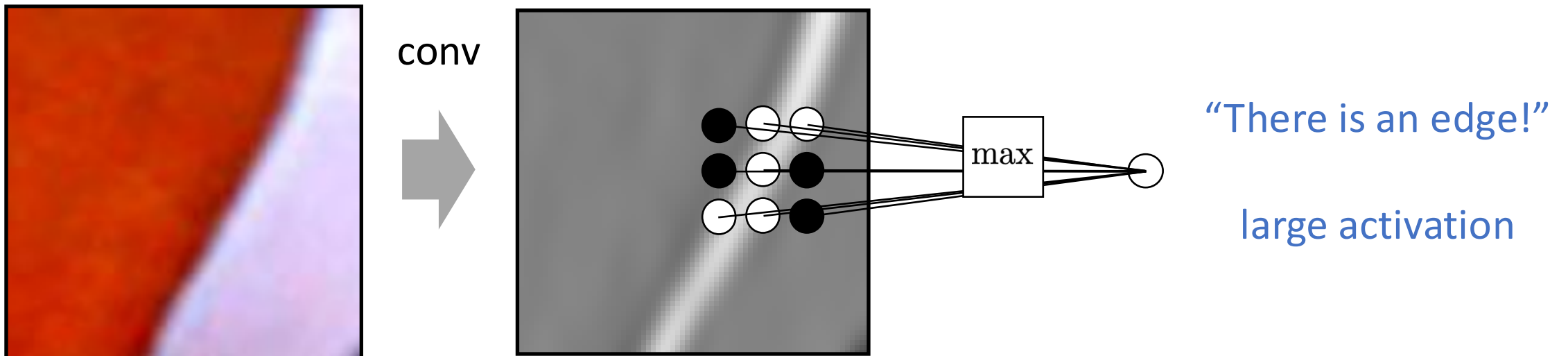
- Sliding window with:
 - mean/max aggregation
 - stride > 1
 - local translation-invariance

max pooling:

$$y[i] = \max_{j \in \mathcal{N}(i)} x[j]$$

average pooling:

$$y[i] = \frac{1}{|\mathcal{N}|} \sum_{j \in \mathcal{N}(i)} x[j]$$



Pooling

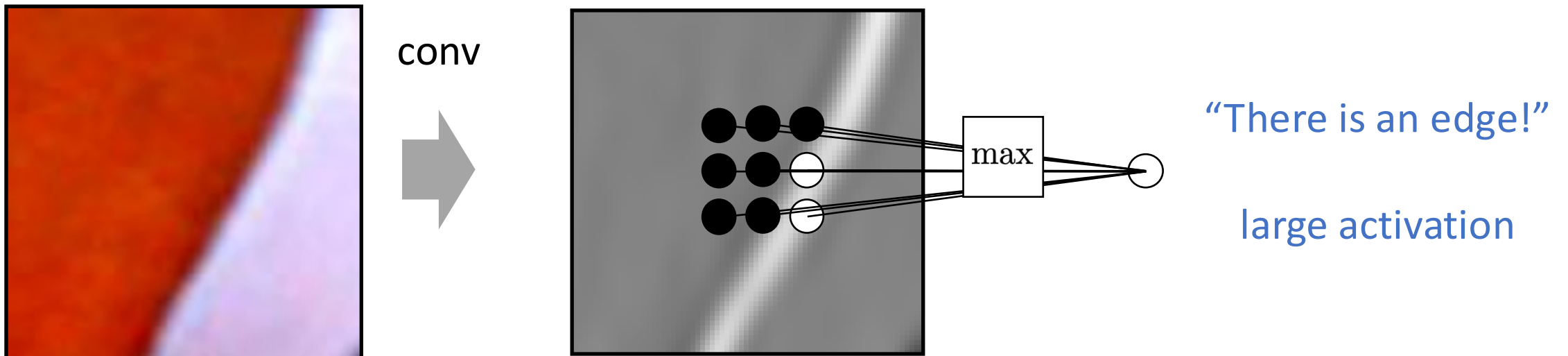
- Sliding window with:
 - mean/max aggregation
 - stride > 1
 - local translation-invariance

max pooling:

$$y[i] = \max_{j \in \mathcal{N}(i)} x[j]$$

average pooling:

$$y[i] = \frac{1}{|\mathcal{N}|} \sum_{j \in \mathcal{N}(i)} x[j]$$



Pooling

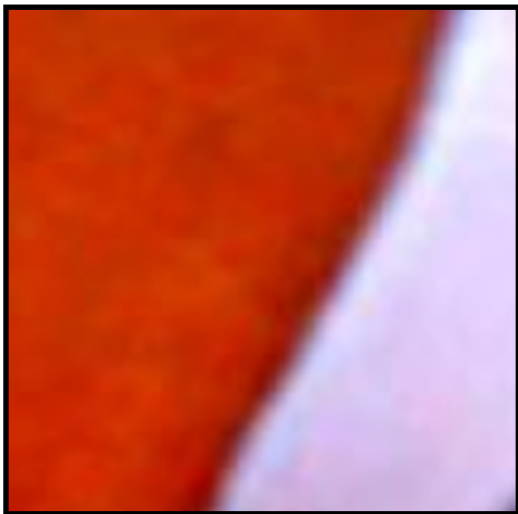
- Sliding window with:
 - mean/max aggregation
 - stride > 1
 - local translation-invariance

max pooling:

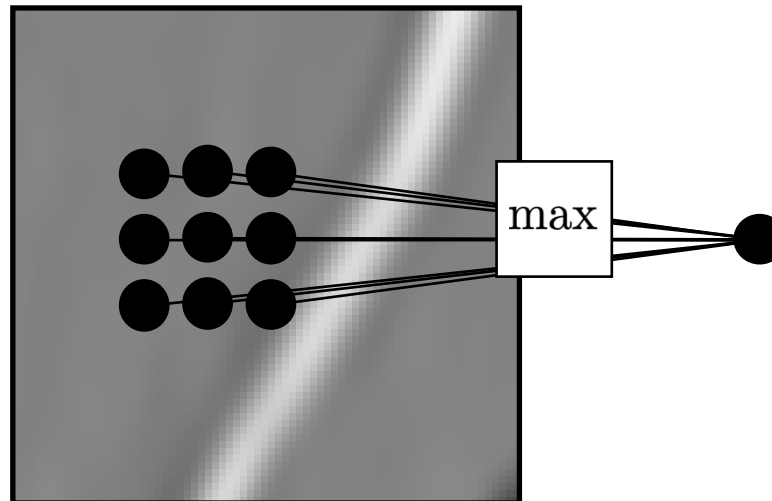
$$y[i] = \max_{j \in \mathcal{N}(i)} x[j]$$

average pooling:

$$y[i] = \frac{1}{|\mathcal{N}|} \sum_{j \in \mathcal{N}(i)} x[j]$$



conv



“No edge!”

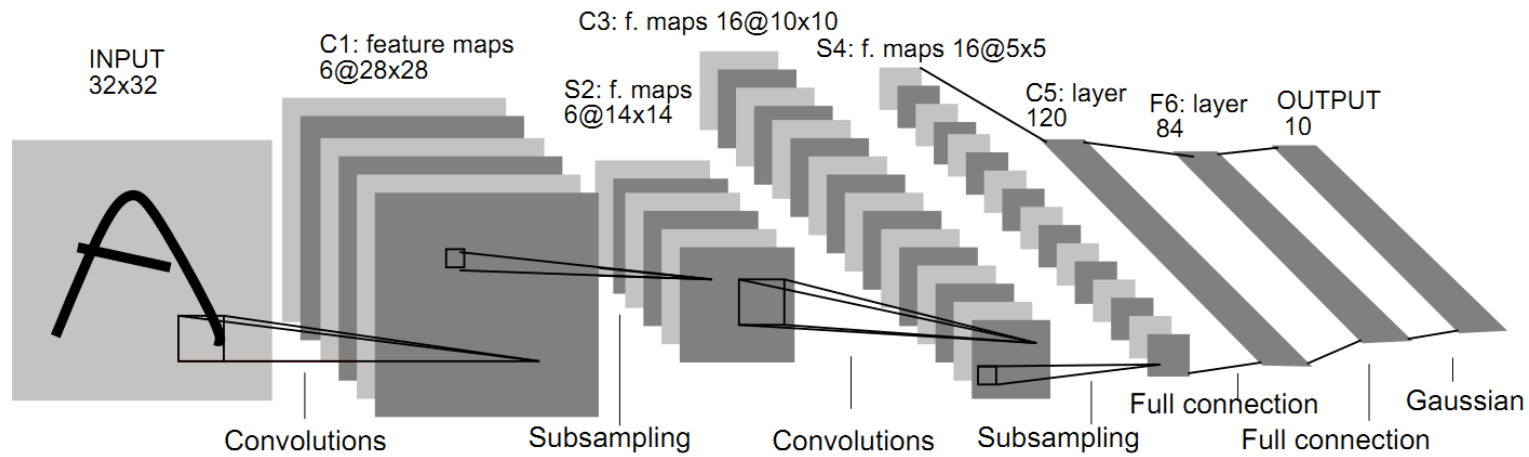
no activation

Takeaways: ConvNet

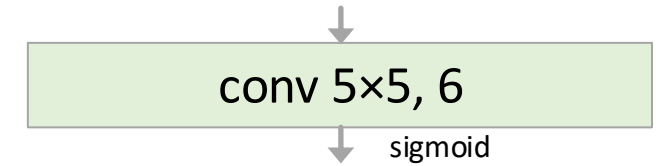
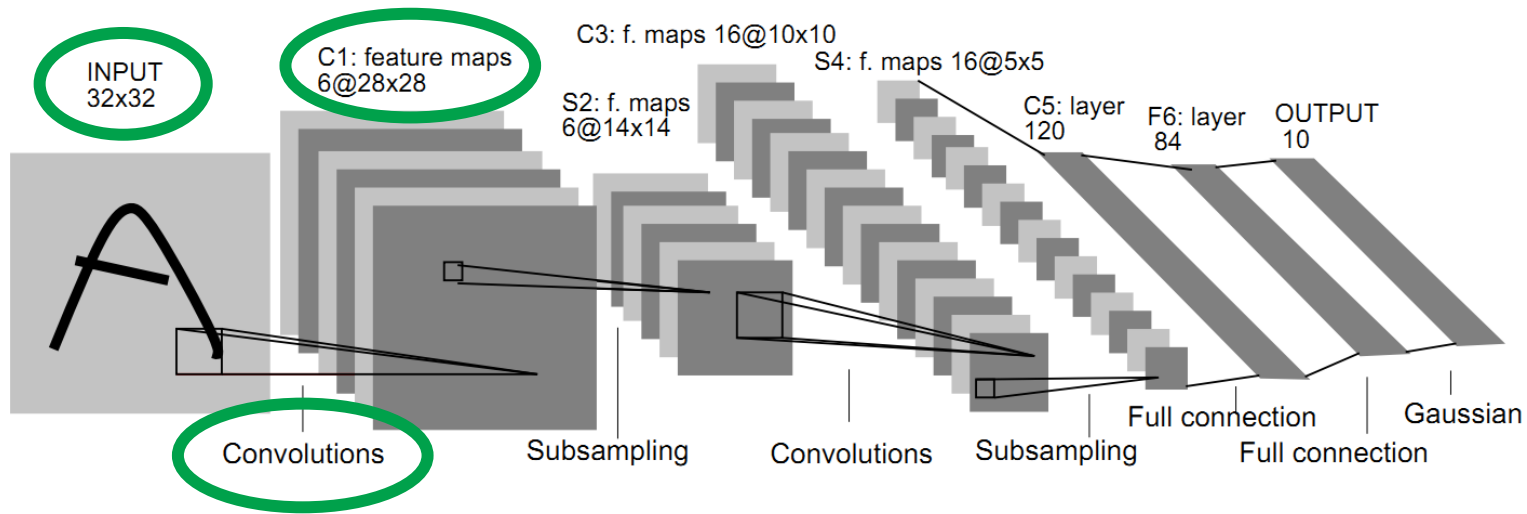
- **Composing basic operations into complex functions**
- **Conv**: local connection + weight-sharing
- **Inductive bias**: translation-equivariant

Case Study: LeNet-5

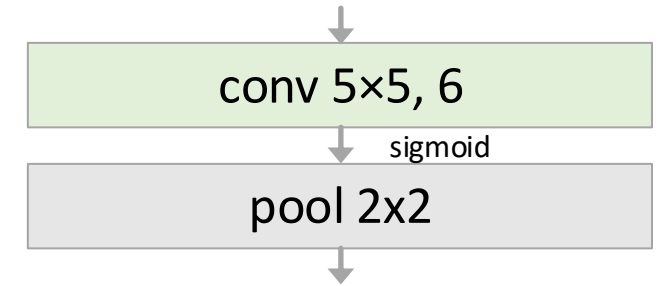
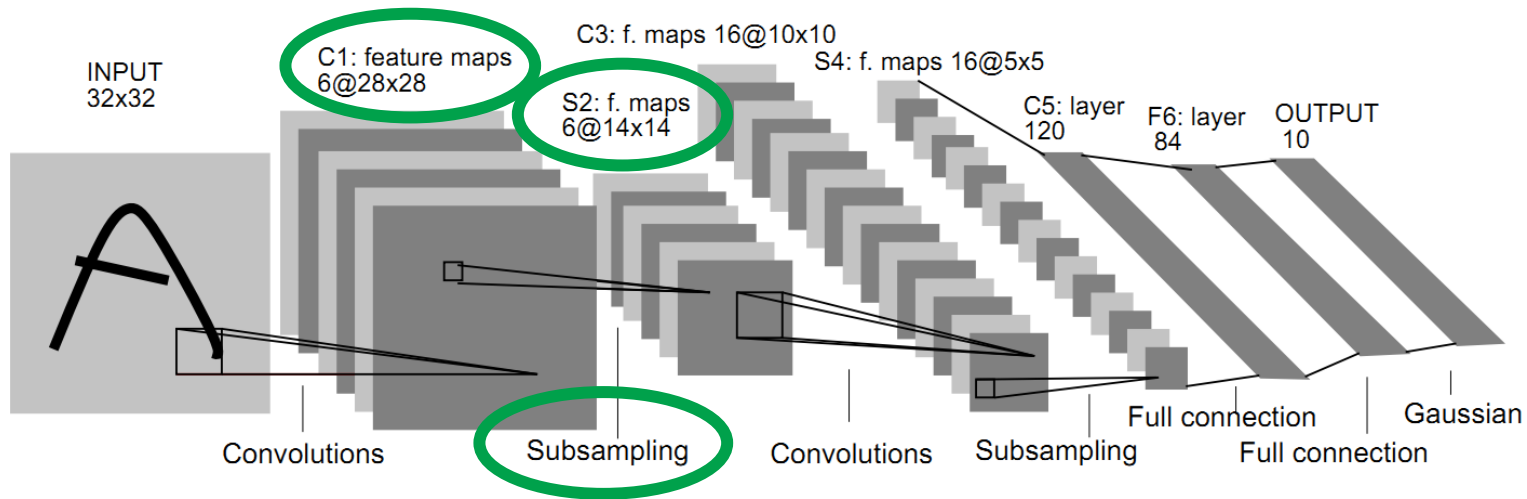
LeNet-5, 1998



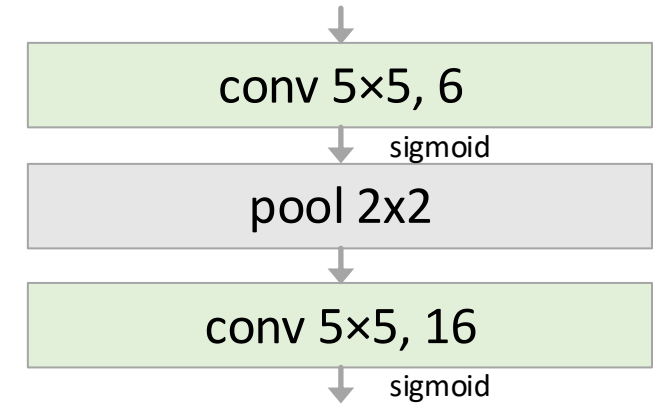
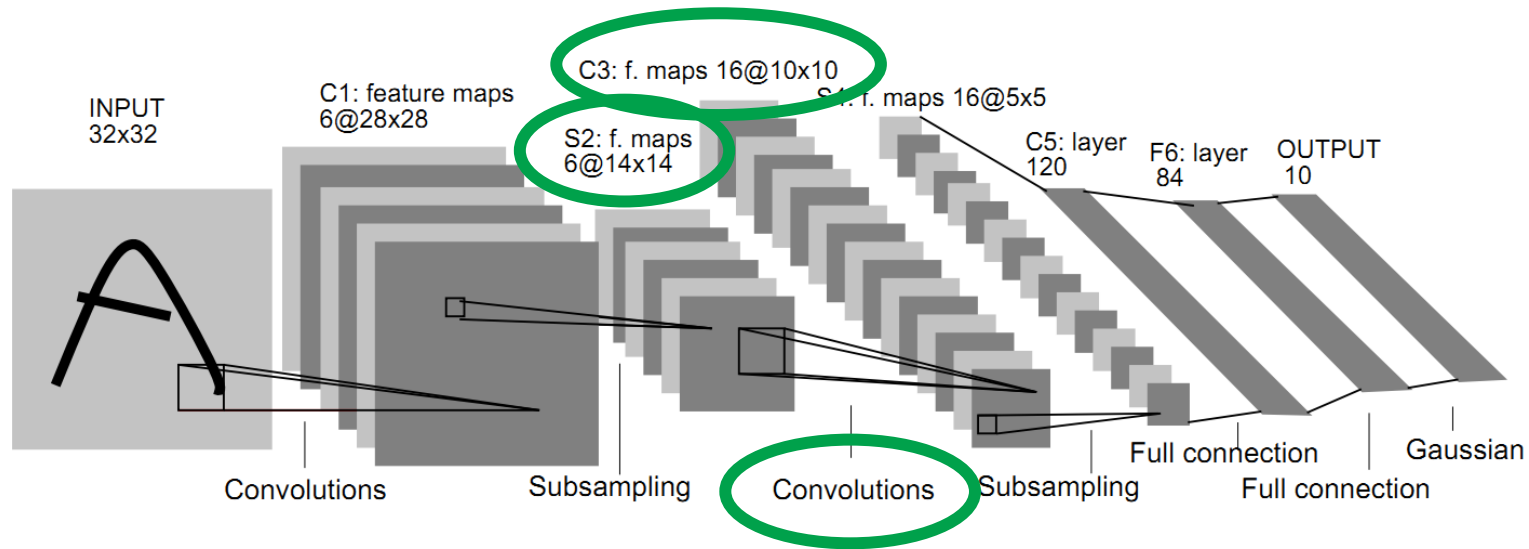
LeNet-5, 1998



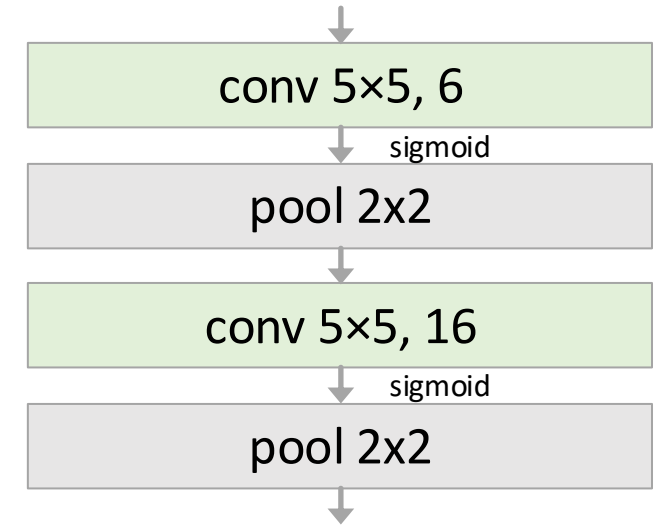
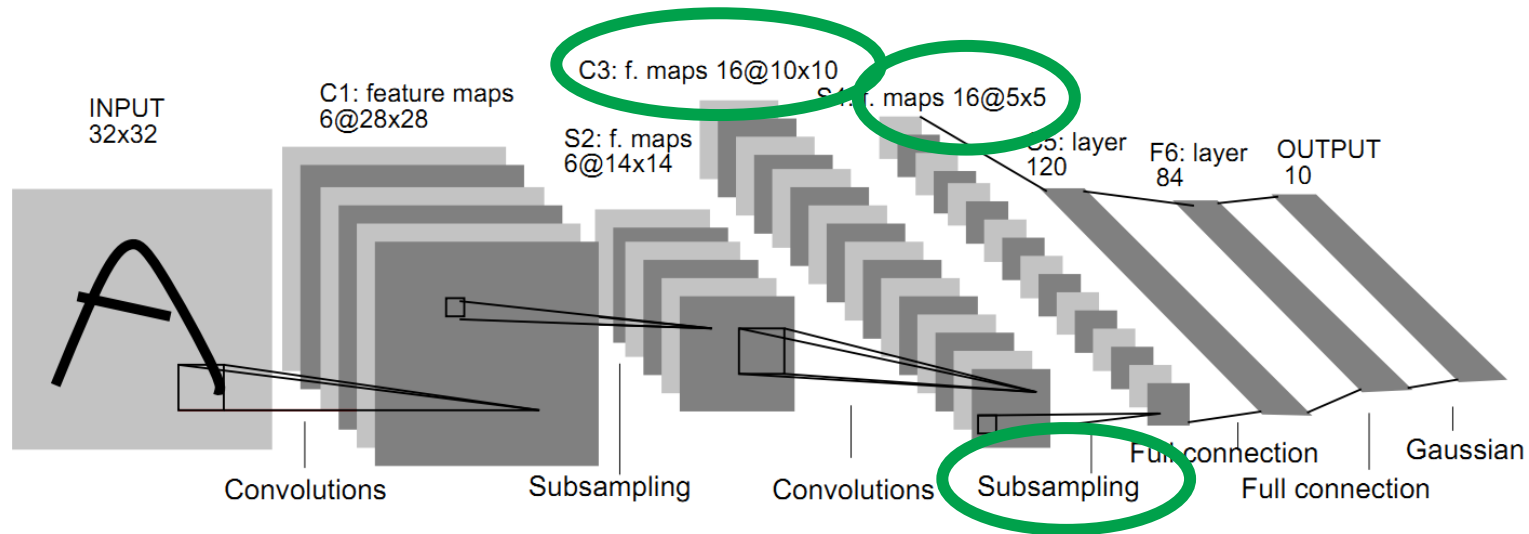
LeNet-5, 1998



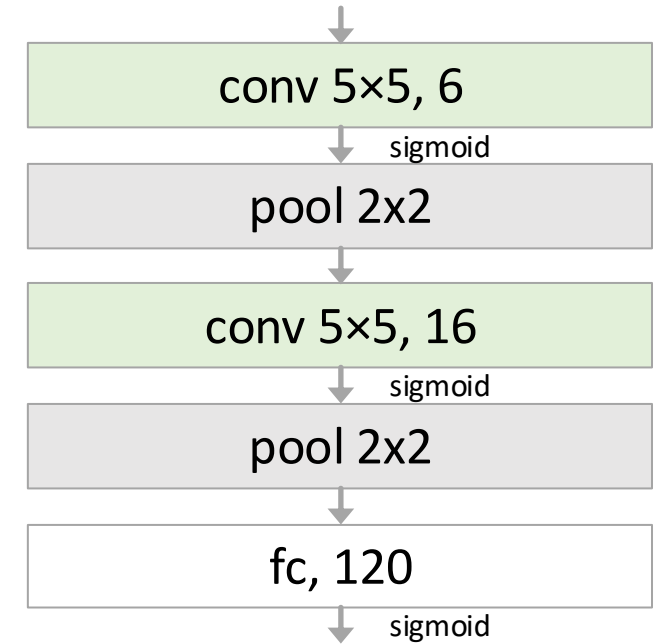
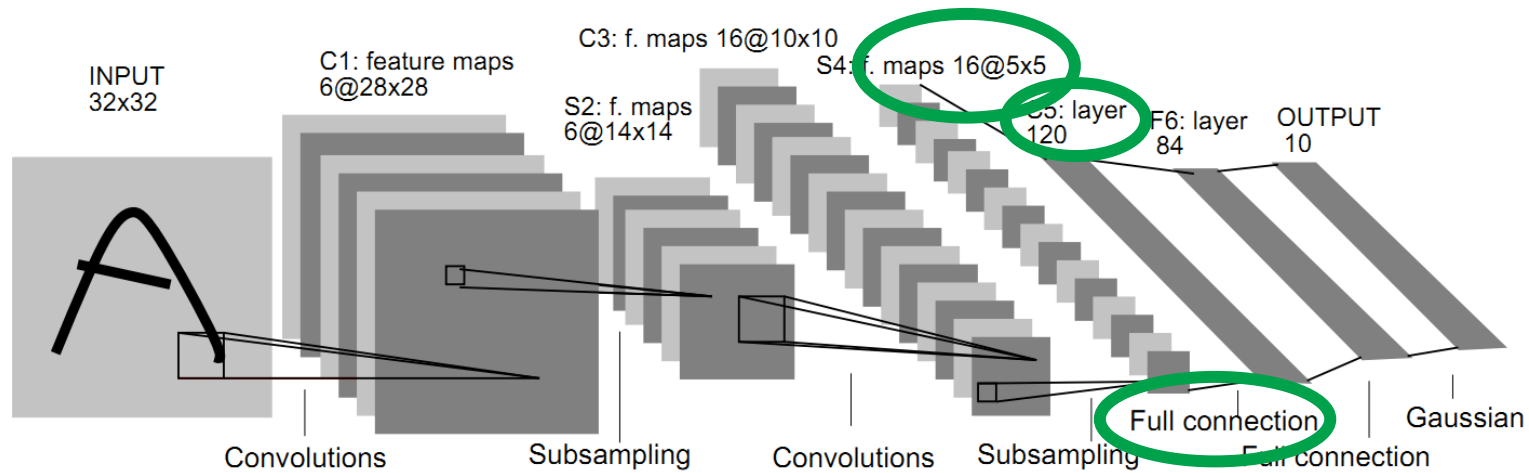
LeNet-5, 1998



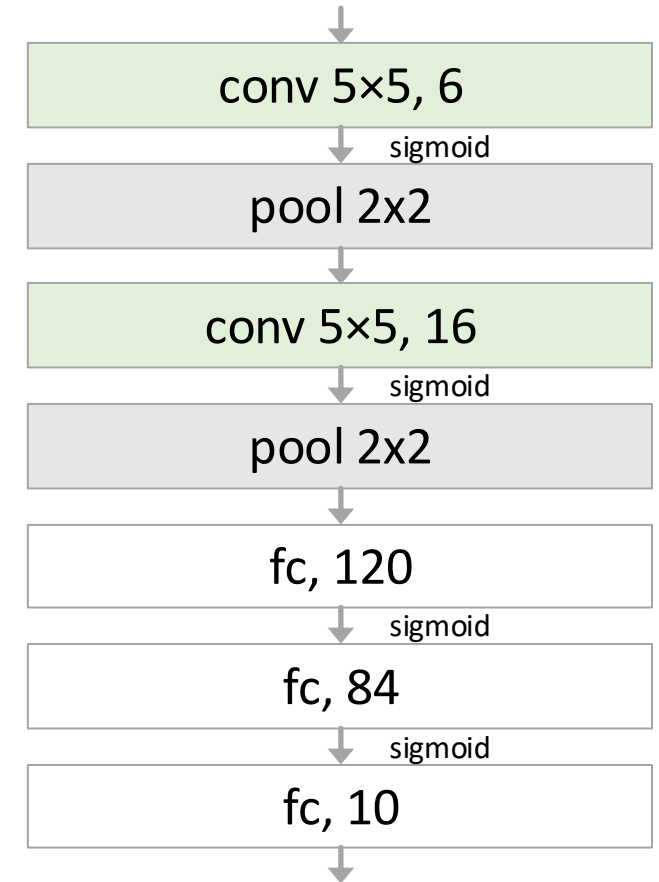
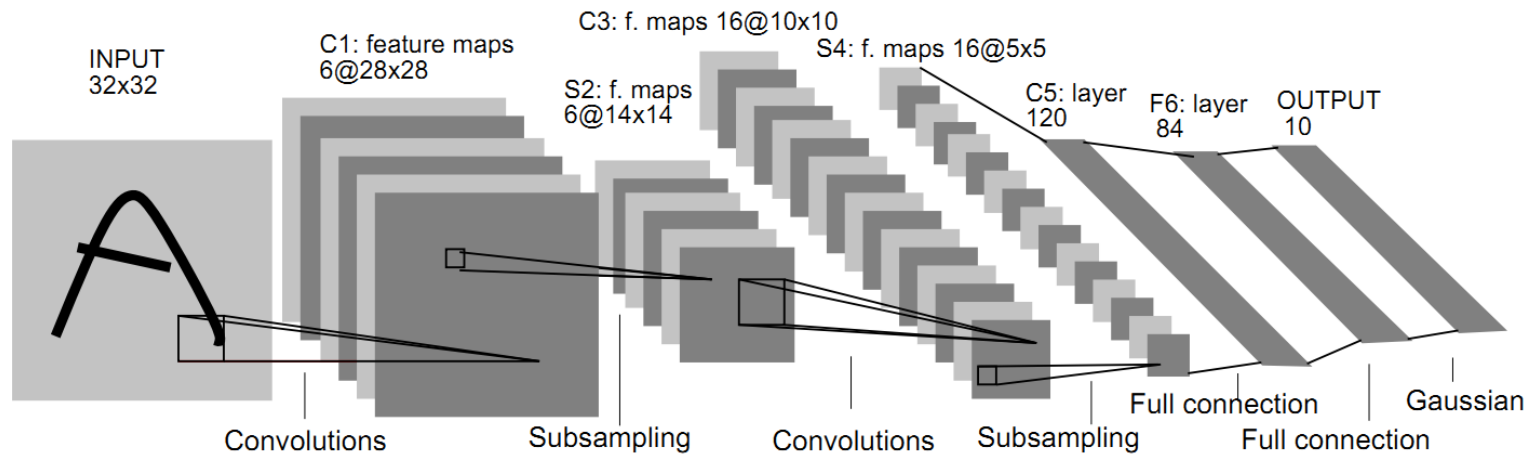
LeNet-5, 1998



LeNet-5, 1998



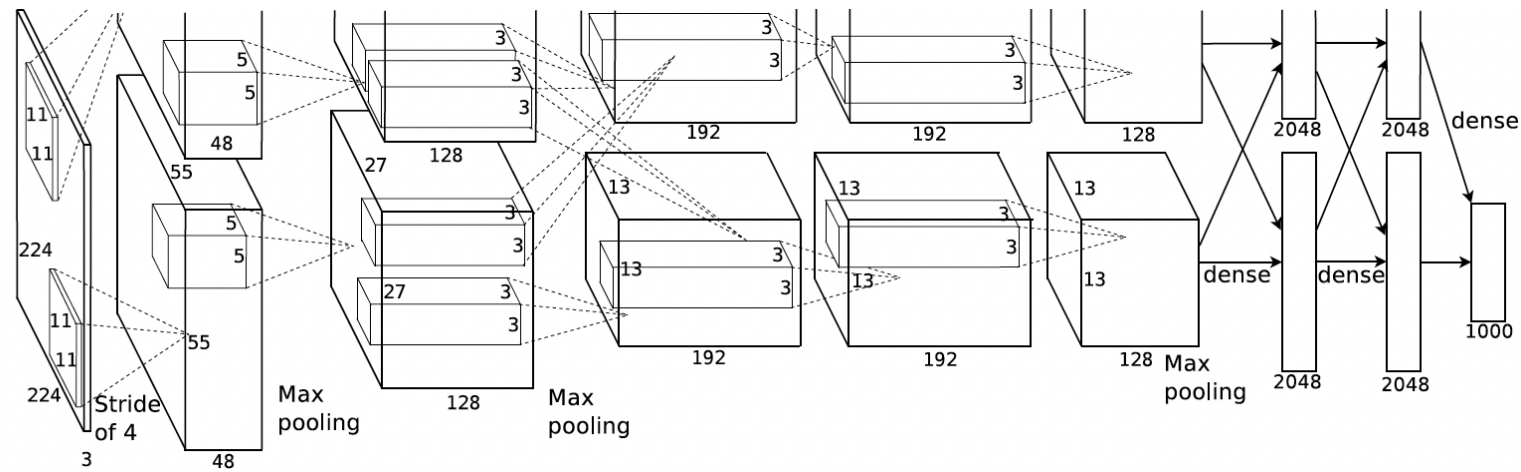
LeNet-5, 1998

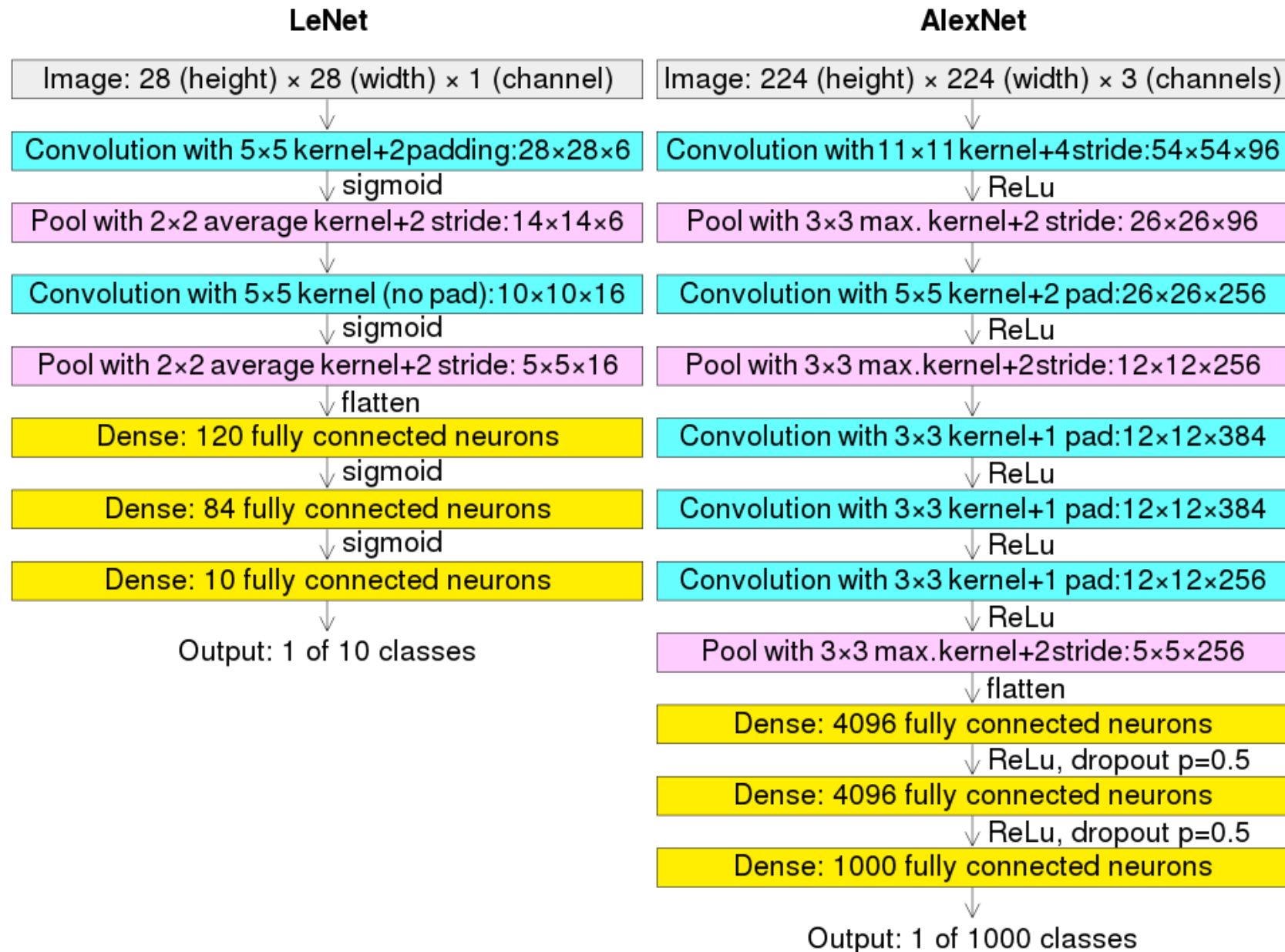


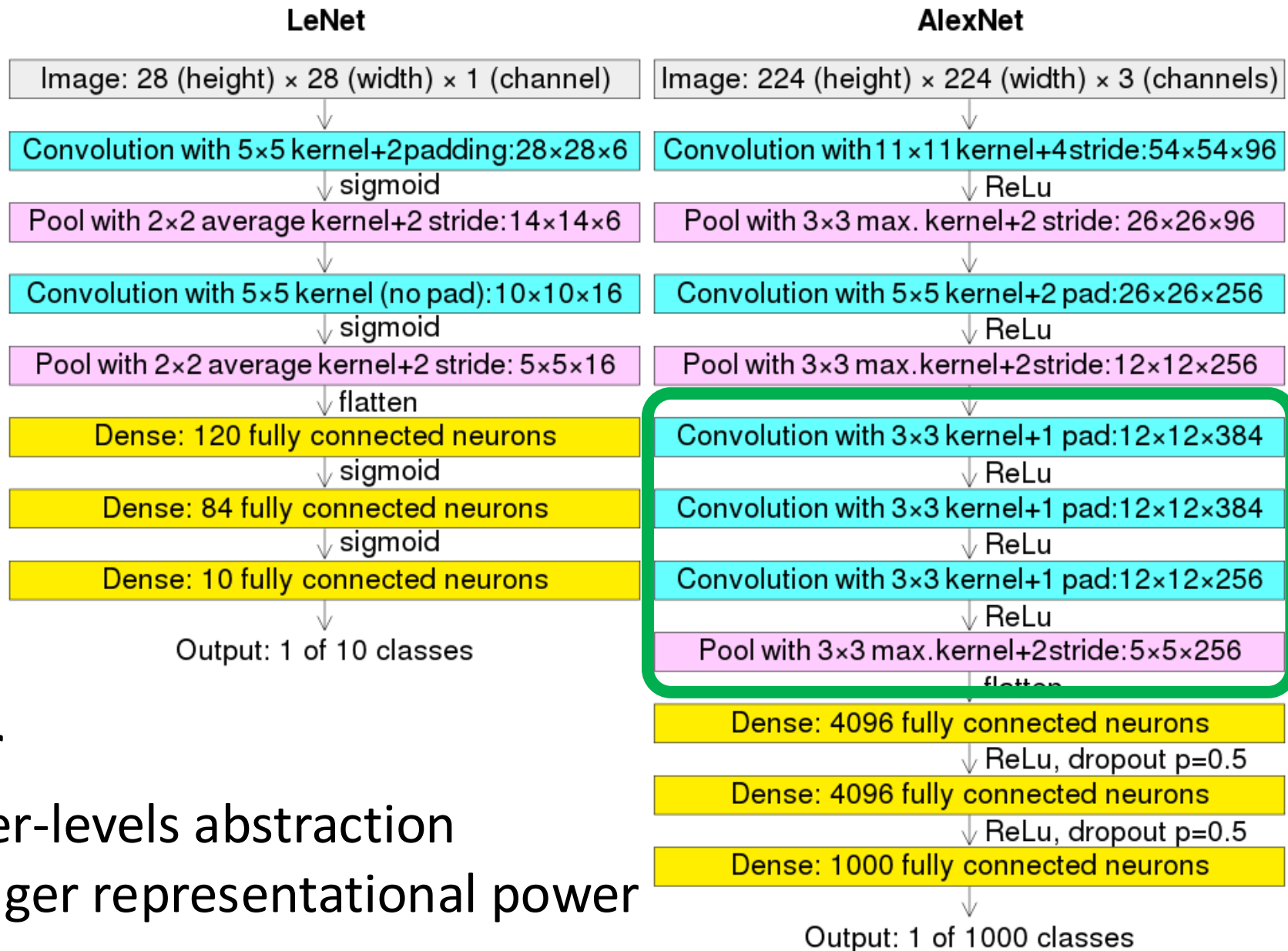
2012: AlexNet

The DL revolution

- Scaling up ConvNet
 - Data scaling
 - ImageNet, 1.28 million images, 1000 classes
 - Model scaling
 - “60 million parameters and 650,000 neurons”
 - Reducing overfitting
 - data augmentation
 - dropout
 - GPU training
 - data distribution
 - model distribution

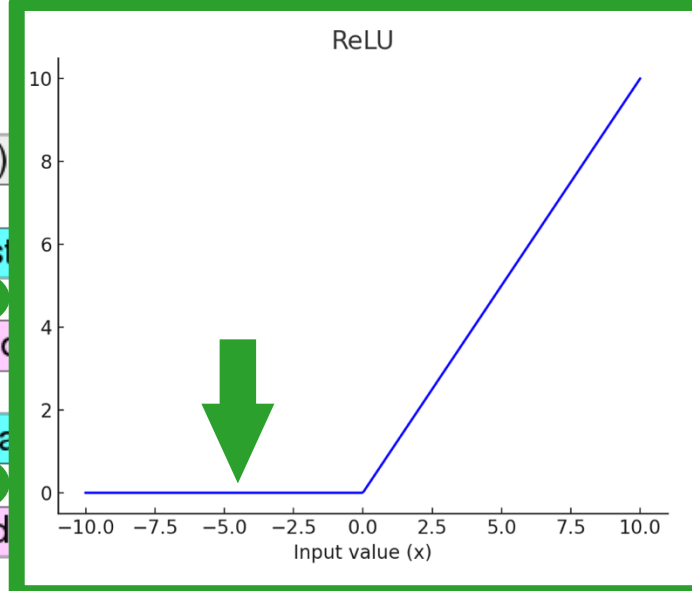
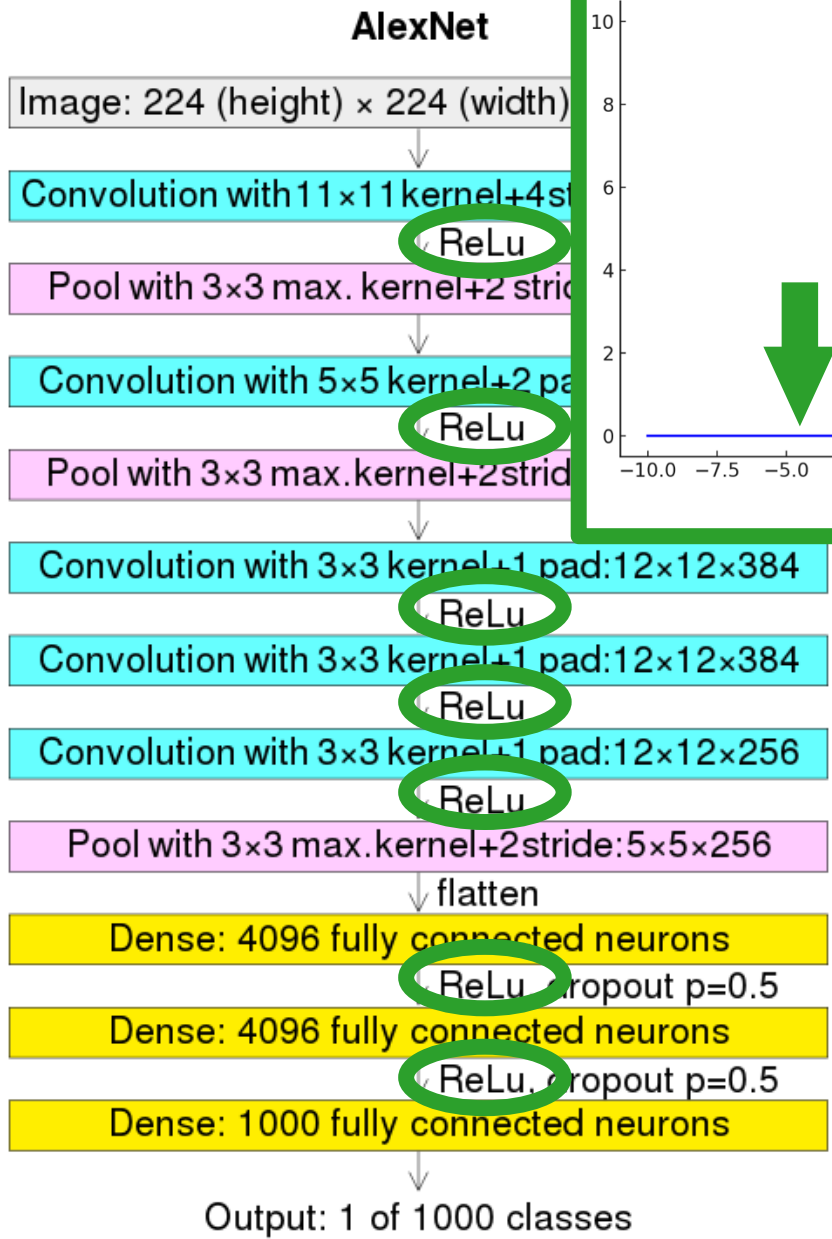
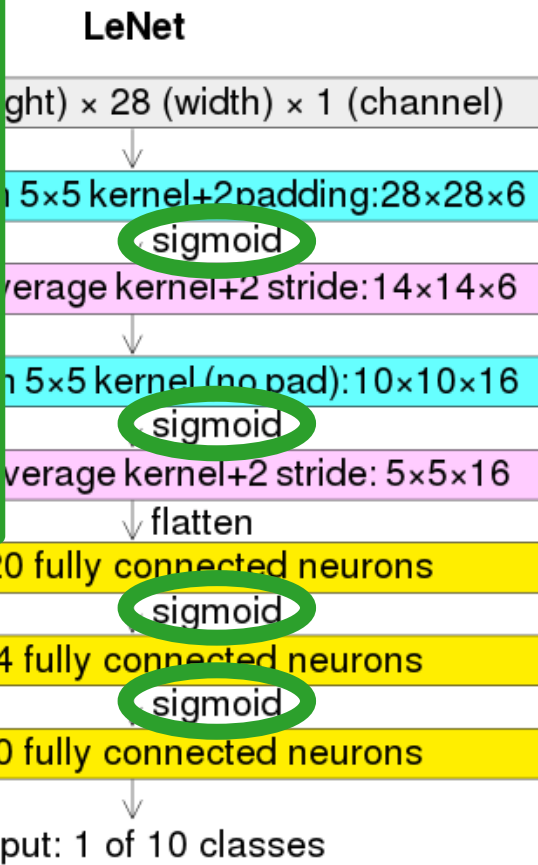
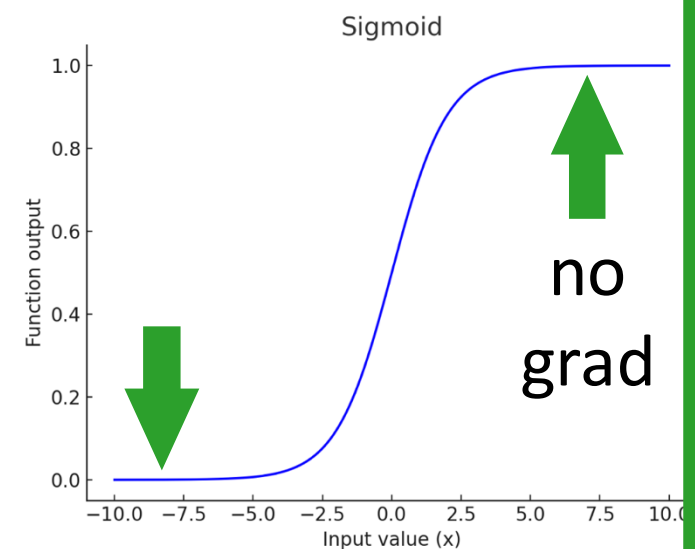






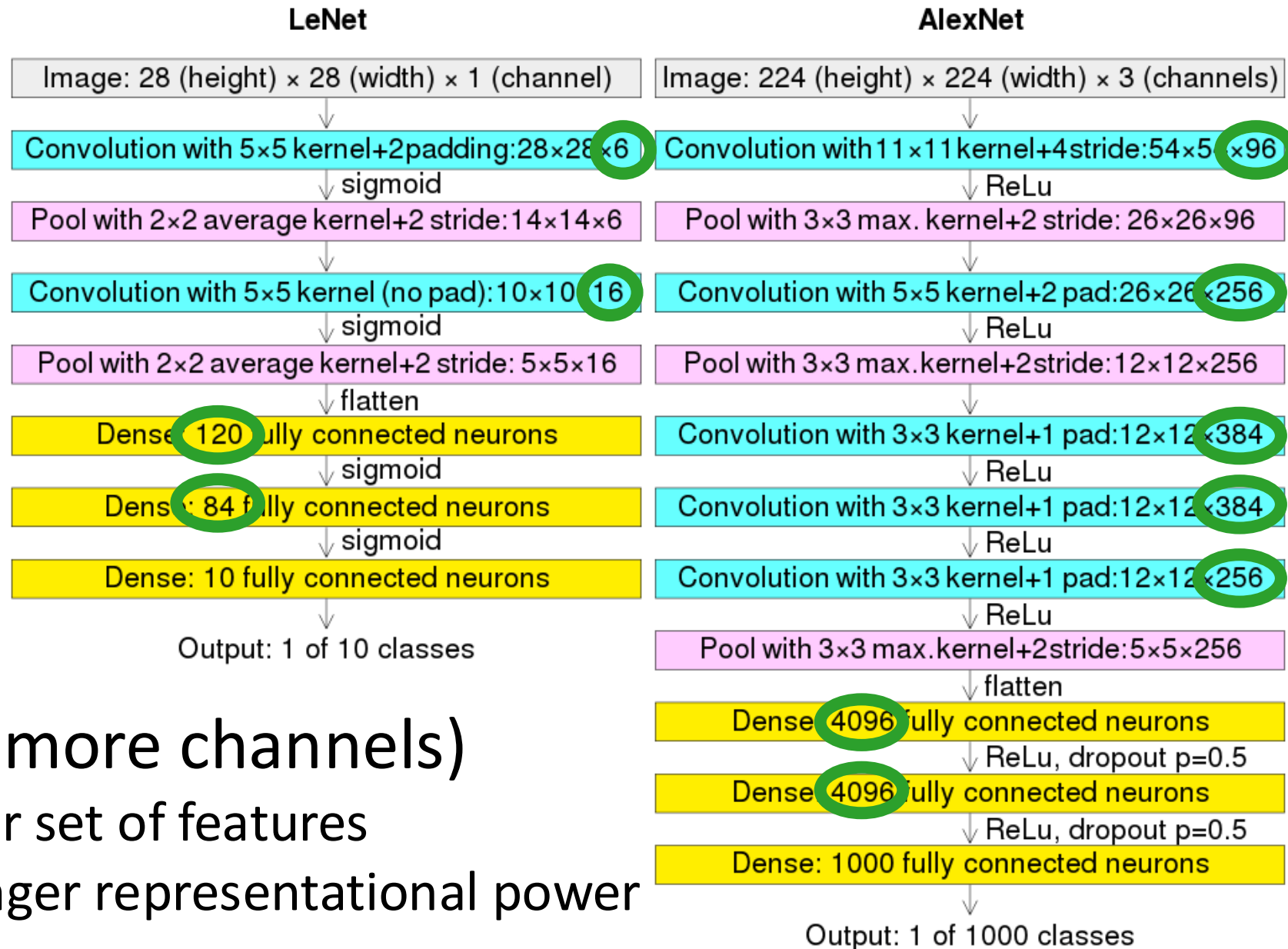
deeper

- higher-levels abstraction
- stronger representational power



Rectified Linear Unit (ReLU)

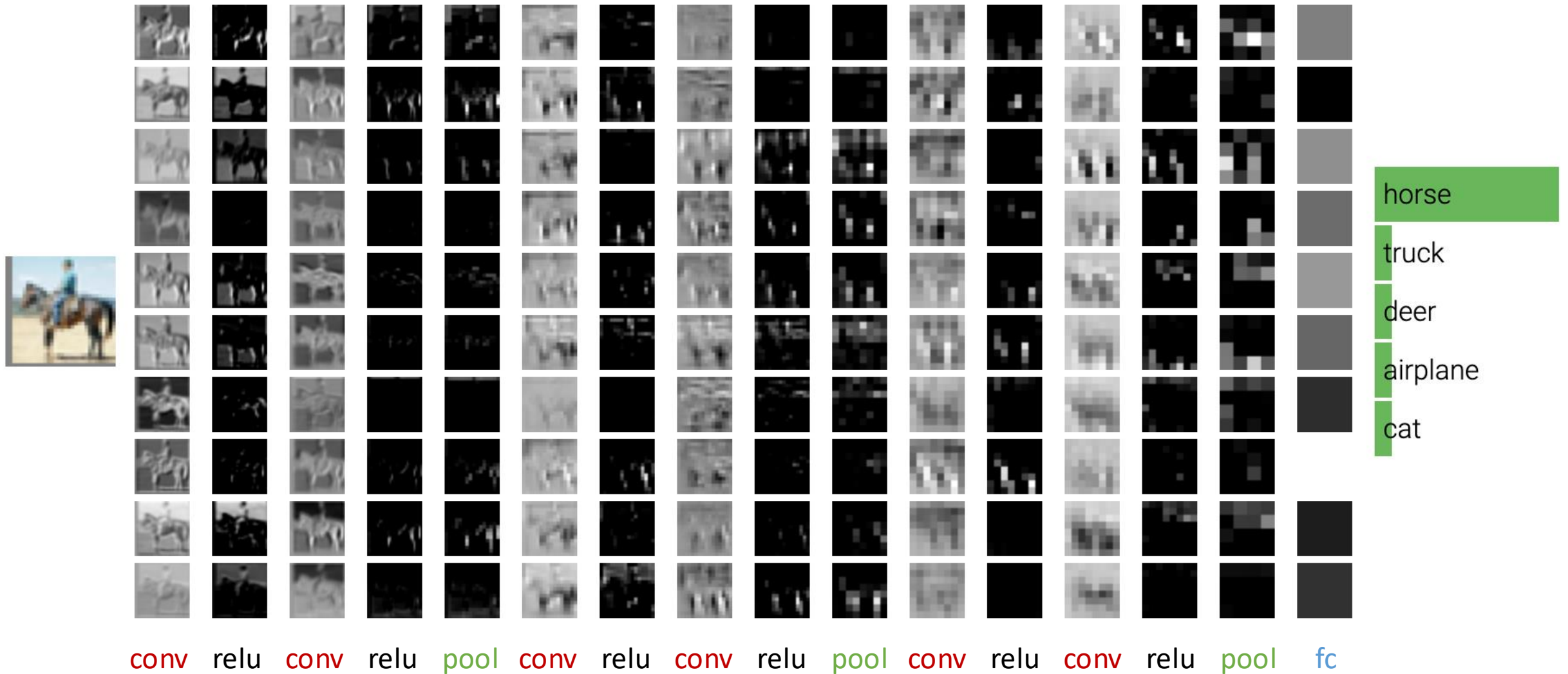
- better gradient propagation
- facilitate training deep models



wider (more channels)

- richer set of features
- stronger representational power

Visualizing feature maps of a ConvNet

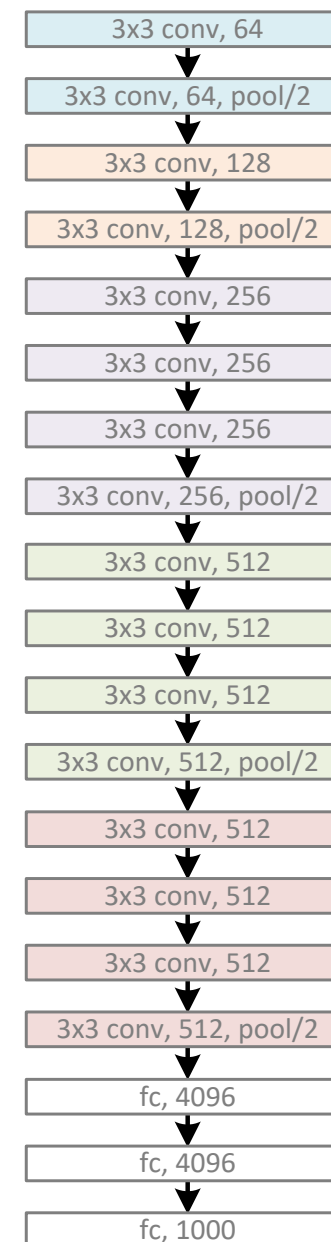


see more examples in <http://cs231n.stanford.edu/>

2014: VGG nets

Very deep ConvNets

- simple, highly modularized design
- conv: only 3x3
- stack the same modules
- up to 19 layers
- clear evidence: **deeper is better**



VGG nets: deeper is better

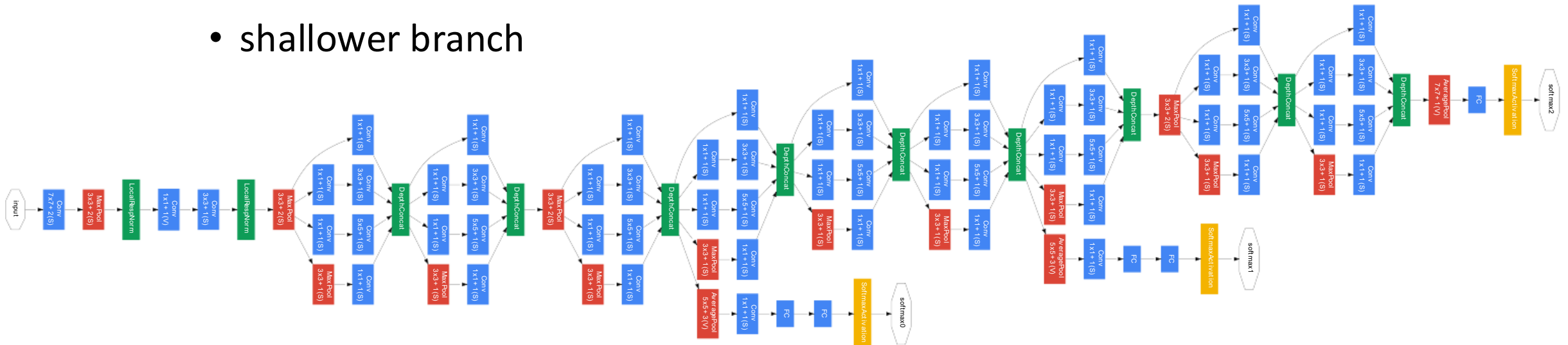
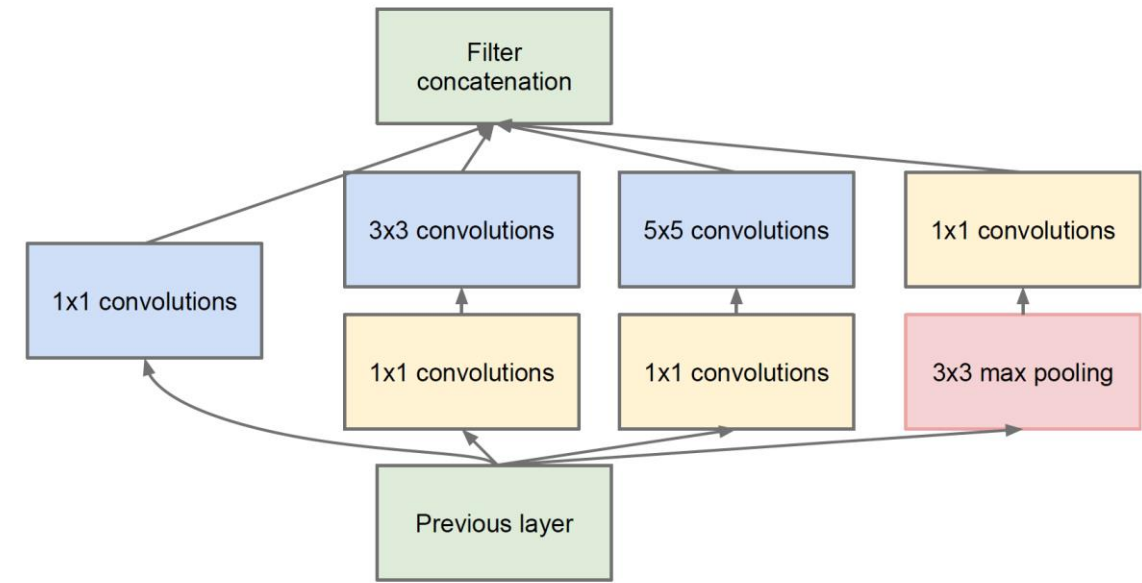
- previous: little clear evidence
- controlled experiments
 - remove bells-and-whistles
 - little “config engineering”
 - simple config of # channels
 - add 3x3 conv
- simple but hard-to-beat
 - before deeper nets come along

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
		input (224 × 224 RGB image)			
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
		max pool			
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
		max pool			
conv3-256	conv3-256	conv3-256	conv3-256	conv3-256	conv3-256
conv3-256	conv3-256	conv3-256	conv3-256 conv1-256	conv3-256 conv3-256	conv3-256 conv3-256 conv3-256
		max pool			
conv3-512	conv3-512	conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512 conv1-512	conv3-512	conv3-512 conv3-512
		max pool			
conv3-512	conv3-512	conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512 conv1-512	conv3-512	conv3-512 conv3-512
		max pool			
		max pool			
		FC-4096			
		FC-4096			
		FC-1000			
		soft-max			

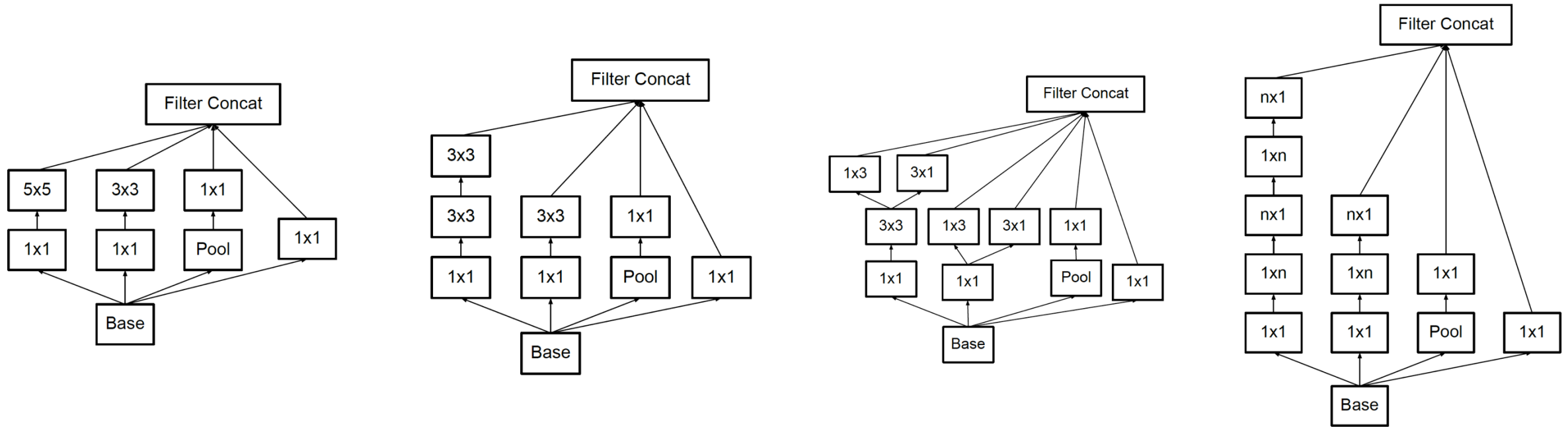
2014: GoogLeNet / Inception

Deep and economical ConvNets

- multiple branches
 - 1x1, 3x3, 5x5, pool
- 1x1 bottleneck
 - dimension reduction
- 1x1 shortcut
 - shallower branch



Inception v1, v2, v3, ...

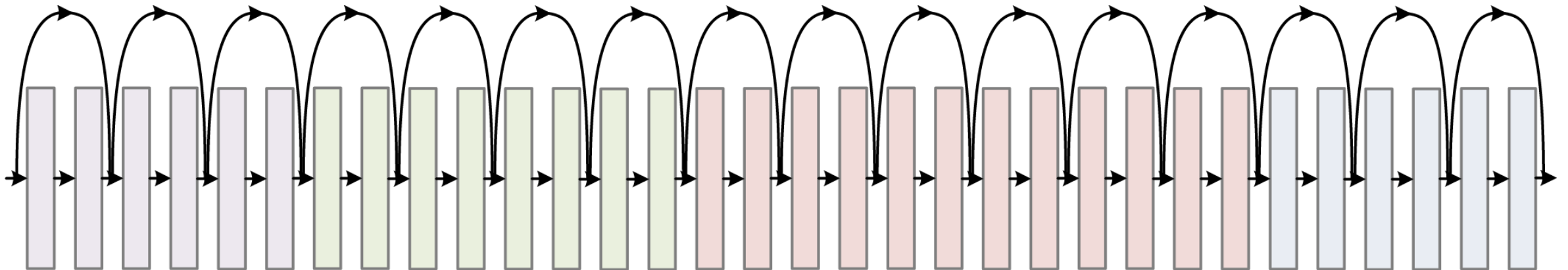


- rich design space for compute/accuracy trade-off

2015: ResNet

Deep Learning gets way deeper

- identity shortcut
- enabling 100's of layers
- beyond ConvNet
- more in Lecture 8



Summary

- Convolution
- Convolutional Neural Networks
- Core takeaways:
 - local connection + weight-sharing